NOSQL

CHEAT SHEETS

MongoDB, Redis, DynamoDB, Cassandra, Neo4j, search and vectors, with SQL equivalents

Document · Key-Value · Wide-Column · Graph · CAP · Modeling · mongosh · CRUD · Aggregation · Indexes · Transactions · Change Streams · Sharding · Redis Types · Caching · Streams · DynamoDB Keys · Single-Table · CQL · Cypher · Search · Vectors · Polyglot · Testing

61 REFERENCE SHEETS

CONTENTS

CONCEPTS & CHOOSING
01 What Is NoSQL?
02 SQL vs NoSQL: When to Use Which
03 CAP, Consistency & BASE
04 Modeling From Access Patterns
05 Embedding vs Referencing
06 Choosing an Engine

MONGODB BASICS
07 MongoDB Introduction & Setup
08 Documents, BSON & Types
09 Databases, Collections & _id
10 mongosh Essentials

MONGODB CRUD
11 Inserting Documents
12 Finding Documents
13 Query Operators
14 Querying Arrays & Nested Fields
15 Updating Documents
16 Delete, Replace, Upsert & Bulk

MONGODB AGGREGATION
17 Aggregation Pipeline Basics
18 $group & Accumulators
19 $lookup & $unwind
20 $facet, $bucket & Window Stages
21 Expression Operators & Output

MONGODB INDEXING & SEARCH
22 Index Types
23 Index Strategy & explain()
24 Text, Atlas Search & Vector Search

MONGODB SCHEMA & INTEGRITY
25 Schema Design Patterns
26 Schema Validation
27 Transactions, Read & Write Concerns
28 Change Streams

MONGODB OPERATIONS & DRIVERS
29 Replica Sets & Sharding
30 Security & Backup
31 Drivers & ODMs

REDIS
32 Redis Introduction & Setup
33 Keys & Strings
34 Expiry & Eviction
35 Hashes
36 Lists & Sets
37 Sorted Sets
38 Streams & Pub/Sub
39 Caching Patterns
40 JSON, Search & Vectors
41 Transactions, Lua & Operations

AMAZON DYNAMODB
42 DynamoDB Concepts
43 Item Operations
44 Query vs Scan
45 Secondary Indexes
46 Single-Table Design
47 Streams, TTL & Transactions

APACHE CASSANDRA
48 Cassandra Architecture
49 CQL: Keyspaces & Tables
50 Query-First Modeling
51 Writes, TTL & Operations

GRAPH DATABASES — NEO4J
52 The Graph Model & Neo4j
53 Cypher Basics
54 Paths, Patterns & Aggregation
55 Graph Modeling, Indexes & Constraints

SEARCH & VECTOR DATABASES
56 Search Engines: Elasticsearch & OpenSearch
57 Vector Databases & Embeddings

NOSQL IN PRACTICE
58 Polyglot Persistence
59 Testing & Schema Migration

QUICK REFERENCE
60 Quick Reference — MongoDB & Redis
61 Quick Reference — DynamoDB, Cassandra, Cypher & SQL Map

NOSQL

Table of Contents

61 sheets · syntax · examples · notes

CONCEPTS & CHOOSING 01 – 06

- 01 What Is NoSQL?
- 02 SQL vs NoSQL: When t…
- 03 CAP, Consistency & BA…
- 04 Modeling From Access…
- 05 Embedding vs Referen…
- 06 Choosing an Engine

MONGODB BASICS 07 – 10

- 07 MongoDB Introduction…
- 08 Documents, BSON & T…
- 09 Databases, Collections…
- 10 mongosh Essentials

MONGODB CRUD 11 – 16

- 11 Inserting Documents
- 12 Finding Documents
- 13 Query Operators
- 14 Querying Arrays & Nes…
- 15 Updating Documents
- 16 Delete, Replace, Upser…

MONGODB AGGREGATION 17 – 21

- 17 Aggregation Pipeline B…
- 18 $group & Accumulators
- 19 $lookup & $unwind
- 20 $facet, $bucket & Win…
- 21 Expression Operators …

MONGODB INDEXING & SEARCH 22 – 24

- 22 Index Types
- 23 Index Strategy & expla…
- 24 Text, Atlas Search & Ve…

MONGODB SCHEMA & INTEGRITY 25 – 28

- 25 Schema Design Patter…
- 26 Schema Validation
- 27 Transactions, Read & …
- 28 Change Streams

MONGODB OPERATIONS & DRIVERS 29 – 31

- 29 Replica Sets & Sharding
- 30 Security & Backup
- 31 Drivers & ODMs

REDIS 32 – 41

- 32 Redis Introduction & S…
- 33 Keys & Strings
- 34 Expiry & Eviction
- 35 Hashes
- 36 Lists & Sets
- 37 Sorted Sets
- 38 Streams & Pub/Sub
- 39 Caching Patterns
- 40 JSON, Search & Vectors
- 41 Transactions, Lua & O…

AMAZON DYNAMODB 42 – 47

- 42 DynamoDB Concepts
- 43 Item Operations
- 44 Query vs Scan
- 45 Secondary Indexes
- 46 Single-Table Design
- 47 Streams, TTL & Transa…

APACHE CASSANDRA 48 – 51

- 48 Cassandra Architecture
- 49 CQL: Keyspaces & Tabl…
- 50 Query-First Modeling
- 51 Writes, TTL & Operatio…

GRAPH DATABASES — NEO4J 52 – 55

- 52 The Graph Model & Ne…
- 53 Cypher Basics
- 54 Paths, Patterns & Aggr…
- 55 Graph Modeling, Index…

SEARCH & VECTOR DATABASES 56 – 57

- 56 Search Engines: Elastic…
- 57 Vector Databases & E…

NOSQL IN PRACTICE 58 – 59

- 58 Polyglot Persistence
- 59 Testing & Schema Migr…

QUICK REFERENCE 60 – 61

- 60 Quick Reference — Mo…
- 61 Quick Reference — Dy…

Sheet 01

What Is NoSQL?

non-relational · document · key-value · wide-column · graph · search / vector · schema-flexible

DATABASES BEYOND TABLES

NoSQL ("not only SQL") is the family name for databases that don't store data as rows in fixed relational tables. The main kinds are **document** stores (MongoDB), **key-value** stores (Redis, DynamoDB), **wide-column** stores (Cassandra) and **graph** databases (Neo4j), plus **search** and **vector** engines. Each gives up some relational features, such as joins, a fixed schema or multi-table transactions, in exchange for a flexible schema, easy horizontal scale or a data model that matches how the app reads.

NOTES

- Schema-flexible, not schemaless: the app still expects a shape
- Several now offer ACID transactions and SQL-like languages (CQL, PartiQL)
- PostgreSQL `jsonb` blurs the line from the SQL side

ONE ORDER IN EACH MODEL

```
// Document (MongoDB): the order with its items embedded
{ _id: 1001, customerId: 42, status: "pending", total: 49.97,
  items: [ { productId: 7, qty: 2, unitPrice: 19.99 },
           { productId: 3, qty: 1, unitPrice: 9.99 } ] }

// Key-value (Redis): a key points at a value or structure
HSET order:1001 customerId 42 status pending total 49.97
RPUSH order:1001:items "7:2:19.99" "3:1:9.99"

// Key-value item (DynamoDB): partition key + sort key
{ "PK": "CUSTOMER#42", "SK": "ORDER#2026-11-01#1001",
  "status": "pending", "total": 49.97 }

// Wide-column (Cassandra CQL): rows grouped into partitions
CREATE TABLE orders_by_customer (
  customer_id int, order_date date, order_id int,
  status text, total decimal,
  PRIMARY KEY ((customer_id), order_date, order_id));

// Graph (Neo4j Cypher): nodes joined by stored relationships
(:Customer {id: 42})-[:PLACED]->(:Order {id: 1001})
  -[:CONTAINS {qty: 2}]->(:Product {id: 7})
```

CODE EXPLAINED

- → A document keeps related data together, so one read returns the whole order
- → Key-value stores look up by key only; the key name carries the structure
- → DynamoDB groups items by partition key and sorts them by sort key
- → Cassandra's partition key picks the node; clustering columns sort rows inside it
- → A graph stores relationships as data, so following them is cheap

SQL EQUIVALENT

- ⇄ In SQL the same order is two tables, `orders` and `order_items`, joined when read
- ⇄ SQL normalizes first and answers any question later; most NoSQL designs start from the questions · See SQL 4

ACROSS ENGINES

- **MongoDB** documents (BSON) in collections; rich queries and secondary indexes
- **Redis** in-memory data structures by key; sub-millisecond reads
- **DynamoDB** managed key-value and document store on AWS; you design the keys
- **Cassandra** wide-column rows across many equal nodes; built for heavy writes
- **Neo4j** nodes and relationships; traversals instead of joins

IN YOUR CODE

- **Node** `mongodb` · `redis` · `@aws-sdk/lib-dynamodb` · `cassandra-driver` · `neo4j-driver`
- **Python** `pymongo` · `redis` · `boto3` · `cassandra-driver` · `neo4j`
- **Java** `mongodb-driver-sync` · `jedis` · `dynamodb-enhanced` · `java-driver-core` · `neo4j-java-driver`

FAMILIES

FAMILY	DATA UNIT	EXAMPLES
Document	JSON-like document	MongoDB, Couchbase, Firestore
Key-value	Key → value / structure	Redis, Valkey, DynamoDB
Wide-column	Rows in partitions	Cassandra, ScyllaDB, Bigtable
Graph	Nodes + relationships	Neo4j, Neptune
Search	Inverted index	Elasticsearch, OpenSearch
Vector	Embeddings	Pinecone, Qdrant, pgvector

WHAT YOU TRADE

YOU GAIN	YOU USUALLY GIVE UP
Flexible schema	Structure enforced by the database
Horizontal scale	Joins and transactions across partitions
Model fits the access pattern	Easy ad hoc queries
Very fast key lookups	Rich aggregation (key-value stores)

⚠ COMMON MISTAKES

- Choosing NoSQL "because SQL doesn't scale" without a measured need
- Reading "schemaless" as "no design needed"
- Using one engine for every workload

✓ TIPS & BEST PRACTICES

- Start from the access patterns (sheet 04)
- Keep PostgreSQL in mind as the baseline to beat
- Learn on the managed version of each engine

QUICK SUMMARY

- Document — MongoDB
- Key-value — Redis, DynamoDB
- Wide-column — Cassandra
- Graph — Neo4j

Sheet 02

SQL vs NoSQL: When to Use Which

access patterns · joins vs embedding · scale-out · schema flexibility · transactions · decision table

PICK BY ACCESS PATTERN, NOT HYPE

SQL databases shine when data is relational, questions are unpredictable and integrity matters: joins, constraints and multi-row transactions come built in. **NoSQL engines** shine when the access patterns are known and narrow, the data is naturally nested or connected, or the load has to spread across many machines. NoSQL reads are fast because the data was arranged ahead of time for that exact question. Many real systems use both (sheet 58).

NOTES

- MongoDB, DynamoDB and Neo4j all support ACID transactions now
- Most "scale problems" are missing indexes or bad schemas first
- Team skills and operations cost matter as much as features

SAME QUESTION, THREE DESIGNS

```
// Question: a customer's 10 newest orders with line counts

// SQL: two tables, joined and grouped at read time
SELECT o.order_id, o.order_date, o.total, COUNT(*) AS lines
FROM orders o JOIN order_items i USING (order_id)
WHERE o.customer_id = 42
GROUP BY o.order_id
ORDER BY o.order_date DESC LIMIT 10;

// MongoDB: items embedded, one indexed read, no join
db.orders.find({ customerId: 42 },
               { orderDate: 1, total: 1, lines: { $size: "$items" } })
  .sort({ orderDate: -1 }).limit(10)

// DynamoDB: one Query on the customer's partition
aws dynamodb query --table-name Shop \
  --key-condition-expression "PK = :c AND begins_with(SK, :o)" \
  --expression-attribute-values \
    '{":c":{"S":"CUSTOMER#42"},":o":{"S":"ORDER#"}}' \
  --no-scan-index-forward --limit 10
```

CODE EXPLAINED

- → SQL builds the answer's shape at read time from normalized tables
- → MongoDB stores the shape: the line items are already inside the order
- → The DynamoDB sort key `ORDER#<date>#<id>` sorts by date; `--no-scan-index-forward` returns newest first
- → A new question (say, "orders containing product 7") is easy in SQL and may need a new index or table in NoSQL

SQL EQUIVALENT

- ⇄ What SQL keeps that NoSQL often drops: ad hoc joins, `FOREIGN KEY` and `CHECK` constraints, multi-table transactions
- ⇄ Normalization removes duplication; NoSQL designs duplicate on purpose · See SQL 61, 62

ACROSS ENGINES

- **MongoDB** nested records that vary per item, with rich queries: catalogs, content, profiles
- **Redis** caching, sessions, counters, leaderboards, rate limits, queues
- **DynamoDB** serverless AWS apps with known key-based access at any scale
- **Cassandra** huge write volumes across regions: events, IoT, messaging
- **Neo4j** questions about connections: recommendations, fraud, permissions

IN YOUR CODE

- **Node** SQL: Prisma or `pg` · NoSQL: Mongoose or the `mongodb` driver
- **Python** SQL: SQLAlchemy · NoSQL: PyMongo or Beanie
- **Java** Spring Data JPA and Spring Data MongoDB share the same `Repository` style

DECISION TABLE

IF YOU NEED…	LEAN TOWARD
Ad hoc reporting and joins	SQL
Integrity rules across tables	SQL
Nested records that vary	Document
Key lookups at very high rates	Key-value
Massive writes, many regions	Wide-column
Relationship-heavy queries	Graph
Full-text relevance	Search engine

MYTHS

MYTH	REALITY
"NoSQL has no schema"	The schema lives in your code
"NoSQL has no transactions"	MongoDB, DynamoDB, Neo4j have ACID transactions
"SQL can't scale"	Replicas, partitioning and Citus scale far
"NoSQL is faster"	Only for the access patterns it was modeled for

⚠ COMMON MISTAKES

- Switching to NoSQL and then needing joins everywhere
- Modeling NoSQL data as normalized tables
- Forgetting that reporting and analytics will still want SQL

✓ TIPS & BEST PRACTICES

- Write down the top five queries before choosing
- Prototype the hardest query in both
- A common mix: SQL as source of truth, Redis as cache

QUICK SUMMARY

- SQL — joins, constraints, ad hoc
- NoSQL — known access patterns
- Duplicate on purpose in NoSQL
- Mixing both is normal

Sheet 03

CAP, Consistency & BASE

CAP theorem · PACELC · strong vs eventual consistency · read-your-writes · quorum · BASE vs ACID

CONSISTENCY IS A DIAL

The **CAP theorem** says that during a network **partition** a distributed database must choose between **consistency** (every read sees the latest write) and **availability** (every request gets an answer). **PACELC** adds that even without a partition you trade **latency** for consistency. With **eventual consistency** replicas converge after a short delay. Quorums give strong reads when **R + W > N** (replicas read + replicas written > replication factor). **BASE** (basically available, soft state, eventually consistent) is the counterpart to ACID.

NOTES

- CAP only describes behavior during a network partition
- Most engines let you choose per request
- "Eventual" is usually milliseconds, not hours
- CAP's C (latest value) is not ACID's C (valid state)

PRODUCTION CHECK

- A user saves a profile and the next page reads a stale replica. Read from the primary, or use a strongly consistent read, right after a write
- Choose per request: strong for stock and balances, eventual for feeds and counts
- Strong reads cost more: in DynamoDB they use twice the read capacity, in Cassandra they wait on more replicas

CONSISTENCY KNOBS PER ENGINE

```
// MongoDB: write and read concern, read preference
db.orders.insertOne(doc, { writeConcern: { w: "majority" } })
db.orders.find({ customerId: 42 })
  .readConcern("majority")      // only majority-acknowledged data
  .readPref("primary")          // read your own writes

// DynamoDB: eventual by default, strong on request
aws dynamodb get-item --table-name Shop \
  --key '{"PK":{"S":"ORDER#1001"},"SK":{"S":"META"}}' \
  --consistent-read

// Cassandra (cqlsh): consistency level per statement
CONSISTENCY QUORUM;
SELECT * FROM orders_by_customer WHERE customer_id = 42;

// Redis: async replicas; WAIT blocks until n replicas ack
SET order:1001:status "paid"
WAIT 1 100                      // replicas that acknowledged

// Neo4j cluster: bookmarks give causal consistency
const session = driver.session({ bookmarks: lastBookmarks })
```

CODE EXPLAINED

- → `w: "majority"` returns only after most replica-set members have the write
- → `--consistent-read` works on tables and local indexes, not on global secondary indexes
- → `QUORUM` with replication factor 3 means 2 replicas; reading and writing at `QUORUM` gives 2 + 2 > 3
- → `WAIT` narrows Redis's replication gap but doesn't make it strongly consistent

SQL EQUIVALENT

- ⇄ One PostgreSQL primary is strongly consistent; its read replicas are eventually consistent, the same trade-off
- ⇄ Isolation levels are about concurrent transactions, a different axis from replica lag · See SQL 69

ACROSS ENGINES

- **MongoDB** strong on the primary; secondaries lag; tune with read and write concern
- **Redis** asynchronous replicas; replica reads may be stale
- **DynamoDB** eventual by default; `ConsistentRead` on tables and LSIs only
- **Cassandra** tunable per query: `ONE`, `QUORUM`, `LOCAL_QUORUM`, `ALL`
- **Neo4j** causal consistency in clusters through bookmarks

IN YOUR CODE

- **Node** `orders.find(q, { readPreference: "primary" })`
- **Python** `db.get_collection("orders", read_concern=ReadConcern("majority"))`
- **Java** `orders.withReadConcern(ReadConcern.MAJORITY)`

CONSISTENCY MODELS

MODEL	GUARANTEE
Strong (linearizable)	Every read sees the latest write
Causal	You see the writes yours depended on
Read-your-writes	A client sees its own writes
Monotonic reads	Reads never go back in time
Eventual	Replicas converge once writes stop

QUORUM MATH (N = 3)

WRITE / READ	RESULT
QUORUM / QUORUM	2 + 2 > 3: strong
ONE / ONE	1 + 1 ≤ 3: eventual, fastest
ALL / ONE	Strong; writes fail if a node is down
ONE / ALL	Strong; reads fail if a node is down

⚠ COMMON MISTAKES

- Assuming a replica read is current
- Using the strongest setting everywhere and paying the latency
- Confusing CAP consistency with ACID consistency

✓ TIPS & BEST PRACTICES

- List which reads must be strong
- Read from the primary right after writing in the same session
- Monitor replication lag

QUICK SUMMARY

- CAP — choose during partitions
- PACELC — latency vs consistency
- R + W > N — strong quorums
- Pick per request

Sheet 04

Modeling From Access Patterns

list queries first · read / write ratio · query-driven design · denormalization · duplication trade-offs

LIST THE QUESTIONS FIRST

NoSQL modeling starts from the **access patterns**: the questions the app asks, with how often and how fast each must answer. Rank them, then shape the data so each common question is **one read** from one place. That usually means **denormalizing**: copying data to where it's read and accepting extra work on writes. Rare or slow questions can fall back to secondary indexes, batch jobs or an analytics store.

NOTES

- Reads usually outnumber writes, so optimize them first
- Every copy of data needs a plan for updating it
- Key design is hard to change later in DynamoDB and Cassandra
- Keep the access-pattern list in the repo

ACCESS-PATTERN WORKSHEET

```
# shop access patterns, most frequent first
- id: AP1
  question: Show a product page
  key: productId
  rate: 2000/s
  needs: product, average rating, 3 newest reviews
- id: AP2
  question: Customer's recent orders
  key: customerId, newest first
  rate: 150/s
  needs: date, status, total, line count
- id: AP3
  question: One order with its items
  key: orderId
  rate: 300/s
- id: AP4
  question: Reserve stock when an order is placed
  key: productId
  rate: 80/s
  rule: stock never below 0
- id: AP5
  question: Monthly sales by category
  rate: nightly
  rule: can be slow, read from analytics
```

CODE EXPLAINED

- → AP1: keep the 3 newest reviews and the rating total on the product (subset and computed patterns, sheet 25)
- → AP2: store orders keyed by customer, sorted by date
- → AP3: embed line items in the order, since they're always read together
- → AP4: a conditional atomic update on one record
- → AP5: don't model the operational store for it; export or aggregate offline

SQL EQUIVALENT

- ⇄ SQL models entities first and adds indexes for queries; NoSQL models the queries first
- ⇄ Denormalizing is an optimization in SQL and the default in NoSQL · See SQL 61, 63

ACROSS ENGINES

- **MongoDB** embed what's read together, reference what grows
- **Redis** one key per lookup; pick the structure by operation
- **DynamoDB** choose PK / SK so each pattern is a single Query
- **Cassandra** one table per query
- **Neo4j** model the traversals: which things connect to which

IN YOUR CODE

- **Node** `const recent = (id) => orders.find({ customerId: id }).sort({ orderDate: -1 }).limit(10).toArray()`
- **Python** `def recent(cid): return list(orders.find({"customerId": cid}).sort("orderDate", -1).limit(10))`
- **Java** `List<Order> findTop10ByCustomerIdOrderByOrderDateDesc(int customerId);`

QUESTIONS PER PATTERN

ASK	WHY
What is the key?	Decides the partition or index
How often?	Rank and optimize the top few
Read / write ratio?	How much write cost copies can add
How big does it grow?	Embed or reference
How fresh must it be?	Strong or eventual read

DENORMALIZING TRADE-OFFS

CHOICE	READ	WRITE
Embed	One read	Rewrites the parent
Copy a field	No join	Update every copy
Precompute a total	Instant	Recompute on change
Reference	Extra read	One place to update

⚠ COMMON MISTAKES

- Designing from the ER diagram instead of the queries
- Ignoring how fast arrays and partitions grow
- Copying data without a way to keep copies in sync

✓ TIPS & BEST PRACTICES

- Rank patterns by frequency and latency
- Design for the top patterns; index or batch the rest
- Revisit with real traffic numbers

QUICK SUMMARY

- Questions first
- One read per common question
- Denormalize on purpose
- Plan the update path

Sheet 05

Embedding vs Referencing

one-to-few · one-to-many · many-to-many · embed · reference · extended reference · document size

TOGETHER OR APART

Embed data inside its parent when it's read together, owned by the parent and **bounded** in size, such as a customer's two addresses. **Reference** it by id when it grows without limit, is shared by many parents or changes on its own, such as reviews. An **extended reference** copies just the few fields you display (a customer name on an order) next to the id. Record size limits set the hard ceiling: 16 MB per MongoDB document, 400 KB per DynamoDB item.

NOTES

- One-to-few: embed
- One-to-many: embed ids, or reference from the child
- One-to-millions: the child stores the parent's id
- Many-to-many: arrays of ids, or a link collection

PRODUCTION CHECK

- An array that grows forever (reviews, events) slows every read and eventually hits the 16 MB limit. Move children to their own collection with the parent's id
- Large documents rewrite and replicate more on each update; keep frequently updated documents small
- Copied fields go stale; decide who updates the copies and when (sheet 58)

EMBED, REFERENCE, EXTENDED REFERENCE

```
// Embed: one-to-few, read with the parent, owned by it
db.customers.insertOne({
  _id: 42, name: "Ada Lovelace",
  addresses: [ { type: "ship", city: "Des Moines", country: "US" },
               { type: "bill", city: "Ames", country: "US" } ]
})

// Reference: one-to-many that grows; the child points at the parent
db.reviews.insertOne({ productId: 7, customerId: 42, stars: 5,
                       text: "Clear and short", createdAt: new Date() })
db.reviews.createIndex({ productId: 1, createdAt: -1 })

// Extended reference: copy only the fields you display
db.orders.insertOne({
  _id: 1001,
  customer: { id: 42, name: "Ada Lovelace" },
  items: [ { productId: 7, name: "SQL Pocket Guide", qty: 2, unitPrice: 19.99 },
           { productId: 3, name: "USB-C Cable", qty: 1, unitPrice: 9.99 } ]
})

// Many-to-many: arrays of ids on one side
db.products.updateOne({ _id: 7 }, { $addToSet: { categoryIds: 3 } })
```

CODE EXPLAINED

- Addresses never grow past a handful and are always shown with the customer
- Reviews can number in the thousands, so each is its own document with an index on `productId`
- `unitPrice` is copied on purpose: the order must keep the price paid, even if the product's price changes
- The copied customer name may go stale; it's refreshed by whoever changes the name, or accepted as a snapshot

SQL EQUIVALENT

- Reference ↔ foreign key + `JOIN`; embed has no direct form (closest: a `jsonb` column)
- Many-to-many ↔ a junction table · See SQL 56, 62

ACROSS ENGINES

- **Redis** embed as a JSON value or hash fields; reference with key names like `customer:42`
- **DynamoDB** embed as map / list attributes (400 KB cap); reference with items under one partition key
- **Cassandra** embed with collections or UDTs; reference by copying into query tables
- **Neo4j** references are relationships; embedded values are properties

IN YOUR CODE

- **Node** Mongoose: `addresses: [addressSchema]` vs `customer: { type: ObjectId, ref: "Customer" }` + `.populate()`
- **Python** Beanie: `addresses: list[Address]` vs `customer: Link[Customer]`
- **Java** Spring Data: `List<Address> addresses` vs `@DocumentReference Customer customer`

RULES OF THUMB

RELATIONSHIP	USUALLY
One-to-one	Embed
One-to-few	Embed
One-to-many (hundreds)	Embed ids or reference
One-to-millions	Child stores parent id
Many-to-many	Id arrays or link collection

RECORD SIZE LIMITS

ENGINE	LIMIT
MongoDB	16 MB per document
DynamoDB	400 KB per item
Redis	512 MB per string value
Cassandra	Keep partitions under ~100 MB
Neo4j	No record limit; keep properties small

⚠ COMMON MISTAKES

- Embedding a list that grows without bound
- Referencing everything and then `$lookup`-ing on every read
- Copying fields without deciding how they get updated

✓ TIPS & BEST PRACTICES

- Ask "is it read together?" and "does it grow?" first
- Cap embedded arrays with `$slice` (subset pattern)
- Copy values that should freeze in time, like prices on orders

QUICK SUMMARY

- Embed — bounded, read together
- Reference — grows or is shared
- Extended reference — copy display fields
- Watch the size limits

Choosing an Engine

MongoDB · Redis · DynamoDB · Cassandra · Neo4j · Elasticsearch · managed vs self-hosted · comparison table

DATA SHAPE FIRST, THEN OPERATIONS

Pick the engine whose data model fits your top access patterns, then weigh the practical side: **managed or self-hosted**, cloud lock-in, the licence, the cost model and what your team already knows. Managed services remove most of the hard parts of running a distributed database, so start there unless you have a reason not to. Run them all locally with Docker while you learn.

NOTES

- Licences differ: MongoDB SSPL, Redis 8 AGPLv3 / RSAL / SSPL, Valkey BSD, Cassandra Apache 2.0, Neo4j Community GPLv3
- DynamoDB exists only on AWS
- Check the licence before offering a database as a service

EVERY ENGINE LOCALLY

```yaml
# compose.yaml: the engines in this set, for local development
services:
  mongo:
    image: mongo:8
    ports: ["27017:27017"]
  redis:
    image: redis:8
    ports: ["6379:6379"]
  dynamodb:
    image: amazon/dynamodb-local
    ports: ["8000:8000"]
  cassandra:
    image: cassandra:5.0
    ports: ["9042:9042"]
    environment:
      MAX_HEAP_SIZE: 1G
      HEAP_NEWSIZE: 256M
  neo4j:
    image: neo4j:latest
    ports: ["7474:7474", "7687:7687"]
    environment:
      NEO4J_AUTH: neo4j/shop-pass-123

# docker compose up -d
```

CODE EXPLAINED

- → DynamoDB Local accepts any credentials; point tools at `--endpoint-url http://localhost:8000`
- → Cassandra takes about a minute to start; the heap settings keep it small on a laptop
- → Neo4j Browser runs at port 7474; drivers connect over Bolt on 7687
- → Neo4j requires a password of at least 8 characters

SQL EQUIVALENT

- ⇄ PostgreSQL stays the safe default for relational data; add a NoSQL engine for a specific job, not instead of thinking
- ⇄ Many teams run one SQL database plus Redis and nothing else · See SQL 1

ACROSS ENGINES

- **MongoDB** Atlas on AWS, GCP or Azure; or self-host Community / Enterprise
- **Redis** Redis Cloud, ElastiCache, MemoryDB, or self-host; Valkey for a BSD licence
- **DynamoDB** AWS only; DynamoDB Local for development
- **Cassandra** self-host, DataStax Astra, or Amazon Keyspaces (CQL-compatible)
- **Neo4j** Aura (managed) or self-host Community / Enterprise

IN YOUR CODE

- **Node** `new MongoClient(uri)` · `createClient()` · `new cassandra.Client({...})` · `neo4j.driver(uri)`
- **Python** `MongoClient(uri)` · `redis.Redis()` · `boto3.resource("dynamodb")` · `GraphDatabase.driver(uri)`
- **Java** `MongoClients.create(uri)` · `new JedisPooled()` · `CqlSession.builder().build()` · `GraphDatabase.driver(uri)`

COMPARISON

ENGINE	MODEL	QUERY	BEST AT
MongoDB	Document	MQL, aggregation	Flexible records
Redis	Data structures	Commands	Speed, caching
DynamoDB	Key-value / doc	API, PartiQL	Serverless scale
Cassandra	Wide-column	CQL	Writes, multi-region
Neo4j	Graph	Cypher	Relationships
Elasticsearch	Search	Query DSL	Text relevance

WHAT YOU PAY FOR (MANAGED)

SERVICE	BILLED BY
MongoDB Atlas	Cluster tier, or usage on Flex
Redis Cloud / ElastiCache	Memory size
DynamoDB	Requests + storage, or provisioned capacity
Astra / Keyspaces	Reads, writes, storage
Neo4j Aura	Memory size

⚠ COMMON MISTAKES

- Choosing by popularity instead of access patterns
- Self-hosting a distributed database with no one to run it
- Being surprised by lock-in after building on DynamoDB

✓ TIPS & BEST PRACTICES

- Prototype your top three patterns in the candidates
- Use the compose file to try engines side by side
- Prefer managed services in production

QUICK SUMMARY

- Fit the data model first
- Then: managed, licence, cost
- Start managed
- Try locally with Docker

Sheet 07

MongoDB Introduction & Setup

Atlas · Docker `mongo:8` · `mongosh` · Compass · connection string · shop data as documents

DOCUMENTS IN COLLECTIONS

MongoDB stores **BSON documents** (binary JSON) in **collections** inside **databases**. Each document has a unique `_id` and can hold nested objects and arrays, so one document often replaces several SQL rows. This set targets **MongoDB 8.2**. Run it in **Atlas** (managed, with a free tier) or self-host Community Server, talk to it with **mongosh**, and browse data with **Compass**. Below, the SQL set's shop data becomes three collections; order lines move inside each order.

NOTES

- Default port 27017
- `mongodb+srv://` connection strings look up hosts through DNS (Atlas)
- Databases and collections are created on first write
- Atlas M0 is free for learning

RUN, CONNECT AND LOAD THE SHOP DATA

```
// docker run -d --name mongo -p 27017:27017 \
//   -e MONGO_INITDB_ROOT_USERNAME=admin -e MONGO_INITDB_ROOT_PASSWORD=secret mongo:8
// mongosh "mongodb://admin:secret@localhost:27017/?authSource=admin"
// mongosh "mongodb+srv://user:pass@cluster0.abcde.mongodb.net/shop"

use shop
db.customers.insertOne({ _id: 42, name: "Ada Lovelace",
  email: "ada@example.com", city: "Des Moines", country: "US",
  createdAt: new Date() })

db.products.insertMany([
  { _id: 7, name: "SQL Pocket Guide", category: "Books",
    price: NumberDecimal("19.99"), stock: 10 },
  { _id: 3, name: "USB-C Cable", category: "Electronics",
    price: NumberDecimal("9.99"), stock: 120 }
])

db.orders.insertOne({ _id: 1001, customerId: 42,
  orderDate: ISODate("2026-11-01"), status: "pending",
  total: NumberDecimal("49.97"),
  items: [ { productId: 7, qty: 2, unitPrice: NumberDecimal("19.99") },
           { productId: 3, qty: 1, unitPrice: NumberDecimal("9.99") } ] })

db.orders.findOne({ _id: 1001 })
```

CODE EXPLAINED

- → The `customers` and `products` tables map straight to collections
- → `order_items` disappears: its rows become the embedded `items` array
- → SQL ids are reused as `_id` here; new documents usually get a generated `ObjectId`
- → `NumberDecimal` stores money exactly (sheet 08)

SQL EQUIVALENT

- ⇄ database ↔ database · collection ↔ table · document ↔ row · field ↔ column
- ⇄ The `orders` + `order_items` join becomes one document · See SQL 2, 4

ACROSS ENGINES

- **Redis** `docker run redis:8`, then `redis-cli`
- **DynamoDB** DynamoDB Local in Docker; `aws dynamodb --endpoint-url …`
- **Cassandra** `cassandra:5.0` image, then `cqlsh`
- **Neo4j** `neo4j` image; Browser at port 7474

IN YOUR CODE

- **Node** `const db = new MongoClient(uri).db("shop")`
- **Python** `db = MongoClient(uri)["shop"]`
- **Java** `MongoDatabase db = MongoClients.create(uri).getDatabase("shop")`

SQL TO MONGODB

SQL	MONGODB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary key	`_id`
Join	Embedding or `$lookup`
Index	Index

TOOLS

TOOL	FOR
`mongosh`	Interactive shell and scripts
Compass	GUI: browse, query, explain
Atlas	Managed clusters, search, backups
Database Tools	`mongodump`, `mongorestore`, `mongoimport`
VS Code extension	Playgrounds and browsing

⚠ COMMON MISTAKES

- Forgetting `authSource=admin` when logging in as the root user
- Storing prices as plain numbers (doubles)
- Copying the SQL schema table by table instead of embedding order lines

✓ TIPS & BEST PRACTICES

- Use Atlas M0 or Docker for learning
- Keep a `seed.js` you can `load()` in mongosh
- Name collections in plural, lower camel case

QUICK SUMMARY

- BSON documents in collections
- `_id` is the key
- mongosh, Compass, Atlas
- Order lines embed in orders

Sheet 08

Documents, BSON & Types

BSON · ObjectId · Date · Decimal128 · arrays · embedded documents · 16 MB limit

JSON WITH MORE TYPES

Documents are stored as **BSON**, a binary JSON with extra types: **ObjectId**, **Date**, **Decimal128**, 32- and 64-bit integers, binary data and more. Field order is kept, values can nest up to 100 levels deep, and a document can be at most **16 MB**. The type matters for queries and sorting: `"10"` (string) and `10` (number) never match each other.

NOTES

- Use Decimal128 or integer cents for money
- Dates are stored as UTC milliseconds
- `null` and a missing field are different things
- Use GridFS for files over 16 MB

BSON TYPES IN MONGOSH

```
db.products.insertOne({
  _id: ObjectId(),                    // 12 bytes: time + random + counter
  sku: "BK-0007",                     // String
  name: "SQL Pocket Guide",
  price: NumberDecimal("19.99"),       // Decimal128: exact
  stock: NumberInt(10),                // Int32
  views: NumberLong("9000000000"),     // Int64
  rating: 4.5,                         // Double
  active: true,                        // Boolean
  tags: ["sql", "reference"],          // Array
  dims: { w: 11, h: 18, unit: "cm" },  // embedded document
  createdAt: new Date(),               // Date (UTC)
  discontinuedAt: null                 // Null
})

ObjectId("6929a1c2e4b0a1b2c3d4e5f6").getTimestamp()
db.products.find({ price: { $type: "decimal" } })
db.products.find({ discontinuedAt: null })       // null OR missing
db.products.find({ discontinuedAt: { $type: "null" } })   // null only
typeof db.products.findOne().stock
```

CODE EXPLAINED

- → An ObjectId starts with a timestamp, so ids sort roughly by creation time
- → `NumberDecimal("19.99")` is exact; `19.99` as a Double can show rounding errors
- → `{ field: null }` matches both null and missing fields; `$type: "null"` matches only real nulls
- → `$type` accepts names (`"decimal"`) or numbers

SQL EQUIVALENT

- ⇄ ObjectId ↔ identity / UUID · Decimal128 ↔ `NUMERIC(10,2)` · Date ↔ `TIMESTAMPTZ`
- ⇄ Arrays and embedded documents ↔ child tables or `jsonb` · See SQL 7, 9, 10

ACROSS ENGINES

- **Redis** values are strings at heart; RedisJSON stores real JSON
- **DynamoDB** types `S`, `N`, `B`, `BOOL`, `NULL`, `M`, `L`, sets; numbers travel as strings
- **Cassandra** typed columns: `int`, `decimal`, `text`, `timestamp`, `uuid`, collections, UDTs
- **Neo4j** properties: numbers, strings, booleans, temporal, point, and lists of these

IN YOUR CODE

- **Node** `import { ObjectId, Decimal128 } from "mongodb"; Decimal128.fromString("19.99")`
- **Python** `from bson import ObjectId, Decimal128; Decimal128("19.99")`
- **Java** `new Decimal128(new BigDecimal("19.99"))` · `new ObjectId()`

COMMON BSON TYPES

TYPE	MONGOSH	$TYPE
Double	4.5	"double"
String	"text"	"string"
Object	{ … }	"object"
Array	[…]	"array"
ObjectId	ObjectId()	"objectId"
Boolean	true	"bool"
Date	new Date()	"date"
Int32 / Int64	NumberInt / NumberLong	"int" / "long"
Decimal128	NumberDecimal("1.0")	"decimal"

LIMITS & RULES

RULE	VALUE
Max document size	16 MB
Max nesting depth	100 levels
_id	Required, unique, immutable
Field names	Avoid leading $ and dots
Large files	GridFS (chunks of 255 KB)

⚠ COMMON MISTAKES

- Storing prices as Doubles and summing them
- Storing dates as strings, which breaks range queries
- Mixing types in one field, such as `"10"` and `10`

✓ TIPS & BEST PRACTICES

- Pick one type per field and enforce it with validation (sheet 26)
- Store dates as Date in UTC; format in the app
- Use short field names only if storage really matters

QUICK SUMMARY

- BSON — JSON plus types
- Decimal128 for money
- Date in UTC
- 16 MB per document

Sheet 09

Databases, Collections & _id

use · db.createCollection · _id · capped collections · time series collections · naming

NAMESPACES AND THE PRIMARY KEY

A **database** groups **collections**, and both appear the first time you write to them. `db.createCollection()` is optional except for special kinds: **capped** collections (fixed size, oldest documents overwritten), **time series** collections (compressed measurements) and **clustered** collections (stored in `_id` order). Every document needs an **`_id`**: unique, immutable and indexed. It can be any type except an array.

NOTES

- Names are case-sensitive
- The `system.` prefix is reserved
- Time series collections replace the hand-built bucket pattern
- A view is a read-only collection defined by a pipeline (sheet 21)

DATABASES, COLLECTIONS & _ID

```
show dbs
use shop                                  // created on first write
db.createCollection("orders")             // optional for normal collections
show collections

// capped: fixed size in bytes, insertion order, oldest overwritten
db.createCollection("auditLog", { capped: true, size: 10485760, max: 100000 })

// time series: compressed measurements, auto-expiring
db.createCollection("readings", {
  timeseries: { timeField: "ts", metaField: "sensor", granularity: "minutes" },
  expireAfterSeconds: 2592000              // 30 days
})

db.orders.renameCollection("orders_2025")
db.orders_2025.drop()

// _id: generated, any unique value, or a compound document
db.customers.insertOne({ name: "Grace Hopper" })          // ObjectId
db.customers.insertOne({ _id: "cust-43", name: "Linus" })
db.coupons.insertOne({ _id: { code: "FALL26", region: "US" }, pct: 10 })
db.customers.getIndexes()     // [ { key: { _id: 1 }, name: "_id_" } ]
```

CODE EXPLAINED

- → `use` only switches context; nothing exists until the first insert
- → A capped collection is sized in bytes; `max` also caps the document count
- → `granularity` hints how often readings arrive so MongoDB can bucket them well
- → A compound `_id` works as a natural composite key, but its field order matters for matching

SQL EQUIVALENT

- ⇄ `use shop` ↔ `\c shop` · `createCollection` ↔ `CREATE TABLE` without columns
- ⇄ `_id` ↔ `PRIMARY KEY` ; ObjectId ↔ identity or UUID · See SQL 55, 58

ACROSS ENGINES

- **Redis** numbered databases 0–15 (`SELECT 1`); key prefixes act as namespaces
- **DynamoDB** no databases: tables per account and region; the key schema is the primary key
- **Cassandra** keyspace ↔ database; every table declares a `PRIMARY KEY`
- **Neo4j** several databases in Enterprise; labels group nodes

IN YOUR CODE

- **Node** `await db.createCollection("readings", { timeseries: { timeField: "ts" } })`
- **Python** `db.create_collection("readings", timeseries={"timeField": "ts"})`
- **Java** `db.createCollection("readings", new CreateCollectionOptions().timeSeriesOptions(new TimeSeriesOptions("ts")))`

COLLECTION KINDS

KIND	USE FOR
Regular	Almost everything
Capped	Fixed-size logs, newest N
Time series	Metrics, IoT readings
Clustered	Range scans on _id, TTL by _id
View	Read-only pipeline results

SHELL COMMANDS

COMMAND	DOES
`show dbs`	List databases
`use name`	Switch database
`show collections`	List collections
`db.stats()`	Database size
`db.coll.stats()`	Collection size and indexes
`db.dropDatabase()`	Delete the current database

⚠ COMMON MISTAKES

- A typo in `use` silently creates a new database on the next write
- Using a capped collection for data you can't lose
- Trying to change `_id` ; delete and re-insert instead

✓ TIPS & BEST PRACTICES

- Use natural keys as `_id` when they're stable and unique
- Prefer time series collections for measurements
- Script collection setup (options, indexes, validation) in a migration file

QUICK SUMMARY

- Created on first write
- Capped, time series, clustered
- `_id` unique and immutable
- Any type but an array

Sheet 10

mongosh Essentials

show dbs · show collections · db.collection · cursors · .pretty() · load() · .mongoshrc.js

THE MONGODB SHELL

mongosh is a Node.js REPL connected to MongoDB: you can write JavaScript, define functions and loop over results. `find()` returns a **cursor** that prints the first 20 documents; type `it` for more. Load scripts with `load()`, run one-off commands with `--eval`, and put helpers in **`~/.mongoshrc.js`**. Output is already formatted, so the old `.pretty()` isn't needed.

NOTES

- Tab completes collections and methods
- `help` and `db.orders.help()` list commands
- `exit` or Ctrl+D quits
- Every collection method also exists in the drivers

CURSORS, SCRIPTS AND SETTINGS

```
db                                  // current database
db.orders.find()                    // cursor: first 20 documents
it                                  // next 20
db.orders.find().toArray().length
db.orders.findOne({}, { items: 0 })  // first match, without items

const c = db.orders.find({ status: "pending" })
while (c.hasNext()) printjson(c.next())
db.orders.find().forEach(o => print(o._id, o.total))

db.orders.countDocuments({ status: "pending" })
const topBooks = () => db.products.find({ category: "Books" }).sort({ price: -1 }).limit(3)
topBooks()

config.set("displayBatchSize", 50)   // documents per page
load("seed-shop.js")                 // run a script file

// ~/.mongoshrc.js runs at startup
prompt = () => `${db.getName()}> `

// from the terminal
// mongosh shop --quiet --eval 'db.orders.countDocuments()'
// mongosh shop --file report.js
```

CODE EXPLAINED

- → A cursor fetches documents in batches as you iterate
- → `printjson` prints a document; `print` prints plain values
- → Functions you define live until the shell exits, or forever if placed in `.mongoshrc.js`
- → `--quiet --eval` is handy in shell scripts and CI

SQL EQUIVALENT

- ⇄ mongosh ↔ psql · `show collections` ↔ `\dt` · `--eval` ↔ `psql -c`
- ⇄ mongosh is a full JavaScript runtime, unlike psql's meta-commands · See SQL 2

ACROSS ENGINES

- ◈ **Redis** `redis-cli` : `--scan` , `MONITOR` , `--eval script.lua`
- ◈ **DynamoDB** AWS CLI `aws dynamodb …` or NoSQL Workbench
- ◈ **Cassandra** `cqlsh` : `DESCRIBE KEYSPACES` , `SOURCE 'file.cql'`
- ◈ **Neo4j** `cypher-shell` or Neo4j Browser (`:help`)

IN YOUR CODE

- **Node** `for await (const o of orders.find({ status: "pending" })) console.log(o._id)`
- **Python** `for o in orders.find({"status": "pending"}): print(o["_id"])`
- **Java** `for (Document o : orders.find(eq("status", "pending"))) System.out.println(o.get("_id"));`

SHELL HELPERS

COMMAND	DOES
`show dbs / show collections`	List
`use name`	Switch database
`it`	Next page of results
`load("f.js")`	Run a script
`cls`	Clear the screen
`exit`	Quit

CURSOR METHODS

METHOD	DOES
`.toArray()`	All results as an array
`.forEach(fn)`	Run a function per document
`.hasNext() / .next()`	Manual iteration
`.sort() / .limit() / .skip()`	Shape results
`.explain()`	Show the query plan
`.batchSize(n)`	Documents per round trip

⚠ COMMON MISTAKES

- Calling `.toArray()` on a huge result and running out of memory
- Expecting `count()` on a cursor; use `countDocuments()`
- Running destructive scripts against the wrong connection

✓ TIPS & BEST PRACTICES

- Show the database in the prompt to avoid mistakes
- Keep reusable helpers in `.mongoshrc.js`
- Use `--file` scripts for repeatable data fixes

QUICK SUMMARY

- mongosh — JavaScript REPL
- Cursors print 20 at a time
- `load()` and `--eval`
- `.mongoshrc.js` for helpers

Inserting Documents

insertOne · insertMany · ordered · generated _id · write result · duplicate key error

ADDING DOCUMENTS

insertOne adds one document and **insertMany** adds an array of them in batches. If a document has no _id, the driver generates an ObjectId before sending it. A duplicate _id or unique-index value fails with error **E11000**. With ordered: true (the default) insertMany stops at the first error; with ordered: false it skips failures and inserts the rest. The **write concern** says how many replicas must confirm the write before the call returns.

NOTES

- insert() is deprecated; use insertOne / insertMany
- Default write concern is w: "majority"
- Schema validation runs on insert (sheet 26)

INSERTONE, INSERTMANY & ERRORS

```
db.products.insertOne({ name: "Mechanical Keyboard", category: "Electronics",
  price: NumberDecimal("89.00"), stock: 25 })
// { acknowledged: true, insertedId: ObjectId("…") }

db.products.insertMany([
  { _id: 21, name: "Desk Mat", price: NumberDecimal("24.50"), stock: 40 },
  { _id: 22, name: "Monitor Arm", price: NumberDecimal("79.00"), stock: 12 }
])
// { acknowledged: true, insertedIds: { '0': 21, '1': 22 } }

// duplicate _id → E11000; ordered: false keeps going
db.products.insertMany(
  [ { _id: 22, name: "Duplicate" }, { _id: 23, name: "Desk Lamp" } ],
  { ordered: false })
// MongoBulkWriteError: E11000 duplicate key error … insertedCount: 1

// stronger durability for an important write
db.orders.insertOne(
  { _id: 1002, customerId: 42, status: "pending", items: [] },
  { writeConcern: { w: "majority", j: true, wtimeout: 5000 } })
```

CODE EXPLAINED

- → insertedId is the generated or supplied _id
- → With ordered: false the "Desk Lamp" document is inserted even though the first one failed
- → j: true waits for the on-disk journal; wtimeout fails the call if replicas don't confirm in time
- → A failed write-concern wait doesn't undo the write; it may still be applied

SQL EQUIVALENT

- ⇄ insertOne ↔ INSERT INTO products (…) VALUES (…) RETURNING product_id
- ⇄ insertMany ↔ multi-row VALUES ; E11000 ↔ unique violation (23505) · See SQL 50

ACROSS ENGINES

- **Redis** SET and HSET create or overwrite; SET … NX inserts only if missing
- **DynamoDB** PutItem overwrites; add attribute_not_exists(PK) for insert-only
- **Cassandra** INSERT is an upsert; IF NOT EXISTS for insert-only (lightweight transaction)
- **Neo4j** CREATE always adds; MERGE avoids duplicates

IN YOUR CODE

- **Node** const { insertedId } = await products.insertOne(doc)
- **Python** products.insert_one(doc).inserted_id
- **Java** products.insertOne(doc).getInsertedId()

INSERT OPTIONS

OPTION	EFFECT
ordered	Stop at first error (true) or continue
writeConcern	How many replicas confirm
bypassDocumentValidation	Skip schema validation
comment	Tag the operation in logs and profiler

WRITE CONCERN

SETTING	MEANING
w: 1	Primary confirmed
w: "majority"	Most members confirmed (default)
w: 0	Fire and forget
j: true	Written to the journal
wtimeout: ms	Give up waiting after ms

⚠ COMMON MISTAKES

- Assuming ordered: false inserts in a predictable order
- Parsing error message strings instead of checking error code 11000
- Using w: 0 for data you can't lose

✓ TIPS & BEST PRACTICES

- Supply your own _id when you need idempotent retries
- Batch inserts with insertMany instead of loops
- Catch duplicate-key errors by code (err.code === 11000)

QUICK SUMMARY

- insertOne / insertMany
- _id generated if missing
- E11000 — duplicate key
- ordered: false — keep going

Finding Documents

READING DOCUMENTS

find(filter, projection) returns a cursor over every match and **findOne** returns the first. A **filter** is a document of conditions joined by AND; a **projection** chooses fields with 1 (include) or 0 (exclude). Chain **sort**, **skip** and **limit** to shape the result. **countDocuments** counts matches with a query; **estimatedDocumentCount** reads collection metadata and is instant.

NOTES

- Don't mix 1 and 0 in one projection (except `_id: 0`)
- Sort runs before skip and limit, whatever order you chain them
- An empty filter `{}` matches everything

PRODUCTION CHECK

- A query without a matching index scans every document (COLLSCAN). Check with `explain("executionStats")` before shipping (sheet 23)
- Deep `skip()` pages read and throw away every skipped document; page with a range on an indexed field instead
- Always `limit()` user-facing queries and project only the fields you display

FIND, PROJECTION, SORT & PAGING

```
db.products.find({ category: "Books" })
db.products.find({ category: "Books", price: { $lt: 20 } })   // AND
db.products.findOne({ _id: 7 })

// projection: choose fields
db.products.find({ category: "Books" }, { name: 1, price: 1, _id: 0 })
db.orders.find({}, { items: 0 })                              // all but items

// sort, skip, limit
db.orders.find({ customerId: 42 }).sort({ orderDate: -1 }).limit(10)
db.orders.find().sort({ _id: 1 }).skip(20).limit(10)          // page 3

// range paging: fast at any depth
db.orders.find({ _id: { $gt: lastSeenId } }).sort({ _id: 1 }).limit(10)

// counting and distinct values
db.orders.countDocuments({ status: "pending" })
db.orders.estimatedDocumentCount()
db.products.distinct("category")
```

CODE EXPLAINED

- → Two conditions in one filter must both match
- → `{ items: 0 }` returns every field except `items`
- → Range paging remembers the last `_id` shown and asks for the next ones
- → `distinct` returns an array of the unique values

SQL EQUIVALENT

- ⇄ `find({ category: "Books" }, { name: 1 })` ↔ `SELECT name FROM products WHERE category = 'Books'`
- ⇄ `sort` / `skip` / `limit` ↔ `ORDER BY` / `OFFSET` / `LIMIT` · See SQL 13, 16

ACROSS ENGINES

- **Redis** no ad hoc queries; read by key, or add Search (`FT.SEARCH`)
- **DynamoDB** `Query` needs the partition key; other fields need a GSI or a `Scan`
- **Cassandra** `WHERE` must include the partition key, or use SAI / `ALLOW FILTERING`
- **Neo4j** `MATCH (p:Product {category: "Books"}) RETURN p.name ORDER BY p.price LIMIT 10`

IN YOUR CODE

- **Node** `await products.find({ category: "Books" }).project({ name: 1 }).limit(10).toArray()`
- **Python** `list(products.find({"category": "Books"}, {"name": 1}).limit(10))`
- **Java** `products.find(eq("category", "Books")).projection(include("name")).limit(10)`

SHAPING RESULTS

METHOD	DOES
`.sort({ f: 1 })`	Ascending (−1 descending)
`.limit(n)`	At most n documents
`.skip(n)`	Skip n documents
`.collation({ locale: "en" })`	Language-aware sorting
`.maxTimeMS(ms)`	Abort slow queries

COUNTING

METHOD	SPEED / ACCURACY
`countDocuments(filter)`	Runs a query; exact
`estimatedDocumentCount()`	Metadata; instant, approximate
`distinct(field)`	Unique values
`$count stage`	Count inside a pipeline

⚠ COMMON MISTAKES

- Mixing included and excluded fields in a projection
- Using `skip(100000)` for deep pages
- Fetching whole documents when the page shows two fields

✓ TIPS & BEST PRACTICES

- Sort on an indexed field
- Use `findOne` when you expect one result
- Add `.maxTimeMS()` to protect user-facing queries

QUICK SUMMARY

- `find` — cursor of matches
- Projection — pick fields
- `sort` · `skip` · `limit`
- Range paging beats `skip`

Query Operators

CONDITIONS BEYOND EQUALITY

Query operators start with $. **Comparison** operators (`$gt`, `$lte`, `$in`, `$ne`) test a field's value; **logical** operators (`$or`, `$and`, `$nor`, `$not`) combine conditions; **element** operators (`$exists`, `$type`) test whether and how a field is stored. **`$regex`** matches patterns and **`$expr`** lets you compare two fields with aggregation expressions.

COMPARISON, LOGICAL, ELEMENT & REGEX

```
db.products.find({ price: { $gte: 10, $lte: 50 } })
db.products.find({ category: { $in: ["Books", "Electronics"] } })
db.products.find({ category: { $nin: ["Clearance"] } })
db.products.find({ stock: { $ne: 0 } })        // also matches missing stock

db.orders.find({ $or: [ { status: "pending" }, { total: { $gt: 100 } } ] })
db.orders.find({ $and: [                       // needed to combine two $or
  { $or: [ { status: "pending" }, { status: "paid" } ] },
  { $or: [ { total: { $gt: 100 } }, { rush: true } ] } ] })
db.products.find({ price: { $not: { $gt: 50 } } })

db.customers.find({ email: { $exists: false } })
db.products.find({ price: { $type: "decimal" } })

db.customers.find({ name: { $regex: "^ada", $options: "i" } })
db.customers.find({ name: /^Ada/ })                 // literal form

// compare two fields of the same document
db.products.find({ $expr: { $lt: ["$stock", "$reorderLevel"] } })
```

NOTES

- `{ f: null }` matches null and missing fields
- Anchored, case-sensitive regexes (`/^Ada/`) can use an index
- `$ne` and `$nin` also match documents without the field
- `$expr` usually can't use indexes well

CODE EXPLAINED

→ Two operators on one field (`$gte` , `$lte`) form a range

→ `$and` is implicit unless you need the same key twice, such as two `$or` clauses

→ `$not` also matches documents where the field is missing

→ In `$expr` , `"$stock"` means "the value of the stock field"

SQL EQUIVALENT

⇄ `$in` ↔ `IN` · `$exists: false` ↔ `IS NULL` (roughly) · `$regex` ↔ `~` or `LIKE`

⇄ `$expr` ↔ `WHERE stock < reorder_level` · See SQL 18, 19

ACROSS ENGINES

- **Redis** no operators on values; ranges come from sorted sets (`ZRANGE … BYSCORE`)
- **DynamoDB** key conditions allow `=` , `<` , `BETWEEN` , `begins_with` on the sort key; filters allow more but run after the read
- **Cassandra** ranges on clustering columns only; `IN` on the partition key
- **Neo4j** `WHERE p.price >= 10 AND p.category IN ['Books']` ; `=~` for regex

IN YOUR CODE

- **Node** `find({ price: { $gte: 10, $lte: 50 } })`
- **Python** `find({"price": {"$gte": 10, "$lte": 50}})`
- **Java** `find(and(gte("price", 10), lte("price", 50)))`

COMPARISON

OPERATOR	MATCHES
`$eq` / `$ne`	Equal / not equal
`$gt` / `$gte`	Greater / or equal
`$lt` / `$lte`	Less / or equal
`$in` / `$nin`	In / not in a list

LOGICAL & ELEMENT

OPERATOR	MATCHES
`$and` / `$or`	All / any condition
`$nor`	None of the conditions
`$not`	Negates one operator
`$exists`	Field present or absent
`$type`	Field has a BSON type
`$regex`	Pattern match
`$expr`	Aggregation expression

⚠ COMMON MISTAKES

- Expecting `$ne: 0` to skip documents without the field
- Unanchored or case-insensitive regexes on big collections
- Writing `{ price: { $gt: 10 }, price: { $lt: 50 } }` : the second key overwrites the first

✓ TIPS & BEST PRACTICES

- Put both bounds in one operator document
- Use `$in` instead of long `$or` chains on one field
- Store a lowercase copy of a field for fast case-insensitive matches

QUICK SUMMARY

- `$gt` `$lt` `$in` — compare
- `$or` `$and` `$not` — combine
- `$exists` `$type` — element
- `$expr` — compare fields

Sheet 14

Querying Arrays & Nested Fields

dot notation · $elemMatch · $all · $size · array index · positional projection

REACHING INSIDE DOCUMENTS

Use **dot notation** (`"address.city"`, always quoted) to query nested fields and array elements. A plain condition on an array matches if **any element** matches. For arrays of documents, separate conditions may be met by **different** elements; **`$elemMatch`** requires **one** element to meet them all. **`$all`** and **`$size`** test the array as a whole, and projection operators (`$`, `$slice`, `$elemMatch`) return part of an array.

NOTES

- Exact embedded-document matches compare field order too
- Indexes on array fields are multikey (sheet 22)
- `"tags.0"` addresses an element by position

DOT NOTATION, $ELEMMATCH & PROJECTIONS

```
db.customers.find({ "address.city": "Des Moines" })    // nested field
db.products.find({ tags: "sql" })                      // array contains value
db.products.find({ tags: { $all: ["sql", "reference"] } })
db.products.find({ tags: { $size: 2 } })
db.products.find({ "tags.0": "sql" })                  // first element

// separate conditions: may match DIFFERENT elements
db.orders.find({ "items.productId": 7, "items.qty": { $gte: 2 } })

// $elemMatch: ONE element must match both
db.orders.find({ items: { $elemMatch: { productId: 7, qty: { $gte: 2 } } } })

// exact embedded document: same fields, same order
db.customers.find({ address: { city: "Ames", country: "US" } })

// return part of an array
db.orders.find({ "items.productId": 7 }, { "items.$": 1 })   // matched element
db.orders.find({}, { items: { $slice: 2 } })                 // first 2
db.orders.find({}, { items: { $elemMatch: { qty: { $gte: 2 } } } })
```

CODE EXPLAINED

- → `{ tags: "sql" }` matches any document whose `tags` array contains "sql"
- → Without `$elemMatch`, an order with product 7 (qty 1) and product 3 (qty 5) still matches
- → `$slice: -2` would return the last two elements
- → `items.$` needs the array in the filter to know which element matched

SQL EQUIVALENT

- ⇄ Arrays are child tables in SQL: `WHERE EXISTS (SELECT 1 FROM order_items i WHERE i.order_id = o.order_id AND i.product_id = 7 AND i.quantity >= 2)`
- ⇄ PostgreSQL `jsonb @>` is the closest in-row form · See SQL 36, 78

ACROSS ENGINES

- **Redis** sets: `SISMEMBER product:7:tags sql`; RedisJSON paths like `$.items[?(@.qty>1)]`
- **DynamoDB** nested paths `items[0].qty` in expressions; `contains(tags, :t)` in filters
- **Cassandra** collection `CONTAINS` needs an index; frozen UDT fields
- **Neo4j** list predicates: `'sql' IN p.tags`, `any(x IN list WHERE …)`

IN YOUR CODE

- **Node** `find({ items: { $elemMatch: { productId: 7, qty: { $gte: 2 } } } })`
- **Python** `find({"items": {"$elemMatch": {"productId": 7, "qty": {"$gte": 2}}}})`
- **Java** `find(elemMatch("items", and(eq("productId", 7), gte("qty", 2))))`

ARRAY QUERY OPERATORS

OPERATOR	MATCHES WHEN
`value`	Any element equals it
`$all`	Contains every listed value
`$size`	Has exactly n elements
`$elemMatch`	One element meets all conditions
`"arr.N"`	Element at position N matches

ARRAY PROJECTIONS

PROJECTION	RETURNS
`"arr.$": 1`	First element matched by the filter
`{ $slice: n }`	First n (or last n if negative)
`{ $slice: [skip, n] }`	n elements after skip
`{ $elemMatch: … }`	First element meeting the condition

⚠ COMMON MISTAKES

- Expecting separate conditions on an array of documents to apply to one element
- Forgetting quotes around dotted field names
- Matching whole embedded documents when field order may differ

✓ TIPS & BEST PRACTICES

- Use `$elemMatch` whenever two conditions describe one array element
- Query nested fields with dot notation, not whole subdocuments
- Return only the matching array elements to save bandwidth

QUICK SUMMARY

- Dot notation — nested fields
- Value on array — any element
- `$elemMatch` — one element, all conditions
- `$`, `$slice` — partial arrays

Sheet 15

Updating Documents

updateOne · updateMany · $set · $inc · $unset · $push / $pull · $[] / $[elem]

CHANGE FIELDS, NOT WHOLE DOCUMENTS

`updateOne(filter, update, options)` changes the first matching document and **`updateMany`** changes every match. The update document is built from **operators**: `$set` writes fields, `$inc` adds to numbers, `$unset` removes fields, and `$push` / `$pull` edit arrays, so only the named fields change. A change to one document is **atomic**, including all of its nested fields and arrays. The **positional operators** reach into arrays: `$` for the element the filter matched, `$[]` for every element and `$[name]` with `arrayFilters` for the ones that match a condition.

NOTES

- An update without operators is rejected; use `replaceOne` to swap the whole document (sheet 16)
- `$set` creates missing fields and nested paths (`"shipping.carrier"`)
- An aggregation pipeline `[ { $set: … } ]` can compute from other fields

PRODUCTION CHECK

- `updateMany` isn't all-or-nothing: a failure partway leaves some documents changed. Wrap it in a transaction (sheet 27) when that matters
- `$inc` and `$push` double-apply if your code retries after a timeout; retryable writes cover driver retries, but app-level retries need an idempotency check
- Filter on indexed fields: an unindexed `updateMany` scans the whole collection

FIELD, ARRAY & POSITIONAL UPDATES

```
// $set and $currentDate on one order
db.orders.updateOne(
  { _id: 1001 },
  { $set: { status: "shipped", "shipping.carrier": "UPS" },
    $currentDate: { updatedAt: true } }
)
// { acknowledged: true, matchedCount: 1, modifiedCount: 1, ... }
db.products.updateOne({ _id: 7 }, { $inc: { stock: -2 } })

// arrays: add a line item, tag once, remove a line
db.orders.updateOne({ _id: 1001 }, {
  $push: { items: { productId: 7, qty: 2, unitPrice: 19.99 } },
  $addToSet: { tags: "gift" }
})
db.orders.updateOne({ _id: 1001 }, { $pull: { items: { productId: 3 } } })

// positional: the matched element, then filtered elements
db.orders.updateOne(
  { _id: 1001, "items.productId": 7 },
  { $set: { "items.$.qty": 3 } }
)
db.orders.updateMany(
  { status: "pending" },
  { $set: { "items.$[big].backordered": true } },
  { arrayFilters: [ { "big.qty": { $gt: 5 } } ] }
)
```

CODE EXPLAINED

- Several operators can share one update as long as they touch different fields
- `matchedCount` is how many matched the filter; `modifiedCount` is how many actually changed (setting a value to itself counts as matched only)
- `$addToSet` adds only if the value isn't already in the array; `$pull` removes every element matching the condition
- `items.$` needs the array in the filter (`"items.productId": 7`) and updates the first element that matched
- `$[big]` names a placeholder; `arrayFilters` says which elements it stands for

SQL EQUIVALENT

- `$set` ↔ `UPDATE orders SET status = 'shipped' WHERE order_id = 1001`
- `$inc` ↔ `SET stock = stock - 2` · `$mul` ↔ `SET price = price * 0.9`
- Arrays have no SQL form: `items` is the `order_items` table, so `$push` ↔ `INSERT INTO order_items …` · See SQL 51

IN YOUR CODE

- **Node** `await orders.updateOne({ _id: 1001 }, { $set: { status: "shipped" } })`
- **Python** `orders.update_one({"_id": 1001}, {"$set": {"status": "shipped"}})`
- **Java** `orders.updateOne(eq("_id", 1001), set("status", "shipped"))`

ACROSS ENGINES

- **Redis** `HSET order:1001 status shipped` · `HINCRBY product:7 stock -2` ; nested arrays need RedisJSON (`JSON.ARRAPPEND`)
- **DynamoDB** `UpdateItem` with `SET #s = :s, stock = stock - :n` ; `status` is reserved, so alias it with `#s`
- **Cassandra** `UPDATE orders SET status = 'shipped' WHERE order_id = 1001` — every write is an upsert; lists use `items = items + [...]`
- **Neo4j** `MATCH (o:Order {id: 1001}) SET o.status = 'shipped'` · `SET p.stock = p.stock - 2`

FIELD OPERATORS

OPERATOR	DOES
`$set`	Write a field (creates it if missing)
`$unset`	Remove a field
`$inc / $mul`	Add to / multiply a number
`$min / $max`	Write only if lower / higher
`$currentDate`	Set to the current date
`$setOnInsert`	Write only when an upsert inserts

ARRAY OPERATORS

OPERATOR	DOES
`$push`	Append (`$each`, `$slice`, `$sort`, `$position`)
`$addToSet`	Append if not present
`$pull / $pullAll`	Remove matching / listed values
`$`	First element the filter matched
`$[]`	Every element
`$[name]`	Elements matching `arrayFilters`

⚠ COMMON MISTAKES

- Passing `{ status: "shipped" }` without `$set` ; the update is rejected (or, with `replaceOne` , the rest of the document is lost)
- Using `updateOne` when the filter matches many; only the first document changes
- `$push` onto a field that isn't an array, which fails the whole update

✓ TIPS & BEST PRACTICES

- Check `matchedCount` vs `modifiedCount` to tell "not found" from "already set"
- Cap growing arrays with `$push` + `$each` + `$slice` to stay far from the 16 MB limit
- Use `findOneAndUpdate` when you need the updated document back (sheet 16)

QUICK SUMMARY

- `$set` / `$unset` — fields
- `$inc` / `$mul` — numbers
- `$push` / `$pull` — arrays
- `$` · `$[]` · `$[name]` — array elements

Sheet 16

Delete, Replace, Upsert & Bulk

deleteOne · deleteMany · replaceOne · upsert: true · findOneAndUpdate · bulkWrite

REMOVE, REPLACE, UPSERT, BATCH

deleteOne and **deleteMany** remove matching documents. **replaceOne** swaps a whole document except its _id. An **upsert** (`upsert: true`) updates a match or inserts a new document built from the filter and update. The **findOneAnd…** methods change one document and return it, before or after the change. **bulkWrite** sends many operations in one round trip.

NOTES

- `$setOnInsert` writes fields only when the upsert inserts
- Concurrent upserts on unindexed filters can create duplicates
- MongoDB 8.0 adds a client-level `bulkWrite` across collections
- Deleting all documents keeps indexes; `drop()` removes them too

DELETE, REPLACE, UPSERT, FIND-AND-MODIFY, BULK

```
db.orders.deleteOne({ _id: 1001 })
db.orders.deleteMany({ status: "cancelled", orderDate: { $lt: ISODate("2025-01-01") } })

// replace: everything except _id
db.products.replaceOne({ _id: 7 },
  { name: "SQL Pocket Guide, 5th ed.", category: "Books",
    price: NumberDecimal("21.99"), stock: 10 })

// upsert: update if found, insert if not
db.carts.updateOne(
  { customerId: 42 },
  { $push: { items: { productId: 3, qty: 1 } },
    $setOnInsert: { createdAt: new Date() } },
  { upsert: true })

// find-and-modify: returns the document
db.counters.findOneAndUpdate(
  { _id: "orderId" }, { $inc: { seq: 1 } },
  { upsert: true, returnDocument: "after" })
db.jobs.findOneAndDelete({ status: "queued" }, { sort: { createdAt: 1 } })

// bulk: several operations, one round trip
db.products.bulkWrite([
  { updateOne: { filter: { _id: 7 }, update: { $inc: { stock: -2 } } } },
  { insertOne: { document: { _id: 30, name: "Notebook" } } },
  { deleteOne: { filter: { _id: 21 } } }
], { ordered: false })
```

CODE EXPLAINED

- The upserted cart gets `customerId: 42` from the filter, the pushed item, and `createdAt`
- `returnDocument: "after"` returns the incremented counter; the default is the document before the change
- `findOneAndDelete` with `sort` takes the oldest queued job, a simple work queue
- `bulkWrite` returns counts per operation type

SQL EQUIVALENT

- `deleteMany` ↔ `DELETE … WHERE` · `replaceOne` ↔ `UPDATE` of every column
- Upsert ↔ `INSERT … ON CONFLICT DO UPDATE`; `findOneAndUpdate` ↔ `UPDATE … RETURNING` · See SQL 52, 53

ACROSS ENGINES

- **Redis** `DEL`, or `UNLINK` to free memory in the background; `SET` always upserts
- **DynamoDB** `PutItem` replaces; `UpdateItem` upserts; `ReturnValues: ALL_NEW`; `BatchWriteItem` takes 25 puts / deletes
- **Cassandra** `DELETE` writes a tombstone; `INSERT` and `UPDATE` both upsert
- **Neo4j** `DETACH DELETE`; `MERGE … ON CREATE SET … ON MATCH SET`

IN YOUR CODE

- **Node** `await counters.findOneAndUpdate(f, u, { upsert: true, returnDocument: "after" })`
- **Python** `find_one_and_update(f, u, upsert=True, return_document=ReturnDocument.AFTER)`
- **Java** `findOneAndUpdate(f, u, new FindOneAndUpdateOptions().upsert(true).returnDocument(ReturnDocument.AFTER))`

WHAT EACH RETURNS

METHOD	RESULT
`deleteOne / deleteMany`	`deletedCount`
`replaceOne`	`matchedCount`, `modifiedCount`
`updateOne + upsert`	`upsertedId` when inserted
`findOneAnd…`	The document (or `null`)
`bulkWrite`	Counts per operation

UPSERT RULES

SITUATION	RESULT
`Filter matches`	Normal update
`No match`	Insert: filter fields + update
`$setOnInsert`	Applied only on insert
`Filter has operators ($gt)`	Those fields aren't copied
`Concurrent upserts`	Use a unique index to prevent duplicates

⚠ COMMON MISTAKES

- `deleteMany({})` when you meant one document
- Using `replaceOne` to change one field and losing the rest
- Upserting on a filter with no unique index

✓ TIPS & BEST PRACTICES

- Run the filter with `find` before a `deleteMany`
- Use `findOneAndUpdate` for counters and queues
- Group many small writes into a `bulkWrite`

QUICK SUMMARY

- `deleteOne` / `deleteMany`
- `replaceOne` — whole document
- `upsert: true` — update or insert
- `bulkWrite` — one round trip

Sheet 17

Aggregation Pipeline Basics

aggregate · $match · $project · $addFields · $sort · $limit · stage order

PIPELINES OF STAGES

An **aggregation pipeline** is an array of **stages**. Documents flow through them top to bottom, and each stage filters, reshapes, groups or sorts what it receives. **$match** filters (use it first so it can use indexes), **$project** and **$addFields** reshape, **$sort** and **$limit** order and trim. Unlike SQL's fixed clause order, you choose the order and can repeat stages.

A FIRST PIPELINE

```
db.orders.aggregate([
  { $match: { status: { $in: ["paid", "shipped"] },
              orderDate: { $gte: ISODate("2026-01-01") } } },
  { $project: { customerId: 1, total: 1,
                lines: { $size: "$items" },
                month: { $month: "$orderDate" } } },
  { $addFields: { big: { $gte: ["$total", 100] } } },
  { $sort: { total: -1 } },
  { $limit: 5 }
])

// count matches
db.orders.aggregate([ { $match: { status: "pending" } }, { $count: "pending" } ])

// sample and unset
db.products.aggregate([ { $sample: { size: 3 } }, { $unset: ["stock"] } ])

// options and the plan
db.orders.aggregate(pipeline, { maxTimeMS: 5000, allowDiskUse: true })
db.orders.explain("executionStats").aggregate(pipeline)
```

NOTES

- Each stage has a 100 MB memory limit; bigger work spills to disk (default since 6.0)
- `$set` is an alias for `$addFields`
- Field paths are strings starting with `$` : `"$total"`

CODE EXPLAINED

- → `$match` at the start uses the same indexes as `find`
- → `$project` keeps `customerId` and `total` and computes two new fields
- → `$sort` followed by `$limit` is merged into a top-N sort that keeps only 5 documents in memory
- → `$unset` removes fields from the output

SQL EQUIVALENT

- ⇄ `$match` ↔ `WHERE` · `$project` ↔ `SELECT` list · `$sort` ↔ `ORDER BY` · `$limit` ↔ `LIMIT`
- ⇄ SQL runs clauses in a fixed logical order; a pipeline runs exactly in the order written · See SQL 5

ACROSS ENGINES

- **Redis** no pipeline engine; `FT.AGGREGATE` works on indexed data
- **DynamoDB** no aggregation; precompute on write or export to analytics
- **Cassandra** simple aggregates (`COUNT` , `SUM`) within a partition; Spark for more
- **Neo4j** chained clauses: `MATCH … WITH … ORDER BY … LIMIT`

IN YOUR CODE

- **Node** `await orders.aggregate([{ $match: { status: "paid" } }, { $limit: 5 }]).toArray()`
- **Python** `list(orders.aggregate([{"$match": {"status": "paid"}}, {"$limit": 5}]))`
- **Java** `orders.aggregate(List.of(match(eq("status", "paid")), limit(5)))`

EVERYDAY STAGES

STAGE	DOES
`$match`	Filter
`$project`	Choose and compute fields
`$addFields / $set`	Add fields, keep the rest
`$unset`	Remove fields
`$sort / $limit / $skip`	Order and trim
`$count`	Count documents
`$sample`	Random documents

PIPELINE PERFORMANCE

RULE	WHY
`$match first`	Uses indexes, shrinks the stream
`$sort + $limit together`	Top-N, less memory
`$project late`	Let MongoDB track needed fields
`Check explain()`	See IXSCAN vs COLLSCAN

⚠ COMMON MISTAKES

- Putting `$match` after `$group` when it could filter first
- Forgetting the `$` in field paths (`"total"` is a string literal)
- Sorting huge streams without an index or a `$limit`

✓ TIPS & BEST PRACTICES

- Build pipelines one stage at a time in Compass's Aggregations tab
- Store pipelines in code as named functions
- Add `maxTimeMS` to user-facing aggregations

QUICK SUMMARY

- Stages run in order
- `$match` first
- `$project` / `$addFields`
- `$sort` + `$limit` = top N

Sheet 18

$group & Accumulators

$group · $sum · $avg · $min / $max · $push · $addToSet · $count

GROUPING AND TOTALS

$group collects documents that share an **_id** expression and computes **accumulators** for each group: $sum, $avg, $min, $max, $push, $addToSet, $first, $count and the top-N family. Use _id: null for one group over everything, or a document for several grouping fields. **$unwind** before $group lets you total array elements, such as order lines.

NOTES

- Only _id and accumulator fields come out of $group
- $sum: 1 counts documents
- $topN, $bottomN, $firstN arrived in 5.2
- Filter groups with a $match after $group

$GROUP, ACCUMULATORS & TOP N

```
// revenue per customer
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$customerId",
              orders: { $sum: 1 },
              revenue: { $sum: "$total" },
              avgOrder: { $avg: "$total" },
              firstOrder: { $min: "$orderDate" } } },
  { $match: { revenue: { $gte: 500 } } },          // like HAVING
  { $sort: { revenue: -1 } }
])

// group by year and status
db.orders.aggregate([
  { $group: { _id: { year: { $year: "$orderDate" }, status: "$status" },
              n: { $count: {} } } }
])

// units sold per product from order lines
db.orders.aggregate([
  { $unwind: "$items" },
  { $group: { _id: "$items.productId",
              units: { $sum: "$items.qty" },
              buyers: { $addToSet: "$customerId" } } }
])

// three most expensive products per category
db.products.aggregate([
  { $group: { _id: "$category",
              top3: { $topN: { n: 3, sortBy: { price: -1 }, output: "$name" } } } }
])
```

CODE EXPLAINED

- → _id: "$customerId" makes one output document per customer
- → The $match after $group filters on computed totals, like SQL's HAVING
- → A compound _id groups by several values at once
- → $addToSet collects unique customer ids; $push would keep duplicates

SQL EQUIVALENT

- ⇄ $group ↔ GROUP BY · $sum: 1 ↔ COUNT(*) · $push ↔ array_agg
- ⇄ $addToSet ↔ array_agg(DISTINCT …) · $match after $group ↔ HAVING · See SQL 28, 29, 31

ACROSS ENGINES

- **Redis** keep counters on write (HINCRBY) instead of grouping later
- **DynamoDB** maintain totals on write (ADD revenue :t) or in a Streams consumer
- **Cassandra** GROUP BY only on partition and clustering columns
- **Neo4j** implicit grouping: RETURN c.id, count(o), sum(o.total)

IN YOUR CODE

- **Node** { $group: { _id: "$customerId", revenue: { $sum: "$total" } } }
- **Python** {"$group": {"_id": "$customerId", "revenue": {"$sum": "$total"}}}
- **Java** group("$customerId", sum("revenue", "$total"), avg("avgOrder", "$total"))

ACCUMULATORS

OPERATOR	RESULT
$sum / $avg	Total / mean
$min / $max	Smallest / largest
$count	Documents in group
$push / $addToSet	Array of values / unique values
$first / $last	By current sort order
$topN / $bottomN	N values by a sort

GROUP PATTERNS

GOAL	_ID
Whole collection	null
One field	"$status"
Several fields	{ y: { $year: "$d" }, s: "$status" }
Array elements	$unwind first
Date buckets	{ $dateTrunc: { date: "$d", unit: "week" } }

⚠ COMMON MISTAKES

- Expecting other fields to pass through $group
- Summing Double prices and getting rounding noise; store Decimal128
- Using $first without a $sort before the group

✓ TIPS & BEST PRACTICES

- $match before $group to shrink the input
- Name accumulator fields for what they mean (revenue , not total2)
- Cache heavy group results with $merge (sheet 21)

QUICK SUMMARY

- $group by _id
- Accumulators compute per group
- $match after = HAVING
- $unwind to total arrays

$lookup & $unwind

JOINS WHEN YOU NEED THEM

$lookup brings in matching documents from another collection as an **array** field, a left outer join. **$unwind** turns one document with an array into one document per element. Together they rebuild joined rows. The **pipeline** form of $lookup adds filters, sorting and limits inside the join. Use joins for occasional or reporting queries; if every read needs one, the data probably belongs embedded.

NOTES

- `$lookup` always returns an array (empty if no match)
- `$unwind` drops documents with empty arrays unless told otherwise
- `from` must be in the same database
- Sharded `from` collections are supported since 5.1

PRODUCTION CHECK

- `$lookup` without an index on `foreignField` scans the other collection once per input document; index the foreign field
- Joins multiply work on big inputs; `$match` and `$limit` before the `$lookup`
- If every request needs the same `$lookup`, embed or copy the fields instead (sheet 05)

$LOOKUP AND $UNWIND

```
// orders → customers
db.orders.aggregate([
  { $match: { status: "pending" } },
  { $lookup: { from: "customers", localField: "customerId",
               foreignField: "_id", as: "customer" } },
  { $unwind: "$customer" },                          // [ {…} ] → {…}
  { $project: { total: 1, "customer.name": 1, "customer.email": 1 } }
])

// add product names to each order line
db.orders.aggregate([
  { $match: { _id: 1001 } },
  { $unwind: "$items" },
  { $lookup: { from: "products", localField: "items.productId",
               foreignField: "_id", as: "product" } },
  { $set: { "items.name": { $first: "$product.name" } } },
  { $group: { _id: "$_id", items: { $push: "$items" } } }
])

// pipeline form: each customer's 3 newest orders
db.customers.aggregate([
  { $lookup: { from: "orders", localField: "_id", foreignField: "customerId",
               as: "recent",
               pipeline: [ { $sort: { orderDate: -1 } }, { $limit: 3 },
                           { $project: { total: 1, orderDate: 1 } } ] } }
])

// keep orders whose customer is missing
{ $unwind: { path: "$customer", preserveNullAndEmptyArrays: true } }
```

CODE EXPLAINED

- → `as` names the new array field
- → Unwinding, looking up and regrouping enriches every line of an order
- → Inside the lookup pipeline, stages run against the `orders` collection
- → `preserveNullAndEmptyArrays` keeps documents that found no match, like an outer join

SQL EQUIVALENT

- ⇄ `$lookup` ↔ `LEFT OUTER JOIN` (always left) · `$unwind` ↔ one row per child
- ⇄ Pipeline `$lookup` with `$limit` ↔ `LATERAL` join · See SQL 33, 37

ACROSS ENGINES

- **Redis** no joins: fetch both keys (pipelined) and combine in the app
- **DynamoDB** no joins: store related items under one partition key and `Query` once
- **Cassandra** no joins: copy data into a table per query
- **Neo4j** joins are traversals: `MATCH (c:Customer)-[:PLACED]->(o:Order)`

IN YOUR CODE

- **Node** `{ $lookup: { from: "customers", localField: "customerId", foreignField: "_id", as: "customer" } }` or Mongoose `.populate()`
- **Python** `{"$lookup": {"from": "customers", "localField": "customerId", "foreignField": "_id", "as": "customer"}}`
- **Java** `lookup("customers", "customerId", "_id", "customer")`

$LOOKUP FORMS

FORM	USE FOR
`localField / foreignField`	Equality join
`+ pipeline`	Equality join plus filter / sort / limit
`let + pipeline`	Non-equality conditions with $expr
`$graphLookup`	Recursive joins (trees, networks)

$UNWIND OPTIONS

OPTION	EFFECT
`"$arr"`	One document per element
`preserveNullAndEmptyArrays`	Keep docs with empty / missing arrays
`includeArrayIndex`	Add the element's position

⚠ COMMON MISTAKES

- No index on the foreign field
- Running `$lookup` before `$match`
- Unwinding large arrays and regrouping when a projection would do

✓ TIPS & BEST PRACTICES

- Index every `foreignField` you join on
- Prefer embedding for data read together on every request
- Use the pipeline form to cap joined arrays

QUICK SUMMARY

- `$lookup` — left join into an array
- `$unwind` — one doc per element
- Index the foreign field
- Frequent joins → embed

Sheet 20

$facet, $bucket & Window Stages

$facet · $bucket · $setWindowFields · $densify · $fill · $sortByCount

FACETS, BUCKETS AND WINDOWS

$facet runs several sub-pipelines over the same input in one pass, which suits search pages with filters and counts. **$bucket** and **$bucketAuto** group values into ranges. **$setWindowFields** (5.0+) adds window functions: running totals, ranks and moving averages without collapsing documents. **$densify** and **$fill** fill gaps in time series.

NOTES

- Each `$facet` output is one array inside one document (16 MB cap)
- `$sortByCount` = `$group` + `$sort` by count
- Windows can count documents or a time / value range

$FACET, $BUCKET, WINDOWS & GAP FILLING

```
// a search page: results plus counts in one pass
db.products.aggregate([
  { $match: { price: { $lt: 100 } } },
  { $facet: {
      byCategory: [ { $sortByCount: "$category" } ],
      byPrice: [ { $bucket: { groupBy: "$price", boundaries: [0, 20, 50, 100],
                              default: "other", output: { n: { $sum: 1 } } } } ],
      page: [ { $sort: { price: 1 } }, { $limit: 10 } ]
  } }
])

// even-sized buckets chosen by MongoDB
db.products.aggregate([ { $bucketAuto: { groupBy: "$price", buckets: 4 } } ])

// running total and order number per customer
db.orders.aggregate([
  { $setWindowFields: {
      partitionBy: "$customerId", sortBy: { orderDate: 1 },
      output: {
        runningTotal: { $sum: "$total", window: { documents: ["unbounded", "current"] } },
        orderNo: { $documentNumber: {} } } } }
])

// daily revenue with no missing days
db.dailySales.aggregate([
  { $densify: { field: "day", range: { step: 1, unit: "day", bounds: "full" } } },
  { $fill: { output: { revenue: { value: 0 } } } }
])
```

CODE EXPLAINED

- `$facet` returns one document with a `byCategory`, `byPrice` and `page` array
- `$bucket` boundaries are inclusive below, exclusive above; `default` collects the rest
- `documents: ["unbounded", "current"]` is everything from the first row to this one
- `$densify` inserts the missing days; `$fill` gives them a value

SQL EQUIVALENT

- `$facet` ↔ several queries or `GROUPING SETS` · `$bucket` ↔ `CASE` + `GROUP BY` (or `width_bucket`)
- `$setWindowFields` ↔ `OVER (PARTITION BY … ORDER BY …)` · `$densify` ↔ `generate_series` + `LEFT JOIN` · See SQL 44, 47, 79

ACROSS ENGINES

- **Redis** facet counts with `FT.AGGREGATE … GROUPBY`
- **DynamoDB** precompute buckets and running totals on write
- **Cassandra** no window functions; compute in Spark or the app
- **Neo4j** `collect` plus list functions; the GDS library for graph analytics

IN YOUR CODE

- **Node** `{ $facet: { byCategory: [{ $sortByCount: "$category" }] } }`
- **Python** `{"$facet": {"byCategory": [{"$sortByCount": "$category"}]}}`
- **Java** `facet(new Facet("byCategory", sortByCount("$category")))`

ANALYTICS STAGES

STAGE	DOES
`$facet`	Several pipelines, one pass
`$bucket / $bucketAuto`	Range groups
`$sortByCount`	Group and sort by count
`$setWindowFields`	Window functions
`$densify / $fill`	Fill gaps in series

WINDOW OPERATORS

OPERATOR	RESULT
`$sum / $avg`	Running or moving totals
`$rank / $denseRank`	Rank within partition
`$documentNumber`	Row number
`$shift`	Value from another row (lag / lead)
`$derivative / $integral`	Rate of change / area

⚠ COMMON MISTAKES

- Putting huge result sets inside `$facet` and hitting 16 MB
- Forgetting `sortBy` in `$setWindowFields`
- Bucket boundaries that don't cover all values without a `default`

✓ TIPS & BEST PRACTICES

- Limit each facet branch
- Use time-range windows (`range` with `unit`) for time series
- Pre-aggregate daily totals, then densify the small result

QUICK SUMMARY

- `$facet` — many results, one pass
- `$bucket` — ranges
- `$setWindowFields` — windows
- `$densify` / `$fill` — gaps

Expression Operators & Output

LOGIC INSIDE PIPELINES

Expression operators compute values inside `$project`, `$set`, `$group` and `$expr`: conditionals (`$cond`, `$switch`, `$ifNull`), strings (`$concat`, `$toUpper`), dates (`$dateTrunc`, `$dateToString`) and conversions (`$toString`, `$toDecimal`). To keep results, **`$out`** replaces a collection with the pipeline output and **`$merge`** upserts into one, which makes it the tool for incremental summaries. A **view** saves a pipeline as a read-only collection.

NOTES

- `$out` and `$merge` must be the last stage
- Views run their pipeline on every read; they store nothing
- `$merge` into the source collection is allowed
- "On-demand materialized view" = `$merge` on a schedule

EXPRESSIONS, $OUT, $MERGE & VIEWS

```
db.orders.aggregate([
  { $project: {
      total: 1,
      size: { $cond: { if: { $gte: ["$total", 100] }, then: "large", else: "small" } },
      label: { $switch: { branches: [
                 { case: { $eq: ["$status", "pending"] }, then: "Awaiting payment" },
                 { case: { $eq: ["$status", "shipped"] }, then: "On the way" } ],
               default: "Other" } },
      month: { $dateTrunc: { date: "$orderDate", unit: "month" } },
      ref: { $concat: ["ORD-", { $toString: "$_id" }] },
      coupon: { $ifNull: ["$couponCode", "none"] }
  } }
])

// replace a summary collection
db.orders.aggregate([
  { $group: { _id: "$customerId", spent: { $sum: "$total" } } },
  { $out: "customerTotals" }
])

// upsert into it instead (incremental refresh)
db.orders.aggregate([
  { $match: { orderDate: { $gte: ISODate("2026-11-01") } } },
  { $group: { _id: "$customerId", spent: { $sum: "$total" } } },
  { $merge: { into: "customerTotals", on: "_id",
              whenMatched: "replace", whenNotMatched: "insert" } }
])

// view: a saved, read-only pipeline
db.createView("pendingOrders", "orders", [ { $match: { status: "pending" } } ])
```

CODE EXPLAINED

- → `$cond` is if / then / else; `$switch` handles several cases
- → `$dateTrunc` rounds a date down to the start of the month
- → `$out` swaps in the new collection when the pipeline finishes
- → `whenMatched` can also be `"merge"`, `"keepExisting"`, `"fail"` or a pipeline

SQL EQUIVALENT

- ⇄ `$cond` ↔ `CASE WHEN` · `$ifNull` ↔ `COALESCE` · `$dateTrunc` ↔ `date_trunc`
- ⇄ `$out` ↔ `CREATE TABLE AS` · `$merge` ↔ `MERGE` · view ↔ `CREATE VIEW` · See SQL 21, 26, 64

ACROSS ENGINES

- **Redis** logic in Lua or the app; `APPLY` expressions in `FT.AGGREGATE`
- **DynamoDB** no server-side expressions beyond update and condition expressions
- **Cassandra** write to several tables, or materialized views (experimental)
- **Neo4j** `CASE WHEN`, `coalesce()`, `date.truncate('month', d)`

IN YOUR CODE

- **Node** `await db.createCollection("pendingOrders", { viewOn: "orders", pipeline: [...] })`
- **Python** `db.create_collection("pendingOrders", viewOn="orders", pipeline=[...])`
- **Java** `db.createView("pendingOrders", "orders", List.of(match(eq("status", "pending"))))`

EXPRESSION OPERATORS

KIND	EXAMPLES
Conditional	$cond, $switch, $ifNull
Comparison	$eq, $gt, $cmp
Arithmetic	$add, $multiply, $round
String	$concat, $toUpper, $substrCP, $split
Date	$dateTrunc, $dateDiff, $dateToString
Array	$size, $filter, $map, $reduce
Conversion	$toString, $toDecimal, $convert

$OUT VS $MERGE

	$OUT	$MERGE
Target	Replaced	Updated in place
Incremental	No	Yes
Other database	Yes	Yes
Sharded target	No	Yes

⚠ COMMON MISTAKES

- Using `$out` when other code still writes to that collection
- Treating a view as a cache; it recomputes every time
- Comparing values of different types inside `$cond`

✓ TIPS & BEST PRACTICES

- Refresh summaries with `$merge` on a schedule
- Use `$filter` / `$map` to work on arrays without `$unwind`
- Keep heavy expressions out of hot read paths

QUICK SUMMARY

- `$cond` / `$switch` — logic
- `$dateTrunc` — date buckets
- `$out` replaces · `$merge` upserts
- Views — saved pipelines

Index Types

INDEX TYPES

An **index** is a sorted structure (a B-tree) that lets MongoDB find documents without scanning the collection. Beyond single-field and **compound** indexes, an index on an array field becomes **multikey** (one entry per element). Options shape what gets indexed: **unique** rejects duplicates, **partial** indexes only matching documents, **TTL** deletes documents after a time, and **wildcard** covers fields you can't list in advance.

NOTES

- `_id` always has a unique index
- A compound index serves queries on its leading fields (prefixes)
- The TTL monitor runs about every 60 seconds
- `hideIndex` tests removal without dropping

CREATING EACH KIND

```
db.products.createIndex({ category: 1 })                    // single field
db.orders.createIndex({ customerId: 1, orderDate: -1 })      // compound
db.products.createIndex({ tags: 1 })                         // multikey (array)

db.customers.createIndex({ email: 1 }, { unique: true })
db.customers.createIndex({ email: 1 },                       // case-insensitive unique
  { unique: true, collation: { locale: "en", strength: 2 }, name: "email_ci" })

db.orders.createIndex({ orderDate: 1 },                      // partial
  { partialFilterExpression: { status: "pending" } })
db.sessions.createIndex({ lastSeen: 1 }, { expireAfterSeconds: 3600 })   // TTL
db.products.createIndex({ "attrs.$**": 1 })                  // wildcard
db.orders.createIndex({ customerId: "hashed" })              // hashed (sharding)
db.stores.createIndex({ location: "2dsphere" })              // geospatial

db.products.getIndexes()
db.products.hideIndex("category_1")
db.products.dropIndex("category_1")
```

CODE EXPLAINED

- → `1` and `-1` set the sort direction; it matters only for compound sorts
- → The partial index holds only pending orders, so it's small and fast for that query
- → TTL deletes a session an hour after its `lastSeen` date
- → Default index names join fields and directions: `customerId_1_orderDate_-1`

SQL EQUIVALENT

- ⇄ `createIndex` ↔ `CREATE INDEX` · `unique` ↔ `UNIQUE` · partial ↔ `CREATE INDEX … WHERE`
- ⇄ TTL has no SQL equivalent; the nearest is a scheduled `DELETE` · See SQL 71, 72

ACROSS ENGINES

- **Redis** no secondary indexes on keys; build them with sets / sorted sets or `FT.CREATE`
- **DynamoDB** GSIs and LSIs (sheet 45); 20 GSIs per table by default
- **Cassandra** SAI secondary indexes (5.0); otherwise a table per query
- **Neo4j** range, text, point, full-text and vector indexes on label properties

IN YOUR CODE

- **Node** `await orders.createIndex({ customerId: 1, orderDate: -1 })`
- **Python** `orders.create_index([("customerId", 1), ("orderDate", -1)])`
- **Java** `orders.createIndex(compoundIndex(ascending("customerId"), descending("orderDate")))`

INDEX KINDS

KIND	USE FOR
`Single field`	One-field filters and sorts
`Compound`	Several fields; order matters
`Multikey`	Arrays
`Text`	Simple word search
`Wildcard`	Unknown or varied fields
`Hashed`	Even shard distribution
`2dsphere`	Geo queries

INDEX OPTIONS

OPTION	EFFECT
`unique`	Reject duplicates
`partialFilterExpression`	Index only matching docs
`sparse`	Skip docs missing the field
`expireAfterSeconds`	TTL deletion
`collation`	Case / accent rules
`hidden`	Ignored by the planner

⚠ COMMON MISTAKES

- One index per field instead of one compound index for the query
- A unique index on a field many documents lack (use partial)
- Expecting TTL deletes to happen at the exact second

✓ TIPS & BEST PRACTICES

- Name indexes explicitly when they carry options
- Prefer partial over sparse indexes
- Hide an index for a week before dropping it

QUICK SUMMARY

- Compound — order matters
- Multikey — arrays
- Unique · partial · TTL
- Hide before drop

Sheet 23

Index Strategy & explain()

ESR rule · covered queries · `explain("executionStats")` · COLLSCAN vs IXSCAN · `hint`

DESIGN INDEXES FOR QUERIES

Build compound indexes with the **ESR rule**: **E**quality fields first, then **S**ort fields, then **R**ange fields. Check every important query with **`explain("executionStats")`**: an **IXSCAN** with keys examined close to documents returned is healthy, a **COLLSCAN** means a full scan. A **covered query** is answered from the index alone, with zero documents examined.

NOTES

- `queryPlanner` shows the plan; `executionStats` also runs it
- The planner caches the winning plan per query shape
- `$indexStats` shows how often each index is used
- Atlas Performance Advisor suggests indexes

PRODUCTION CHECK

- Every index slows writes and uses RAM; drop unused ones (check `$indexStats`) on write-heavy collections
- Order compound fields by ESR; a range field before a sort field forces an in-memory sort
- Build large indexes off-peak: builds hold an exclusive lock briefly at the start and end

ESR, EXPLAIN() & COVERED QUERIES

```
// query: a customer's pending orders over $20, newest first
db.orders.find({ customerId: 42, status: "pending", total: { $gt: 20 } })
         .sort({ orderDate: -1 })

// ESR: Equality (customerId, status) → Sort (orderDate) → Range (total)
db.orders.createIndex({ customerId: 1, status: 1, orderDate: -1, total: 1 })

db.orders.find({ customerId: 42, status: "pending", total: { $gt: 20 } })
  .sort({ orderDate: -1 })
  .explain("executionStats")
// winningPlan.inputStage.stage: "IXSCAN"   (COLLSCAN = full scan)
// nReturned: 3, totalKeysExamined: 4, totalDocsExamined: 3

// covered: every field comes from the index
db.orders.find({ customerId: 42 }, { _id: 0, customerId: 1, orderDate: 1 })
// totalDocsExamined: 0

// force a plan while testing
db.orders.find({ customerId: 42 }).hint({ customerId: 1, status: 1, orderDate: -1, total: 1 })

// how often is each index used?
db.orders.aggregate([ { $indexStats: {} } ])
```

CODE EXPLAINED

- → Equality fields narrow to one slice of the index; the sort field then reads in order; the range filters last
- → Keys examined ≈ returned means the index fits the query
- → A covered query must exclude `_id` unless `_id` is in the index
- → `hint` is for testing; let the planner choose in production

SQL EQUIVALENT

- ⇄ `explain()` ↔ `EXPLAIN ANALYZE` · COLLSCAN ↔ Seq Scan · IXSCAN ↔ Index Scan
- ⇄ Covered query ↔ Index Only Scan · See SQL 73, 74

ACROSS ENGINES

- **Redis** `SLOWLOG GET`, `LATENCY DOCTOR`, `redis-cli --bigkeys`
- **DynamoDB** no planner: a `Query` uses the key you name, or it's a `Scan`
- **Cassandra** `TRACING ON` in cqlsh shows replicas contacted and timings
- **Neo4j** `EXPLAIN` / `PROFILE`: NodeIndexSeek vs NodeByLabelScan

IN YOUR CODE

- **Node** `await orders.find(q).explain("executionStats")`
- **Python** `orders.find(q).explain()`
- **Java** `orders.find(q).explain(ExplainVerbosity.EXECUTION_STATS)`

PLAN STAGES

STAGE	MEANING
`COLLSCAN`	Read every document
`IXSCAN`	Walk an index
`FETCH`	Load documents after IXSCAN
`SORT`	In-memory sort (no index order)
`PROJECTION_COVERED`	Answered from the index

HEALTHY NUMBERS

METRIC	AIM FOR
`totalKeysExamined / nReturned`	Close to 1
`totalDocsExamined / nReturned`	Close to 1 (0 if covered)
`executionTimeMillis`	Stable under load
`Stage SORT`	Absent on big results

⚠ COMMON MISTAKES

- Putting the range field before the sort field
- Adding an index for every query instead of sharing compound prefixes
- Judging speed on a tiny dev dataset

✓ TIPS & BEST PRACTICES

- Explain with production-sized data
- Check `$indexStats` before adding another index
- Turn on the profiler (`db.setProfilingLevel(1, { slowms: 100 })`) to find slow queries

QUICK SUMMARY

- ESR — Equality, Sort, Range
- IXSCAN good, COLLSCAN bad
- Keys ≈ returned
- Covered — 0 docs examined

Text, Atlas Search & Vector Search

WORDS, RELEVANCE AND MEANING

A classic **text index** supports simple word search with `$text`. **Atlas Search** (`$search`) runs Apache Lucene beside the database for real relevance: fuzzy matching, autocomplete, synonyms and facets. **Vector Search** (`$vectorSearch`) finds documents whose **embeddings** are closest in meaning to a query vector, which is the base for semantic search and RAG. Both started in Atlas; MongoDB 8.2 also offers them in self-managed deployments as a preview.

NOTES

- One text index per collection
- Search indexes are created with `createSearchIndex`, not `createIndex`
- Search indexes update asynchronously, a little behind writes
- `$search` / `$vectorSearch` must be the first stage

$TEXT, $SEARCH & $VECTORSEARCH

```
// classic text index
db.products.createIndex({ name: "text", description: "text" })
db.products.find({ $text: { $search: "usb cable -lightning" } },
                 { score: { $meta: "textScore" } })
  .sort({ score: { $meta: "textScore" } })

// Atlas Search: Lucene relevance, typo tolerance
db.products.aggregate([
  { $search: { index: "products",
               text: { query: "keybord", path: ["name", "description"],
                       fuzzy: { maxEdits: 1 } } } },
  { $limit: 10 },
  { $project: { name: 1, score: { $meta: "searchScore" } } }
])

// Vector Search: index once, then query by embedding
db.products.createSearchIndex("product_vectors", "vectorSearch", {
  fields: [ { type: "vector", path: "embedding", numDimensions: 384, similarity: "cosine" },
            { type: "filter", path: "category" } ] })

db.products.aggregate([
  { $vectorSearch: { index: "product_vectors", path: "embedding",
                     queryVector: embed("cable for my laptop"),
                     numCandidates: 100, limit: 5,
                     filter: { category: "Electronics" } } },
  { $project: { name: 1, score: { $meta: "vectorSearchScore" } } }
])
```

CODE EXPLAINED

- `-lightning` excludes a word; quotes would search a phrase
- `fuzzy: { maxEdits: 1 }` finds "keyboard" from "kebord"
- `embed()` stands for your embedding model call; the vector length must match `numDimensions`
- `numCandidates` trades accuracy for speed; 10–20× `limit` is typical

SQL EQUIVALENT

- `$text` ↔ `to_tsvector(…) @@ to_tsquery(…)` with a GIN index
- `$vectorSearch` ↔ pgvector `ORDER BY embedding <=> $1 LIMIT 5` · See SQL 80

IN YOUR CODE

- **Node** `await products.createSearchIndex({ name: "product_vectors", type: "vectorSearch", definition })`
- **Python** `products.create_search_index(SearchIndexModel(definition=d, name="product_vectors", type="vectorSearch"))`
- **Java** `products.createSearchIndexes(List.of(new SearchIndexModel("product_vectors", def, SearchIndexType.vectorSearch())))`

ACROSS ENGINES

- **Redis** `FT.CREATE` with TEXT and VECTOR fields, then `FT.SEARCH` (sheet 40)
- **DynamoDB** no full-text; zero-ETL integration streams tables into OpenSearch
- **Cassandra** SAI vector search: `ORDER BY v ANN OF [...] LIMIT 5`
- **Neo4j** full-text indexes (Lucene) and vector indexes on nodes

WHICH SEARCH?

FEATURE	$TEXT	$SEARCH	$VECTORSEARCH
Engine	B-tree index	Lucene	HNSW
Relevance	Basic score	BM25, boosts	Similarity
Typos	No	Fuzzy	Meaning, not spelling
Autocomplete / facets	No	Yes	No
Index command	createIndex	createSearchIndex	createSearchIndex

VECTOR SIMILARITY

FUNCTION	USE WHEN
cosine	Direction matters, length doesn't (most text)
dotProduct	Vectors already normalized; fastest
euclidean	Distance in space (images, geo-like)

⚠ COMMON MISTAKES

- Using `$text` and expecting fuzzy matching
- Embedding queries with a different model than the documents
- Forgetting the vector filter field in the index definition

✓ TIPS & BEST PRACTICES

- Combine `$vectorSearch` and `$search` for hybrid ranking
- Store the model name and version with each embedding
- Test `numCandidates` against a set of known good answers

QUICK SUMMARY

- `$text` — simple words
- `$search` — Lucene relevance
- `$vectorSearch` — meaning
- First stage in the pipeline

Sheet 25

Schema Design Patterns

bucket · outlier · subset · computed · extended reference · polymorphic · schema versioning

PATTERNS FOR COMMON PROBLEMS

MongoDB's **schema design patterns** are named answers to recurring problems. **Bucket** groups many small records into one document. **Subset** keeps the most-used part of a large list on the parent. **Computed** stores totals on write instead of calculating them on every read. **Outlier** handles the rare oversized document. **Polymorphic** keeps different shapes in one collection, and **schema versioning** lets old and new shapes live side by side during a migration.

NOTES

- Time series collections now do the bucket pattern for you
- Extended reference is on sheet 05
- Patterns combine: subset + computed is common on product pages

SIX PATTERNS

```
// Bucket: one document per sensor per hour
{ sensor: "s1", hour: ISODate("2026-11-07T10:00Z"), count: 3,
  readings: [ { t: 0, v: 21.4 }, { t: 60, v: 21.6 }, { t: 120, v: 21.5 } ] }

// Subset: newest 10 reviews on the product, the rest in `reviews`
{ _id: 7, name: "SQL Pocket Guide", reviewCount: 214,
  recentReviews: [ { stars: 5, text: "Clear" }, { stars: 4, text: "Handy" } ] }

// Computed: keep totals current on each write
db.products.updateOne({ _id: 7 },
  { $inc: { reviewCount: 1, starSum: 5 },
    $push: { recentReviews: { $each: [newReview], $position: 0, $slice: 10 } } })

// Outlier: flag the rare document that overflows
{ _id: 7, buyers: [ /* first 1000 ids */ ], hasOverflow: true }

// Polymorphic: one collection, a type field, different shapes
{ _id: 21, kind: "book",  name: "SQL Pocket Guide", isbn: "978-0-00-000000-0", pages: 360 }
{ _id: 22, kind: "cable", name: "USB-C Cable", lengthM: 2, connector: "USB-C" }

// Schema versioning: tag each document, upgrade lazily
{ _id: 42, schemaVersion: 2, name: { first: "Ada", last: "Lovelace" } }
```

CODE EXPLAINED

- Buckets cut document count and index size for high-volume small records
- `$position: 0` + `$slice: 10` keeps the newest 10 reviews at the front
- Average rating is `starSum / reviewCount`, computed without reading reviews
- Readers check `schemaVersion` and convert old shapes (sheet 59)

SQL EQUIVALENT

- Subset ↔ `LIMIT` in a query · computed ↔ triggers or materialized views · polymorphic ↔ single-table inheritance
- Schema versioning ↔ migrations with `ALTER TABLE` · See SQL 63, 64

ACROSS ENGINES

- **Redis** computed: counters with `INCR`; subset: capped lists with `LTRIM`
- **DynamoDB** polymorphic items are normal (single-table design); computed with `ADD`
- **Cassandra** bucket partitions by time: `PRIMARY KEY ((sensor, day), ts)`
- **Neo4j** intermediate nodes split huge fans; extra labels give polymorphism

IN YOUR CODE

- **Node** Mongoose discriminators: `Product.discriminator("Book", bookSchema)`
- **Python** Beanie: `class Book(Product)` with `is_root = True` on the base
- **Java** Spring Data: `@TypeAlias("book")` `class Book extends Product`

PATTERNS

PATTERN	PROBLEM	FIX
Bucket	Millions of tiny docs	Group by time / owner
Subset	Huge lists on parent	Keep the top N
Computed	Repeated heavy math	Store results on write
Outlier	Rare oversized docs	Overflow flag + extra docs
Polymorphic	Similar things, different fields	One collection + type field
Schema versioning	Changing shape	Version field + upgrade

SIGNALS

YOU SEE…	CONSIDER
Arrays growing without limit	Subset or bucket
$group on every page load	Computed
A few docs near 16 MB	Outlier
A $lookup on every read	Extended reference

⚠ COMMON MISTAKES

- Applying patterns before measuring a problem
- Computed values that drift because one write path forgets to update them
- Polymorphic collections without a type field

✓ TIPS & BEST PRACTICES

- Update computed fields in the same operation as the change
- Index the type field in polymorphic collections
- Document which patterns each collection uses

QUICK SUMMARY

- Bucket — group small records
- Subset — keep the top N
- Computed — totals on write
- Versioning — evolve safely

Schema Validation

RULES ON THE SERVER

Schema validation makes MongoDB check documents on insert and update with a **$jsonSchema** validator: required fields, BSON types, ranges, patterns and allowed values. **validationLevel** decides which documents are checked (`strict`: all; `moderate`: only those already valid) and **validationAction** decides what happens to a bad write (`error` rejects it, `warn` logs it). Change the rules later with **collMod**.

NOTES

- Validation doesn't touch existing documents until they're updated
- Failed writes report which rule failed (`errInfo`)
- Use `bypassDocumentValidation` only for repair jobs
- App-side validation still gives better error messages

$JSONSCHEMA VALIDATOR

```
db.createCollection("customers", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["name", "email", "createdAt"],
    properties: {
      name:      { bsonType: "string", minLength: 1, maxLength: 100 },
      email:     { bsonType: "string", pattern: "^[^@\\s]+@[^@\\s]+$" },
      country:   { enum: ["US", "CA", "GB", "DE"] },
      createdAt: { bsonType: "date" },
      credit:    { bsonType: "decimal", minimum: 0 },
      addresses: { bsonType: "array", maxItems: 5,
                   items: { bsonType: "object", required: ["city"] } }
    }
  } },
  validationLevel: "strict",
  validationAction: "error"
})

// change the rules on an existing collection
db.runCommand({ collMod: "customers",
  validator: { $jsonSchema: newSchema },
  validationLevel: "moderate" })

// find documents that break the rules
db.customers.find({ $nor: [ { $jsonSchema: newSchema } ] })
db.getCollectionInfos({ name: "customers" })[0].options.validator
```

CODE EXPLAINED

- → `required` lists fields that must exist; `properties` sets rules for each
- → `enum` limits a field to listed values
- → `moderate` lets old invalid documents be updated without fixing them first
- → The `$nor` query lists documents to clean up before switching to `strict`

SQL EQUIVALENT

- ⇄ `$jsonSchema` ↔ column types, `NOT NULL` and `CHECK` constraints · `enum` ↔ `CHECK (country IN …)`
- ⇄ `moderate` ↔ adding a constraint `NOT VALID` · See SQL 57

ACROSS ENGINES

- **Redis** no validation; validate in the app
- **DynamoDB** only key attributes are typed; validate in code
- **Cassandra** typed columns enforce types; no `CHECK` constraints
- **Neo4j** uniqueness and key constraints; existence and type constraints in Enterprise

IN YOUR CODE

- **Node** Mongoose: `new Schema({ email: { type: String, required: true, match: /@/ } })`
- **Python** Pydantic models (Beanie / ODMantic) validate before writing
- **Java** Bean Validation: `@NotBlank @Email String email;` on the document class

$JSONSCHEMA KEYWORDS

KEYWORD	CHECKS
`bsonType`	BSON type (or list of types)
`required`	Fields that must exist
`enum`	Allowed values
`minimum / maximum`	Number range
`minLength / pattern`	String length / regex
`minItems / items`	Array size / element rules
`additionalProperties`	Allow unknown fields?

LEVEL & ACTION

SETTING	EFFECT
`strict`	Check every insert and update
`moderate`	Skip updates to already-invalid docs
`off`	No checks
`error`	Reject invalid writes
`warn`	Allow, but log

⚠ COMMON MISTAKES

- Turning on `strict` without checking existing data
- Using `type` instead of `bsonType` for BSON-specific types like decimal
- `additionalProperties: false` without listing `_id`

✓ TIPS & BEST PRACTICES

- Start with `warn`, watch the logs, then switch to `error`
- Keep the schema in version control and apply it with migrations
- Validate in the app too, for friendly messages

QUICK SUMMARY

- `$jsonSchema` — server rules
- `strict` / `moderate`
- `error` / `warn`
- `collMod` — change later

Sheet 27

Transactions, Read & Write Concerns

single-document atomicity · `startSession` · `withTransaction` · write concern `majority` · read concern · read preference

ATOMIC ACROSS DOCUMENTS

Every write to **one document** is atomic, which covers most needs when related data is embedded. For changes across documents or collections, MongoDB has **multi-document transactions** on replica sets and sharded clusters: start a **session**, run the operations, then commit or abort. **Write concern** (how many replicas confirm), **read concern** (how committed the data must be) and **read preference** (which member to read) tune durability and freshness.

NOTES

- `withTransaction()` retries transient errors and commits for you
- Pass the session to every operation inside the transaction
- Transactions can't create collections on sharded clusters
- Prefer a schema that needs one document

PRODUCTION CHECK

- Transactions need a replica set or sharded cluster; a standalone server rejects them. Run a one-node replica set locally
- Keep transactions short: they abort after 60 s by default and hold cache while open
- Retry on `TransientTransactionError` and `UnknownTransactionCommitResult`; `withTransaction()` does this

SINGLE-DOCUMENT ATOMICITY & TRANSACTIONS

```
// one document: already atomic, no transaction needed
db.products.updateOne({ _id: 7, stock: { $gte: 2 } }, { $inc: { stock: -2 } })

// several documents: a transaction
const session = db.getMongo().startSession()
const shop = session.getDatabase("shop")
session.withTransaction(() => {
  const r = shop.products.updateOne({ _id: 7, stock: { $gte: 2 } },
                                    { $inc: { stock: -2 } })
  if (r.modifiedCount === 0) throw new Error("out of stock")   // aborts
  shop.orders.insertOne({ _id: 1002, customerId: 42, status: "pending",
    items: [ { productId: 7, qty: 2, unitPrice: NumberDecimal("19.99") } ] })
}, { readConcern: { level: "snapshot" }, writeConcern: { w: "majority" } })
session.endSession()

// concerns outside transactions
db.orders.insertOne(doc, { writeConcern: { w: "majority", j: true } })
db.orders.find().readConcern("majority")
db.orders.find().readPref("secondaryPreferred")
```

CODE EXPLAINED

- → The filter `stock: { $gte: 2 }` makes the decrement conditional and race-free
- → Throwing inside the callback aborts and rolls back both writes
- → `snapshot` reads see one consistent point in time for the whole transaction
- → `secondaryPreferred` reads may be slightly stale

SQL EQUIVALENT

- ⇄ `startSession` + `withTransaction` ↔ `BEGIN … COMMIT`; throwing ↔ `ROLLBACK`
- ⇄ `snapshot` read concern ↔ snapshot isolation (`REPEATABLE READ` in PostgreSQL) · See SQL 68, 69

ACROSS ENGINES

- **Redis** `MULTI` / `EXEC` without rollback (sheet 41)
- **DynamoDB** `TransactWriteItems`: up to 100 actions, all-or-nothing
- **Cassandra** lightweight transactions per partition only
- **Neo4j** every query runs in a transaction; `session.executeWrite()` retries

IN YOUR CODE

- **Node** `await session.withTransaction(async () => { await products.updateOne(f, u, { session }) })`
- **Python** `with client.start_session() as s: s.with_transaction(lambda s: products.update_one(f, u, session=s))`
- **Java** `session.withTransaction(() -> { products.updateOne(session, f, u); return null; })`

WRITE CONCERN

VALUE	CONFIRMS WHEN
`w: 1`	The primary has it
`w: "majority"`	Most voting members have it (default)
`j: true`	It's in the journal on disk
`wtimeout`	Stop waiting after ms

READ CONCERN & PREFERENCE

SETTING	MEANING
`local`	Latest on that member (default)
`majority`	Acknowledged by a majority
`snapshot`	One point in time (transactions)
`primary`	Read from the primary
`secondaryPreferred`	Secondary if available
`nearest`	Lowest latency member

⚠ COMMON MISTAKES

- Forgetting to pass `session` to an operation, which then runs outside the transaction
- Long transactions that do slow work (HTTP calls) inside
- Using transactions to patch a schema that should embed

✓ TIPS & BEST PRACTICES

- Model so most changes touch one document
- Use conditional updates for stock and balances
- Keep transactions under a few hundred milliseconds

QUICK SUMMARY

- One document — always atomic
- Sessions + `withTransaction()`
- Write concern — durability
- Read concern / preference — freshness

Change Streams

REACT TO EVERY CHANGE

A **change stream** pushes inserts, updates, replaces and deletes to your app as they happen, built on the replication log (oplog). Open one with **watch()** on a collection, database or whole deployment, filter it with a pipeline, and save each event's **resume token** so you can continue after a restart without missing events. Use it for event-driven sync: updating caches, search indexes or other services.

NOTES

- Requires a replica set or sharded cluster
- Resume tokens work while the event is still in the oplog
- `fullDocument: "updateLookup"` fetches the current document for updates
- Events for one document arrive in order

WATCH(), FILTERS & RESUME TOKENS

```
// inserts and updates on orders, with the full document
const cs = db.orders.watch(
  [ { $match: { operationType: { $in: ["insert", "update"] } } } ],
  { fullDocument: "updateLookup" })

while (!cs.isClosed()) {
  const e = cs.tryNext()
  if (!e) { sleep(500); continue }
  printjson({ op: e.operationType, id: e.documentKey._id,
              status: e.fullDocument?.status })
  saveToken(e._id)                        // resume token
}

// continue after a crash
db.orders.watch([], { resumeAfter: loadToken() })

// start from a moment in time
db.orders.watch([], { startAtOperationTime: Timestamp({ t: 1793491200, i: 0 }) })

// pre-images: the document before the change (6.0+)
db.runCommand({ collMod: "orders", changeStreamPreAndPostImages: { enabled: true } })
db.orders.watch([], { fullDocumentBeforeChange: "whenAvailable" })

// a whole database or deployment
db.watch()
db.getMongo().watch()
```

CODE EXPLAINED

- → `documentKey` holds the `_id` (and shard key) of the changed document
- → An update event carries `updateDescription` with changed and removed fields
- → `saveToken` / `loadToken` stand for your own storage, such as a small collection
- → Pre-images must be enabled per collection before you can read them

SQL EQUIVALENT

- ⇄ Change streams ↔ logical replication / logical decoding, or `LISTEN` / `NOTIFY` for simple cases
- ⇄ Triggers run inside the database; change streams run in your app · See SQL 67

ACROSS ENGINES

- **Redis** keyspace notifications (`notify-keyspace-events`) or Streams
- **DynamoDB** DynamoDB Streams (24 h) or Kinesis Data Streams
- **Cassandra** CDC commit-log files, usually read by Debezium
- **Neo4j** CDC (Enterprise and Aura) via `db.cdc.query`

IN YOUR CODE

- **Node** `for await (const e of orders.watch(pipeline)) handle(e)`
- **Python** `with orders.watch(pipeline) as s: for e in s: handle(e)`
- **Java** `orders.watch(pipeline).forEach(e -> handle(e));`

EVENT FIELDS

FIELD	HOLDS
`_id`	Resume token
`operationType`	insert, update, replace, delete, …
`documentKey`	`_id` of the document
`fullDocument`	The document (insert, or with updateLookup)
`updateDescription`	Changed / removed fields
`clusterTime`	When it happened

OPTIONS

OPTION	EFFECT
`fullDocument`	`updateLookup`, `whenAvailable`, `required`
`resumeAfter`	Continue after a token
`startAfter`	Like resumeAfter, also after invalidate
`startAtOperationTime`	Begin at a timestamp
`fullDocumentBeforeChange`	Include pre-images

⚠ COMMON MISTAKES

- Not saving resume tokens, so a restart misses events
- Doing slow work inline and falling off the end of the oplog
- Assuming `updateLookup` returns the document as of the event (it's the current one)

✓ TIPS & BEST PRACTICES

- Store the token after processing each event
- Make handlers idempotent; events can repeat after a resume
- Filter with `$match` on the server, not in the app

QUICK SUMMARY

- `watch()` — live changes
- Filter with a pipeline
- Save resume tokens
- Replica set required

Replica Sets & Sharding

primary / secondary · elections · oplog · shard key · chunks · `mongos` · hashed vs ranged

COPIES AND SLICES

A **replica set** is a group of `mongod` servers holding the same data: one **primary** takes writes and **secondaries** copy them from the **oplog**. If the primary fails, members hold an **election** and promote a secondary in seconds. **Sharding** splits a collection across several replica sets (**shards**) by a **shard key**; the **`mongos`** router sends each query to the right shards. Hashed keys spread writes evenly; ranged keys keep nearby values together.

NOTES

- Use an odd number of voting members (3 or 5)
- Drivers discover the primary and follow elections
- Data moves between shards in ranges (chunks) by the balancer
- Since 6.0 `shardCollection` enables sharding for the database

PRODUCTION CHECK

- The shard key is hard to change: resharding (5.0+) works but copies the whole collection. Choose it from your top queries
- A steadily increasing key (ObjectId, timestamps) sends every insert to one shard; use a hashed or compound key
- Queries without the shard key go to every shard (scatter-gather)

REPLICA SET & SHARDING COMMANDS

```
# three-member replica set for local testing
docker network create mongo-net
for n in m1 m2 m3; do
  docker run -d --name $n --net mongo-net mongo:8 --replSet rs0 --bind_ip_all
done
mongosh --host m1 --eval 'rs.initiate({ _id: "rs0", members: [
  { _id: 0, host: "m1:27017" }, { _id: 1, host: "m2:27017" },
  { _id: 2, host: "m3:27017" } ] })'

# in mongosh
rs.status()                              # PRIMARY / SECONDARY per member
rs.printSecondaryReplicationInfo()       # replication lag
rs.stepDown()                            # force an election

# sharding: connect to mongos
sh.shardCollection("shop.orders", { customerId: "hashed" })
sh.shardCollection("shop.events", { tenantId: 1, ts: 1 })     # ranged
sh.status()
db.orders.getShardDistribution()
db.adminCommand({ reshardCollection: "shop.orders",
                  key: { customerId: 1, orderDate: 1 } })
```

CODE EXPLAINED

- → `--replSet rs0` names the set; `rs.initiate` lists its members
- → A hashed key on `customerId` spreads customers evenly across shards
- → A compound ranged key keeps one tenant's events together and in time order
- → `getShardDistribution` shows how evenly data is spread

SQL EQUIVALENT

- ⇄ Replica set ↔ PostgreSQL streaming replication plus automatic failover (for example Patroni)
- ⇄ PostgreSQL has no built-in sharding; the nearest is Citus · See SQL 74

ACROSS ENGINES

- **Redis** replicas + Sentinel for failover; Cluster shards by 16,384 hash slots
- **DynamoDB** automatic: partitions split by size and traffic; you only choose the key
- **Cassandra** every node is equal; consistent hashing over the token ring
- **Neo4j** clusters replicate; scale-out reads via secondaries, sharding via composite databases (Enterprise)

IN YOUR CODE

- **Node** `new MongoClient("mongodb://m1,m2,m3/?replicaSet=rs0")`
- **Python** `MongoClient("mongodb://m1,m2,m3/?replicaSet=rs0&readPreference=secondaryPreferred")`
- **Java** `MongoClients.create("mongodb://m1,m2,m3/?replicaSet=rs0&w=majority")`

REPLICA SET MEMBERS

ROLE	DOES
`Primary`	Accepts writes
`Secondary`	Copies the oplog; can serve reads
`Arbiter`	Votes only, no data (avoid if you can)
`Hidden / delayed`	Backups, analytics, undo window

SHARD KEY CHOICES

KEY	EFFECT
`Hashed _id`	Even writes; range queries hit all shards
`Ranged ts`	Hot last shard
`Compound { tenantId, ts }`	Spread by tenant, ordered within
`Low cardinality (status)`	Few chunks, uneven

⚠ COMMON MISTAKES

- Two-member replica sets that can't elect a primary when one fails
- Sharding before you need it
- Picking a shard key nobody filters on

✓ TIPS & BEST PRACTICES

- Always connect with the replica set name so drivers follow failover
- Test failover with `rs.stepDown()` before production
- Shard on the field your most frequent queries include

QUICK SUMMARY

- Replica set — primary + secondaries
- Elections fail over in seconds
- Shard key decides the shard
- Hashed for even writes

Security & Backup

USERS, NETWORK, BACKUPS

Turn on **authentication** and give each app its own **user** with only the **roles** it needs, scoped to a database. Limit network access with `bindIp` and firewalls (Atlas: IP access lists or private endpoints) and encrypt traffic with **TLS**. For backups, **`mongodump`** / **`mongorestore`** handle small to medium data; larger deployments use filesystem snapshots or Atlas continuous backups with point-in-time restore.

USERS, CONFIG & BACKUPS

```
# create users (in mongosh, as an admin)
use admin
db.createUser({ user: "shopApp", pwd: passwordPrompt(),
  roles: [ { role: "readWrite", db: "shop" } ] })
db.createUser({ user: "analyst", pwd: passwordPrompt(),
  roles: [ { role: "read", db: "shop" } ] })
db.createRole({ role: "orderReader", roles: [], privileges: [
  { resource: { db: "shop", collection: "orders" }, actions: ["find"] } ] })
db.grantRolesToUser("analyst", [ { role: "orderReader", db: "admin" } ])

# mongod.conf
security:
  authorization: enabled
net:
  bindIp: 127.0.0.1,10.0.0.5
  tls:
    mode: requireTLS
    certificateKeyFile: /etc/ssl/mongo.pem

# backup and restore
mongodump --uri "mongodb://admin@localhost:27017/?authSource=admin" \
  --db shop --gzip --archive=shop-2026-11-07.gz
mongorestore --uri "mongodb://admin@localhost:27017/?authSource=admin" \
  --gzip --archive=shop-2026-11-07.gz --drop
mongoexport --db shop --collection orders --type=csv \
  --fields _id,status,total --out orders.csv
```

NOTES

- Since 3.6, `mongod` listens only on localhost by default
- SCRAM-SHA-256 is the default password mechanism
- Database Tools are a separate download from the server
- Test restores, not just backups

CODE EXPLAINED

- → `passwordPrompt()` keeps the password out of shell history
- → A custom role grants exactly `find` on one collection
- → `--archive` writes one file; `--gzip` compresses it
- → `--drop` replaces collections on restore instead of merging

SQL EQUIVALENT

- ⇄ `createUser` ↔ `CREATE ROLE … LOGIN` · roles ↔ `GRANT` · `mongodump` ↔ `pg_dump`
- ⇄ Atlas point-in-time restore ↔ PostgreSQL WAL archiving and PITR · See SQL 75, 77

ACROSS ENGINES

- **Redis** ACL users (`ACL SETUSER`), TLS; back up RDB / AOF files
- **DynamoDB** IAM policies per table or item; PITR (35 days) and on-demand backups
- **Cassandra** `CREATE ROLE` , `GRANT` ; `nodetool snapshot`
- **Neo4j** role-based access (fine-grained in Enterprise); `neo4j-admin database dump` / `backup`

IN YOUR CODE

- **Node** `new MongoClient(uri, { tls: true, auth: { username, password } })`
- **Python** `MongoClient(uri, tls=True, username=u, password=p)`
- **Java** `MongoCredential.createScramSha256Credential(u, "admin", pw)` in `MongoClientSettings`

BUILT-IN ROLES

ROLE	GRANTS
`read` / `readWrite`	Query / query and modify one database
`dbAdmin`	Indexes, stats, validation
`userAdmin`	Manage users and roles
`clusterMonitor`	Read-only monitoring
`root`	Everything (avoid for apps)

BACKUP OPTIONS

METHOD	GOOD FOR
`mongodump`	Small data, single collections
`Filesystem snapshot`	Large data, fast restore
`Atlas backups`	Continuous, point-in-time
`Delayed secondary`	Undo window, not a backup

⚠ COMMON MISTAKES

- Running with `authorization` off on a reachable server
- Apps connecting as `root`
- Never testing a restore

✓ TIPS & BEST PRACTICES

- One user per app, least privilege
- Keep connection strings in secrets, not code
- Schedule restore drills

QUICK SUMMARY

- Auth on, users per app
- Roles scoped to databases
- TLS + bindIp
- Dump, snapshot or Atlas PITR

Drivers & ODMs

Node driver · Mongoose schemas · PyMongo · Motor · Spring Data MongoDB · Prisma MongoDB

FROM YOUR APP

Official **drivers** exist for every major language and expose the same methods as mongosh. Create **one client per application** and reuse it: it holds a connection pool and tracks the replica set. **ODMs** (object-document mappers) such as Mongoose, Beanie and Spring Data MongoDB add schemas, validation, hooks and model classes on top. Never pass raw request JSON as a filter: `{ "$ne": null }` in a password field is a NoSQL injection.

NOTES

- Default pool size is 100 connections per client
- Retryable writes and reads are on by default
- Motor is deprecated; use PyMongo's async API
- Prisma has a MongoDB provider

NODE: DRIVER AND MONGOOSE

```
// npm i mongodb mongoose
import { MongoClient } from "mongodb"
const client = new MongoClient(process.env.MONGODB_URI, { appName: "shop-api" })
const orders = client.db("shop").collection("orders")        // reuse everywhere
const pending = await orders.find({ status: "pending" }).limit(20).toArray()

// Mongoose: schemas, validation, middleware, populate
import mongoose, { Schema } from "mongoose"
await mongoose.connect(process.env.MONGODB_URI)

const itemSchema = new Schema({
  productId: Number, qty: { type: Number, min: 1 },
  unitPrice: Schema.Types.Decimal128 }, { _id: false })

const Order = mongoose.model("Order", new Schema({
  customer: { type: Schema.Types.ObjectId, ref: "Customer", required: true },
  status: { type: String, enum: ["pending", "paid", "shipped"], default: "pending" },
  items: [itemSchema]
}, { timestamps: true }))

await Order.create({ customer: custId, items: [ { productId: 7, qty: 2, unitPrice: "19.99" } ] })
const list = await Order.find({ status: "pending" })
  .populate("customer", "name").lean()

// block operator injection from request input
const status = String(req.query.status)          // never pass objects through
```

CODE EXPLAINED

- → The driver connects lazily on the first operation
- → `timestamps: true` adds `createdAt` and `updatedAt`
- → `.populate("customer", "name")` runs a second query and fills in names
- → `.lean()` returns plain objects, faster than full Mongoose documents

SQL EQUIVALENT

- ⇄ Driver ↔ JDBC / `pg` · ODM ↔ ORM (Hibernate, Prisma) · `populate` ↔ a join done in the app
- ⇄ Operator injection is NoSQL's cousin of SQL injection · See SQL 76

ACROSS ENGINES

- **Redis** node-redis / ioredis, redis-py, Jedis / Lettuce; Redis OM for object mapping
- **DynamoDB** AWS SDK v3, boto3, SDK v2 Enhanced Client; ElectroDB, PynamoDB
- **Cassandra** Apache Cassandra drivers; Spring Data Cassandra
- **Neo4j** official drivers; Spring Data Neo4j, neomodel

IN YOUR CODE

- **Node** `mongodb` driver · Mongoose · Prisma (MongoDB provider)
- **Python** PyMongo (sync and async) · Beanie · MongoEngine
- **Java** sync / reactive drivers · Spring Data MongoDB · Quarkus Panache

LIBRARIES

LANGUAGE	DRIVER	MAPPER
Node	`mongodb`	Mongoose, Prisma
Python	`pymongo`	Beanie, MongoEngine
Java	`mongodb-driver-sync`	Spring Data MongoDB
C#	`MongoDB.Driver`	EF Core provider
Go	`go.mongodb.org/mongo-driver`	—

CONNECTION SETTINGS

OPTION	DEFAULT / USE
`maxPoolSize`	100 connections
`serverSelectionTimeoutMS`	30 s to find a server
`retryWrites`	On
`w`	`majority`
`appName`	Shows in logs and Atlas metrics

⚠ COMMON MISTAKES

- Creating a new client per request
- Passing `req.body` straight into `find()`
- Using full Mongoose documents in hot read paths

✓ TIPS & BEST PRACTICES

- Create the client at startup and close it on shutdown
- Use Mongoose `sanitizeFilter` or cast inputs to strings
- Set `appName` to find your app in server logs

QUICK SUMMARY

- One client per app
- Drivers mirror mongosh
- ODMs add schemas and hooks
- Guard against operator injection

Sheet 32

Redis Introduction & Setup

in-memory · single-threaded · Redis Open Source 8 · Valkey · Docker · `redis-cli` · Redis Insight

DATA STRUCTURES IN MEMORY

Redis keeps data in RAM as **data structures** under string **keys**: strings, hashes, lists, sets, sorted sets, streams, JSON and more. Commands run one at a time on a single thread, so each is atomic and most take microseconds. **Redis Open Source 8** bundles JSON, the Query Engine (search and vectors), time series and probabilistic types, and is available under AGPLv3 among other licences. **Valkey** is the Linux Foundation fork of Redis 7.2 under BSD, command-compatible for the core types.

NOTES

- Default port 6379
- Name keys `type:id:field`, such as `order:1001:items`
- Persistence is optional (sheet 41)
- Redis Insight is the official GUI

CODE EXPLAINED

- → Each product, customer and order becomes a hash, like a row
- → Order lines become a list under their own key
- → A set links a customer to their orders; a sorted set orders them by date
- → There are no queries across keys: every lookup starts from a key name you can build

RUN, CONNECT AND LOAD THE SHOP DATA

```
docker run -d --name redis -p 6379:6379 redis:8
redis-cli                                  # localhost:6379
redis-cli -h cache.example.com -p 6380 --tls --user app --askpass

> PING
PONG
> HSET product:7 name "SQL Pocket Guide" category Books price 19.99 stock 10
> HSET product:3 name "USB-C Cable" category Electronics price 9.99 stock 120
> HSET customer:42 name "Ada Lovelace" email ada@example.com city "Des Moines"
> HSET order:1001 customerId 42 status pending total 49.97
> RPUSH order:1001:items "7:2:19.99" "3:1:9.99"
> SADD customer:42:orders 1001
> ZADD orders:by_date 1793491200 1001          # score = Unix time
> INFO server                                  # version, mode, uptime
> DBSIZE
```

SQL EQUIVALENT

- ⇄ No tables: a key is like a primary-key lookup, a hash like one row, a set like a junction table
- ⇄ Anything you'd `WHERE` on needs its own key or a Query Engine index · See SQL 4

ACROSS ENGINES

- **MongoDB** disk-based documents with an in-memory cache; Redis often caches it
- **DynamoDB** DAX is DynamoDB's own in-memory cache
- **Cassandra** has caches, but Redis usually sits in front for hot keys
- **Neo4j** page cache keeps the graph in memory; no key-value API

IN YOUR CODE

- **Node** `const r = await createClient({ url: "redis://localhost:6379" }).connect()`
- **Python** `r = redis.Redis(host="localhost", decode_responses=True)`
- **Java** `JedisPooled jedis = new JedisPooled("localhost", 6379);`

DATA TYPES

TYPE	HOLDS	SHEET
String	Text, numbers, bytes	33
Hash	Field → value map	35
List	Ordered values	36
Set	Unique values	36
Sorted set	Values ranked by score	37
Stream	Append-only log	38
JSON	Nested documents	40

TOOLS

TOOL	FOR
redis-cli	Shell, scripting, `--bigkeys`, `--latency`
Redis Insight	GUI: browse, profile, workbench
redis-benchmark	Load testing
Redis Cloud / ElastiCache	Managed hosting

⚠ COMMON MISTAKES

- Treating Redis as the only copy of important data without persistence
- Key names with no structure, impossible to scan or expire by group
- Expecting SQL-style queries across keys

✓ TIPS & BEST PRACTICES

- Pick a key naming convention and stick to it
- Keep values small; split big objects into hashes
- Use Redis Insight to see memory per key

QUICK SUMMARY

- Keys → data structures
- In memory, single-threaded
- Redis 8 bundles JSON and Search
- Valkey — BSD fork

Keys & Strings

THE BASIC BUILDING BLOCK

A **string** value can hold text, a number or up to 512 MB of bytes. **SET** and **GET** read and write it; options on SET make it conditional (**NX**: only if missing, **XX**: only if present) or expiring (**EX** seconds). **INCR** and friends turn strings into atomic counters. Key commands (EXISTS, TYPE, DEL, RENAME) work on any type, and **SCAN** walks keys safely in pages.

NOTES

- `SET … NX EX` is the standard simple lock
- `INCR` on a missing key starts at 0
- `UNLINK` frees memory in the background
- `SCAN` may return a key more than once

PRODUCTION CHECK

- `KEYS *` walks every key and blocks the server; use `SCAN` with a cursor
- Keys without a TTL pile up until memory runs out; give cache and session keys an expiry
- Multi-megabyte values block the single thread while they're sent; split or compress them

SET OPTIONS, COUNTERS & SCAN

```
> SET product:7:name "SQL Pocket Guide"
OK
> GET product:7:name
"SQL Pocket Guide"
> SET lock:order:1001 worker-3 NX EX 30       # only if missing, 30 s TTL
OK
> SET lock:order:1001 worker-4 NX EX 30
(nil)
> SET product:7:name "SQL Pocket Guide 5e" XX GET   # only if exists; old value
"SQL Pocket Guide"

> INCR counter:orders
(integer) 1002
> INCRBY product:7:views 5
> INCRBYFLOAT product:7:rating 0.5
> MSET product:7:stock 10 product:3:stock 120
> MGET product:7:stock product:3:stock

> EXISTS product:7:name
> TYPE product:7
> RENAME product:7:name product:7:title
> UNLINK big:report
> SCAN 0 MATCH product:* COUNT 100
1) "176"                                      # next cursor; "0" = done
2) 1) "product:7:title"
```

CODE EXPLAINED

- → `(nil)` from the second `SET … NX` means another worker holds the lock
- → `GET` on `SET` returns the old value in the same atomic step
- → Counters are stored as strings but `INCR` treats them as 64-bit integers
- → Call `SCAN` again with the returned cursor until it returns `0`

SQL EQUIVALENT

- ⇄ `SET` / `GET` ↔ `INSERT` / `SELECT` by primary key · `INCR` ↔ `UPDATE … SET n = n + 1` or a sequence
- ⇄ `SET … NX` ↔ `INSERT … ON CONFLICT DO NOTHING` · See SQL 58

ACROSS ENGINES

- **MongoDB** counters: `findOneAndUpdate` with `$inc` and `upsert: true`
- **DynamoDB** atomic counters: `UpdateItem … ADD views :one`
- **Cassandra** `counter` columns: `UPDATE … SET views = views + 1`
- **Neo4j** `SET n.views = n.views + 1` inside a transaction

IN YOUR CODE

- **Node** `await r.set("lock:order:1001", "w3", { NX: true, EX: 30 })`
- **Python** `r.set("lock:order:1001", "w3", nx=True, ex=30)`
- **Java** `jedis.set("lock:order:1001", "w3", SetParams.setParams().nx().ex(30))`

STRING COMMANDS

COMMAND	DOES
`SET k v [NX|XX] [EX s] [GET]`	Write, with conditions
`GET / MGET`	Read one / many
`INCR / INCRBY / DECR`	Atomic integer math
`INCRBYFLOAT`	Atomic float math
`APPEND / STRLEN`	Append / length
`GETDEL / GETEX`	Read and delete / re-expire

KEY COMMANDS

COMMAND	DOES
`EXISTS / TYPE`	Present? / which type
`DEL / UNLINK`	Delete now / in background
`RENAME`	Rename (overwrites target)
`SCAN cursor MATCH p`	Iterate keys safely
`OBJECT ENCODING`	Internal encoding

⚠ COMMON MISTAKES

- Running `KEYS *` in production
- Storing JSON blobs as strings and rewriting them for one field (use a hash or JSON)
- Locks without an expiry that stay forever if a worker crashes

✓ TIPS & BEST PRACTICES

- Prefix keys by app and type: `shop:product:7`
- Use `SET … NX EX` for locks and idempotency keys
- Use `UNLINK` for big keys

QUICK SUMMARY

- `SET` / `GET`
- `NX` / `XX` / `EX` options
- `INCR` — atomic counters
- `SCAN`, never `KEYS`

Expiry & Eviction

DATA THAT CLEANS ITSELF UP

Any key can have a **TTL**: set it with EX on SET or with **EXPIRE**, read it with **TTL**, remove it with **PERSIST**. Since 7.4 individual **hash fields** can expire too. When memory reaches **maxmemory**, the **eviction policy** decides what to drop: nothing (writes fail), least recently used, least frequently used, or keys closest to expiring. Cache-only instances usually run **allkeys-lru** or **allkeys-lfu**.

NOTES

- Writing a key with plain `SET` clears its TTL (use `KEEPTTL`)
- `TTL` returns -1 for no expiry, -2 for no key
- Expired keys are removed lazily and by a background sweep
- The default policy is `noeviction`

TTLS, FIELD EXPIRY & EVICTION

```
> SET session:abc123 "{...}" EX 1800        # 30 minutes
> EXPIRE cart:42 86400                      # set or reset the TTL
> EXPIRE cart:42 3600 GT                    # only if longer than now
> TTL cart:42
(integer) 86400
> PTTL cart:42                              # milliseconds
> EXPIREAT promo:fall 1796083200            # at a Unix time
> PERSIST cart:42                           # remove the TTL
> TTL cart:42
(integer) -1
> SET product:7:name "x" KEEPTTL            # keep the existing TTL
> GETEX session:abc123 EX 1800              # read and slide expiry

# hash field expiry (7.4+)
> HSET session:abc123 csrf t0k3n cartId 99
> HEXPIRE session:abc123 600 FIELDS 1 csrf
> HTTL session:abc123 FIELDS 1 csrf

# memory limit and eviction
> CONFIG SET maxmemory 2gb
> CONFIG SET maxmemory-policy allkeys-lru
> INFO memory
> MEMORY USAGE cart:42
```

CODE EXPLAINED

- → `GT` only extends a TTL, never shortens it (`NX`, `XX`, `LT` also exist)
- → `GETEX` makes a sliding session: each read pushes expiry out
- → `HEXPIRE` expires just the `csrf` field; the rest of the hash stays
- → `MEMORY USAGE` reports bytes used by one key

SQL EQUIVALENT

- ⇄ No SQL equivalent for TTL; the nearest is an `expires_at` column plus a scheduled `DELETE`
- ⇄ Eviction has no SQL counterpart: databases don't throw data away · See SQL 52

ACROSS ENGINES

- **MongoDB** TTL indexes delete documents, checked about every 60 s
- **DynamoDB** TTL attribute (epoch seconds); expired items are deleted within a few days, free
- **Cassandra** `USING TTL` per write or `default_time_to_live` per table
- **Neo4j** no built-in TTL; scheduled deletes or APOC's TTL procedures

IN YOUR CODE

- **Node** `await r.set("session:abc", val, { EX: 1800 }); await r.expire("cart:42", 86400)`
- **Python** `r.set("session:abc", val, ex=1800); r.expire("cart:42", 86400)`
- **Java** `jedis.setex("session:abc", 1800, val); jedis.expire("cart:42", 86400);`

TTL COMMANDS

COMMAND	DOES
`EXPIRE / PEXPIRE`	TTL in s / ms
`EXPIREAT`	Expire at Unix time
`TTL / PTTL`	Remaining time
`PERSIST`	Remove TTL
`GETEX`	Read and change TTL
`HEXPIRE / HTTL`	Hash field TTL

EVICTION POLICIES

POLICY	EVICTS
`noeviction`	Nothing; writes fail (default)
`allkeys-lru`	Least recently used key
`allkeys-lfu`	Least frequently used key
`volatile-lru / -lfu`	Same, only keys with a TTL
`volatile-ttl`	Keys closest to expiring
`allkeys-random`	Any key

⚠ COMMON MISTAKES

- Overwriting a key with `SET` and losing its TTL
- Running a cache with `noeviction` until writes fail
- Using `volatile-*` policies when most keys have no TTL

✓ TIPS & BEST PRACTICES

- Add a little random jitter to TTLs to avoid mass expiry
- Set `maxmemory` below the machine's RAM
- Watch `evicted_keys` and `expired_keys` in `INFO stats`

QUICK SUMMARY

- `EX` / `EXPIRE` — set TTL
- `TTL` -1 none, -2 missing
- `HEXPIRE` — field TTL
- `maxmemory-policy` — eviction

Hashes

OBJECTS AS FIELD MAPS

A **hash** maps fields to string values under one key, so it's the natural home for an object such as a product or a session. Read and write single fields without touching the rest, increment numeric fields atomically with **HINCRBY**, and fetch everything with **HGETALL**. Small hashes are stored compactly, which makes many small hashes cheaper than many separate keys. Values are flat: nest with JSON (sheet 40) instead.

NOTES

- `HSET` takes many field / value pairs
- All values are strings; numbers are parsed by `HINCRBY`
- Field expiry from 7.4; `HGETDEL` / `HSETEX` from 8.0
- `HGETALL` on a huge hash blocks; use `HSCAN`

HASH COMMANDS

```
> HSET product:7 name "SQL Pocket Guide" category Books price 19.99 stock 10
(integer) 4
> HGET product:7 price
"19.99"
> HMGET product:7 name stock
1) "SQL Pocket Guide"
2) "10"
> HGETALL product:7
> HINCRBY product:7 stock -2
(integer) 8
> HINCRBYFLOAT product:7 price 1.00
"20.99"
> HSETNX product:7 createdAt 1793491200        # only if the field is missing
> HEXISTS product:7 discount
(integer) 0
> HDEL product:7 discount
> HLEN product:7
> HKEYS product:7
> HSCAN product:7 0 MATCH s*

# Redis 8: set with expiry, get and delete
> HSETEX product:7 EX 300 FIELDS 1 flashPrice 14.99
> HGETDEL cart:42 FIELDS 1 coupon
```

CODE EXPLAINED

- → `HSET` returns how many new fields were added
- → `HINCRBY` is atomic, so two clients decrementing stock never lose an update
- → `HSETNX` sets a field only once, handy for "first seen" timestamps
- → `HSETEX` writes a field that disappears after 5 minutes

SQL EQUIVALENT

- ⇄ A hash ↔ one row · `HINCRBY` ↔ `UPDATE products SET stock = stock - 2`
- ⇄ `HGETALL` ↔ `SELECT *` for one key; there's no query across hashes · See SQL 51

ACROSS ENGINES

- **MongoDB** a document updated with `$set` / `$inc` on fields
- **DynamoDB** an item updated with `SET` / `ADD`
- **Cassandra** a row, or a `map<text, text>` column
- **Neo4j** node properties: `SET p.stock = p.stock - 2`

IN YOUR CODE

- **Node** `await r.hSet("product:7", { name: "SQL Pocket Guide", stock: 10 })`
- **Python** `r.hset("product:7", mapping={"name": "SQL Pocket Guide", "stock": 10})`
- **Java** `jedis.hset("product:7", Map.of("name", "SQL Pocket Guide", "stock", "10"));`

HASH COMMANDS

COMMAND	DOES
`HSET / HSETNX`	Write fields / only if missing
`HGET / HMGET / HGETALL`	Read
`HINCRBY / HINCRBYFLOAT`	Atomic math
`HDEL / HEXISTS / HLEN`	Remove / test / count
`HSCAN`	Iterate fields
`HEXPIRE / HSETEX`	Field expiry

STRING, HASH OR JSON?

STORE AS	WHEN
`String`	One value, or an opaque blob
`Hash`	Flat object; update fields separately
`JSON`	Nested object; query by path or index it
`Several keys`	Fields need different TTLs (pre-7.4)

⚠ COMMON MISTAKES

- Storing an object as one JSON string and rewriting it for each field change
- Calling `HGETALL` on hashes with thousands of fields
- Expecting nested values in a hash

✓ TIPS & BEST PRACTICES

- Use hashes for objects with flat fields
- Use `HINCRBY` for stock and counters
- Keep hashes small for the compact encoding

QUICK SUMMARY

- Hash — one object per key
- Field-level reads and writes
- `HINCRBY` — atomic
- Flat only; nest with JSON

Lists & Sets

ORDERED LISTS, UNIQUE SETS

A **list** is an ordered sequence that allows duplicates: push and pop at either end in constant time, so it works as a **queue**, a stack or a "recent items" list capped with `LTRIM`. Blocking pops (`BLPOP`) let workers wait for jobs. A **set** holds unique, unordered strings: membership tests are instant and **`SINTER`**, **`SUNION`** and **`SDIFF`** combine sets, which suits tags, "who viewed this" and shared interests.

NOTES

- `LMOVE` moves an item atomically, the basis of a reliable queue
- Indexing into the middle of a long list is slow (O(n))
- Sets can't hold duplicates; adding one is ignored
- For real job queues, streams (sheet 38) add acknowledgements

LISTS AND SETS

```
# lists: ordered, duplicates allowed
> RPUSH queue:emails "order:1001" "order:1002"
> LPUSH recent:customer:42 "product:7"
> LTRIM recent:customer:42 0 9                # keep the newest 10
> LRANGE recent:customer:42 0 -1
> LPOP queue:emails
"order:1001"
> BLPOP queue:emails 5                        # wait up to 5 s
> LMOVE queue:emails queue:processing LEFT RIGHT
> LLEN queue:emails

# sets: unique, unordered
> SADD product:7:tags sql reference books
(integer) 3
> SISMEMBER product:7:tags sql
(integer) 1
> SMEMBERS product:7:tags
> SADD customer:42:viewed 7 3 21
> SADD customer:43:viewed 7 22
> SINTER customer:42:viewed customer:43:viewed
1) "7"
> SDIFF customer:42:viewed customer:43:viewed
> SCARD product:7:tags
> SRANDMEMBER product:7:tags 2
```

CODE EXPLAINED

- → `LPUSH` + `LTRIM` keeps a fixed-length "recently viewed" list
- → `BLPOP` returns as soon as an item arrives, or nil after the timeout
- → `LMOVE` to a processing list means a crashed worker's job isn't lost
- → `SINTER` finds products both customers viewed

SQL EQUIVALENT

- ⇄ A list ↔ a table with a position column · a set ↔ a table with a `UNIQUE` constraint
- ⇄ `SINTER` / `SUNION` / `SDIFF` ↔ `INTERSECT` / `UNION` / `EXCEPT` · See SQL 43

ACROSS ENGINES

- **MongoDB** arrays with `$push` / `$addToSet`; `$setIntersection` in pipelines
- **DynamoDB** `L` lists and `SS` / `NS` sets; `ADD` to a set
- **Cassandra** `list<>` and `set<>` collection columns
- **Neo4j** list properties; shared neighbors through `MATCH` patterns

IN YOUR CODE

- **Node** `await r.lPush("recent:customer:42", "product:7"); await r.lTrim("recent:customer:42", 0, 9)`
- **Python** `r.lpush("recent:customer:42", "product:7"); r.ltrim("recent:customer:42", 0, 9)`
- **Java** `jedis.lpush("recent:customer:42", "product:7"); jedis.ltrim("recent:customer:42", 0, 9);`

LIST COMMANDS

COMMAND	DOES
`LPUSH / RPUSH`	Add to head / tail
`LPOP / RPOP`	Remove from head / tail
`BLPOP / BRPOP`	Blocking pop
`LRANGE k 0 -1`	Read a range
`LTRIM`	Keep a range
`LMOVE`	Pop and push atomically

SET COMMANDS

COMMAND	DOES
`SADD / SREM`	Add / remove
`SISMEMBER / SMISMEMBER`	Membership
`SMEMBERS / SSCAN`	All / iterate
`SINTER / SUNION / SDIFF`	Combine sets
`SCARD`	Count
`SRANDMEMBER / SPOP`	Random members

⚠ COMMON MISTAKES

- Using `LRANGE 0 -1` on huge lists
- Using `SMEMBERS` on sets with millions of members
- Building a job queue on `RPOP` without a processing list

✓ TIPS & BEST PRACTICES

- Cap lists with `LTRIM` right after each push
- Use `SSCAN` for big sets
- Store the result of a costly `SINTER` with `SINTERSTORE` and a TTL

QUICK SUMMARY

- List — ordered, queue-friendly
- `BLPOP` / `LMOVE` — workers
- Set — unique members
- `SINTER` / `SUNION` / `SDIFF`

Sorted Sets

ZADD · ZRANGE · REV / BYSCORE · ZINCRBY · ZRANK · leaderboards · rate limits

RANKED MEMBERS

A **sorted set** stores unique members, each with a numeric **score**, kept in score order. Rank, range and score lookups are fast (O(log n)), which makes sorted sets the standard tool for **leaderboards**, "top N" lists, time-ordered indexes and **sliding-window rate limits**. The unified **ZRANGE** reads by rank, score (BYSCORE) or name (BYLEX), in either direction (REV).

NOTES

- Ties sort by member name
- `(` in a score range means exclusive: `(10`
- `-inf` and `+inf` are open bounds
- Scores are 64-bit floats

LEADERBOARDS, RANGES & RATE LIMITS

```
> ZADD bestsellers:books 214 product:7 98 product:12 340 product:5
> ZINCRBY bestsellers:books 1 product:7          # one more sale
"215"
> ZRANGE bestsellers:books 0 2 REV WITHSCORES    # top 3
1) "product:5"
2) "340"
3) "product:7"
4) "215"
...
> ZREVRANK bestsellers:books product:7           # 0-based, from the top
(integer) 1
> ZSCORE bestsellers:books product:7

# products by price
> ZADD products:by_price 19.99 product:7 9.99 product:3 24.50 product:21
> ZRANGE products:by_price 10 20 BYSCORE          # 10 ≤ price ≤ 20
> ZRANGE products:by_price (10 +inf BYSCORE LIMIT 0 5
> ZCOUNT products:by_price 10 20

# sliding window: at most 100 requests per 60 s per IP
> ZREMRANGEBYSCORE rate:1.2.3.4 0 1793491140000  # drop entries older than 60 s
> ZADD rate:1.2.3.4 1793491200000 req-8f2c       # score = now in ms
> ZCARD rate:1.2.3.4                             # > 100 → reject
> EXPIRE rate:1.2.3.4 60
```

CODE EXPLAINED

- → `ZINCRBY` updates a score atomically and re-sorts the member
- → `REV` reads highest first; `WITHSCORES` adds each score after its member
- → `(10` excludes products costing exactly 10
- → Run the rate-limit steps in a `MULTI` or Lua script so they're atomic

SQL EQUIVALENT

- ⇄ Sorted set ↔ an indexed column with `ORDER BY score DESC LIMIT 3`
- ⇄ `ZREVRANK` ↔ `RANK() OVER (ORDER BY score DESC)` · See SQL 45

ACROSS ENGINES

- **MongoDB** index on the score field + `sort().limit()` ; `$rank` window
- **DynamoDB** a GSI with the score as sort key, queried with `ScanIndexForward = false`
- **Cassandra** a clustering column `score DESC` within a partition
- **Neo4j** `ORDER BY p.sales DESC LIMIT 3`

IN YOUR CODE

- **Node** `await r.zIncrBy("bestsellers:books", 1, "product:7")`
- **Python** `r.zincrby("bestsellers:books", 1, "product:7")`
- **Java** `jedis.zincrby("bestsellers:books", 1, "product:7");`

SORTED SET COMMANDS

COMMAND	DOES
`ZADD [NX|XX|GT|LT]`	Add or update scores
`ZINCRBY`	Add to a score
`ZRANGE … [BYSCORE|BYLEX] [REV]`	Read a range
`ZRANK / ZREVRANK`	Position
`ZSCORE / ZCOUNT`	Score / count in range
`ZREM / ZREMRANGEBYSCORE`	Remove
`ZPOPMIN / ZPOPMAX`	Pop lowest / highest

USE CASES

USE	SCORE
Leaderboard	Points or sales
Time index	Unix time
Rate limit	Request time in ms
Priority queue	Priority
Autocomplete	0 for all, read BYLEX

⚠ COMMON MISTAKES

- Using a list and sorting in the app instead of a sorted set
- Forgetting to trim old entries in time-based sets
- Running rate-limit steps as separate, non-atomic calls

✓ TIPS & BEST PRACTICES

- Use `ZADD … GT` to keep only a member's best score
- Pair a time-indexed sorted set with hashes holding the details
- Cap leaderboards with `ZREMRANGEBYRANK`

QUICK SUMMARY

- Members ranked by score
- `ZRANGE … REV` — top N
- `BYSCORE` — ranges
- Rate limits and time indexes

Streams & Pub/Sub

LOGS AND BROADCASTS

A **stream** is an append-only log: each entry gets an ID (`<ms>-<seq>`) and a set of fields. Readers can replay from any ID, block for new entries, and form **consumer groups** where each entry goes to one consumer and must be **acknowledged**. **Pub/Sub** is different: a message goes to everyone subscribed right now and is gone, with no history and no acknowledgement. Use streams for work and events you can't lose, Pub/Sub for live notifications.

NOTES

- `*` lets Redis generate the entry ID
- `$` means "only new entries"; `>` means "never delivered to this group"
- Cap stream length with `MAXLEN ~ n`
- Sharded Pub/Sub (`SPUBLISH`) scales in Cluster

STREAMS, CONSUMER GROUPS & PUB/SUB

```
# append events
> XADD orders:events * type placed orderId 1001 total 49.97
"1793491200123-0"
> XADD orders:events MAXLEN ~ 100000 * type paid orderId 1001
> XLEN orders:events
> XRANGE orders:events - + COUNT 10
> XREAD COUNT 10 BLOCK 5000 STREAMS orders:events $   # wait for new ones

# consumer groups: each entry to one worker, acknowledged
> XGROUP CREATE orders:events billing $ MKSTREAM
> XREADGROUP GROUP billing worker-1 COUNT 10 BLOCK 5000 STREAMS orders:events >
> XACK orders:events billing 1793491200123-0
> XPENDING orders:events billing
> XAUTOCLAIM orders:events billing worker-2 60000 0-0  # take stuck entries

# pub/sub: everyone listening now, no history
> SUBSCRIBE price-changes
> PUBLISH price-changes '{"productId":7,"price":17.99}'
(integer) 2                                    # receivers
> PSUBSCRIBE orders.*
```

CODE EXPLAINED

- → `MAXLEN ~ 100000` trims roughly, which is much cheaper than an exact trim
- → Each group keeps its own position, so billing and shipping can both read every event
- → `XPENDING` lists delivered but unacknowledged entries
- → `XAUTOCLAIM` hands entries idle for 60 s to another worker

SQL EQUIVALENT

- ⇄ Streams ↔ an `events` table read by polling; Pub/Sub ↔ `LISTEN` / `NOTIFY`
- ⇄ Neither has a direct SQL equivalent; triggers are the closest in-database hook · See SQL 67

ACROSS ENGINES

- **MongoDB** change streams (sheet 28) or a capped collection
- **DynamoDB** DynamoDB Streams to Lambda
- **Cassandra** time-bucketed append-only tables; Kafka usually sits alongside
- **Neo4j** CDC feeds; not a queue

IN YOUR CODE

- **Node** `await r.xAdd("orders:events", "*", { type: "placed", orderId: "1001" })`
- **Python** `r.xadd("orders:events", {"type": "placed", "orderId": 1001})`
- **Java** `jedis.xadd("orders:events", StreamEntryID.NEW_ENTRY, Map.of("type", "placed"));`

STREAMS VS PUB/SUB

	STREAMS	PUB/SUB
History	Kept (until trimmed)	None
Offline consumers	Catch up later	Miss messages
Acknowledgement	Yes (groups)	No
Delivery	One consumer per group	Every subscriber
Use for	Jobs, event logs	Live updates, invalidation

STREAM COMMANDS

COMMAND	DOES
XADD	Append
XRANGE / XREVRANGE	Read by ID range
XREAD	Read, optionally blocking
XGROUP CREATE	Create a consumer group
XREADGROUP / XACK	Consume / acknowledge
XTRIM	Cap length or age

⚠ COMMON MISTAKES

- Using Pub/Sub for work that must not be lost
- Never trimming streams
- Forgetting `XACK`, so the pending list grows forever

✓ TIPS & BEST PRACTICES

- Make consumers idempotent; entries can be delivered twice
- Monitor `XPENDING` for stuck work
- Use one stream per event type or aggregate

QUICK SUMMARY

- Stream — durable log
- Consumer groups + `XACK`
- Pub/Sub — live, no history
- `MAXLEN ~` to cap

Caching Patterns

cache-aside · write-through · TTL strategy · invalidation · stampede protection · session store

REDIS AS A CACHE

The most common use of Redis is as a **cache** in front of a slower source of truth. In **cache-aside**, the app checks Redis first, loads from the database on a miss, and stores the result with a **TTL**. On writes, update the database and then **delete** the cache key so the next read refills it. Protect hot keys from a **stampede** (many requests rebuilding at once) with a short lock or early refresh, and add jitter to TTLs.

NOTES

- Cache what's expensive and read often
- Key names should include versions: `v2:product:7`
- Session stores are caches with sliding TTLs
- Measure hit rate (`keyspace_hits` / `keyspace_misses`)

PRODUCTION CHECK

- Cache stampede: a hot key expires and thousands of requests hit the database together. Use a short `SET NX EX` lock, early refresh or TTL jitter
- Stale data after writes: delete the cache key after the database commit, not before
- On cache-only instances set `maxmemory` and `maxmemory-policy allkeys-lru` (or `lfu`)

CACHE-ASIDE, INVALIDATION & STAMPEDE LOCK

```
// cache-aside: read through the cache, fill on a miss
async function getProduct(id) {
  const key = `product:${id}`
  const hit = await redis.get(key)
  if (hit) return JSON.parse(hit)
  const p = await db.collection("products").findOne({ _id: id })
  const ttl = 3600 + Math.floor(Math.random() * 300)      // TTL + jitter
  await redis.set(key, JSON.stringify(p), { EX: ttl })
  return p
}

// write: update the source of truth, then drop the cache entry
async function updatePrice(id, price) {
  await db.collection("products").updateOne({ _id: id }, { $set: { price } })
  await redis.del(`product:${id}`)
}

// stampede protection: one caller rebuilds, others wait briefly
async function getWithLock(key, load, ttl = 3600) {
  const hit = await redis.get(key)
  if (hit) return JSON.parse(hit)
  if (await redis.set(`lock:${key}`, "1", { NX: true, EX: 10 })) {
    const v = await load()
    await redis.set(key, JSON.stringify(v), { EX: ttl })
    await redis.del(`lock:${key}`)
    return v
  }
  await new Promise(r => setTimeout(r, 50))
  return getWithLock(key, load, ttl)
}
```

CODE EXPLAINED

- → Jitter spreads expiry times so a batch of keys doesn't expire together
- → Deleting rather than updating the cache avoids racing writers storing old values
- → The lock expires after 10 s even if the loader crashes
- → Callers that miss the lock retry after 50 ms and usually find the fresh value

SQL EQUIVALENT

- ⇄ A cache is a copy kept outside the database; the nearest SQL feature is a materialized view
- ⇄ The database's own buffer cache saves disk reads, not network round trips · See SQL 64

ACROSS ENGINES

- **MongoDB** WiredTiger keeps hot data in RAM; Redis still saves the network and query work
- **DynamoDB** DAX: a write-through cache with microsecond reads
- **Cassandra** key and row caches; Redis often fronts hot rows
- **Neo4j** page cache; cache computed recommendations in Redis

IN YOUR CODE

- **Node** `await redis.set(key, JSON.stringify(p), { EX: 3600 })`
- **Python** `r.set(key, json.dumps(p), ex=3600)`
- **Java** Spring: `@Cacheable(value = "products", key = "#id")` with `spring-boot-starter-data-redis`

CACHING PATTERNS

PATTERN	HOW
Cache-aside	App reads cache, fills on miss
Read-through	Cache library loads on miss
Write-through	Write cache and database together
Write-behind	Write cache, flush to database later
Refresh-ahead	Refresh hot keys before expiry

TTL STRATEGY

DATA	TYPICAL TTL
Product pages	Minutes to hours
Prices / stock	Seconds, or invalidate on write
Sessions	30 min sliding
Expensive reports	Hours; refresh ahead
Feature flags	Seconds

⚠ COMMON MISTAKES

- Updating the cache before the database commit
- Caching without TTLs and serving stale data forever
- Caching per-user data under shared keys

✓ TIPS & BEST PRACTICES

- Start with cache-aside and delete-on-write
- Version key names when the cached shape changes
- Track hit rate; under ~80% means the cache may not be worth it

QUICK SUMMARY

- Cache-aside — fill on miss
- Delete key after write
- TTL + jitter
- Lock against stampedes

JSON, Search & Vectors

DOCUMENTS AND QUERIES IN REDIS

Redis Open Source 8 includes **JSON** (store and update nested documents by JSONPath) and the **Query Engine** (formerly RediSearch). **FT.CREATE** indexes hashes or JSON documents by key prefix with **TEXT**, **TAG**, **NUMERIC**, **GEO** and **VECTOR** fields; **FT.SEARCH** filters, ranks and sorts, and **FT.AGGREGATE** groups and computes. The index updates automatically as matching keys change.

NOTES

- Commands below are wrapped for space; type each on one line
- TAG fields match exact values: `@category:{Books}`
- Vectors are passed as raw float32 bytes in `PARAMS`
- Indexes live in memory alongside the data

JSON, FT.CREATE & FT.SEARCH

```
> JSON.SET product:7 $ '{"name":"SQL Pocket Guide","category":"Books",
    "price":19.99,"tags":["sql"]}'
> JSON.GET product:7 $.price
"[19.99]"
> JSON.NUMINCRBY product:7 $.price -2
> JSON.ARRAPPEND product:7 $.tags '"reference"'

# index every key starting with product:
> FT.CREATE idx:products ON JSON PREFIX 1 product: SCHEMA
    $.name AS name TEXT
    $.category AS category TAG
    $.price AS price NUMERIC SORTABLE
    $.embedding AS embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 384 DISTANCE_METRIC COSINE

# full-text + tag + range, sorted
> FT.SEARCH idx:products "@name:(pocket guide) @category:{Books} @price:[10 30]"
    RETURN 2 name price SORTBY price ASC LIMIT 0 10

# group and count
> FT.AGGREGATE idx:products "*" GROUPBY 1 @category REDUCE COUNT 0 AS n

# vector nearest neighbors with a filter
> FT.SEARCH idx:products "(@category:{Books})=>[KNN 5 @embedding $v AS score]"
    PARAMS 2 v "<384 float32 bytes>" SORTBY score DIALECT 2
```

CODE EXPLAINED

- → JSONPath `$.price` returns an array because paths can match several values
- → The index covers keys created before and after `FT.CREATE`
- → `@price:[10 30]` is an inclusive numeric range
- → The part before `=>` filters; KNN then ranks the survivors by vector distance

SQL EQUIVALENT

- ⇄ JSON ↔ a `jsonb` column with `->` / `->>` · `FT.SEARCH` ↔ `tsvector @@ tsquery`
- ⇄ Vector KNN ↔ pgvector `ORDER BY embedding <=> $1 LIMIT 5` · See SQL 78, 80

ACROSS ENGINES

- **MongoDB** `$search` and `$vectorSearch` (sheet 24)
- **DynamoDB** zero-ETL to OpenSearch for search and vectors
- **Cassandra** SAI indexes with `ANN OF` vector search
- **Neo4j** full-text and vector indexes on node properties

IN YOUR CODE

- **Node** `await r.json.set("product:7", "$", product); await r.ft.search("idx:products", "@category:{Books}")`
- **Python** `r.json().set("product:7", "$", product); r.ft("idx:products").search("@category:{Books}")`
- **Java** `jedis.jsonSet("product:7", Path2.ROOT_PATH, product); jedis.ftSearch("idx:products", "@category:{Books}")`

JSON COMMANDS

COMMAND	DOES
`JSON.SET key path value`	Write at a path
`JSON.GET key path`	Read
`JSON.NUMINCRBY`	Add to a number
`JSON.ARRAPPEND`	Append to an array
`JSON.DEL`	Remove a path
`JSON.MGET`	Same path from many keys

INDEX FIELD TYPES

TYPE	QUERY SYNTAX
`TEXT`	`@name:(pocket guide)`
`TAG`	`@category:{Books|Music}`
`NUMERIC`	`@price:[10 30]`
`GEO`	`@loc:[lon lat 5 km]`
`VECTOR`	`=>[KNN 5 @embedding $v]`

⚠ COMMON MISTAKES

- Indexing text that should be TAG (ids, categories)
- Forgetting `SORTABLE` on fields you sort by
- Index prefixes that don't match your key names

✓ TIPS & BEST PRACTICES

- Use TAG for exact values and TEXT for words
- Return only needed fields with `RETURN`
- Check an index with `FT.INFO idx:products`

QUICK SUMMARY

- JSON — nested docs by path
- `FT.CREATE` — index by prefix
- `FT.SEARCH` — query and rank
- VECTOR — KNN search

Transactions, Lua & Operations

ATOMIC GROUPS, SCRIPTS AND DURABILITY

Redis runs commands one at a time, so **every single command is atomic**. **`MULTI` … `EXEC`** queues commands and runs them as one block that no other client can interleave with, but there is **no rollback**: if one command fails at run time the others still apply. **`WATCH`** adds optimistic locking, and EXEC returns nil if a watched key changed. **Lua scripts** (EVAL) and **Functions** (FCALL, Redis 7+) run logic on the server atomically. **Pipelining** only saves round trips. For durability, Redis offers **RDB** snapshots, an **AOF** write log, replicas and **Cluster** sharding.

NOTES

- A syntax error while queuing aborts the whole `EXEC`; a wrong-type error at run time doesn't
- Scripts block the server while they run; Redis answers `BUSY` after 5 s (`busy-reply-threshold`)
- Pass every key a script touches in `KEYS`, not built inside the script

PRODUCTION CHECK

- Out of the box Redis saves only RDB snapshots, so a crash loses writes since the last one. Set `appendonly yes` with `appendfsync everysec` to lose at most about 1 s
- In Cluster, a transaction or script touching keys in different slots fails with `CROSSSLOT`. Group keys with hash tags: `{order:1001}:items`
- Replication is asynchronous; a failover can drop acknowledged writes. `WAIT 1 100` narrows the gap but isn't strong consistency

TRANSACTIONS, SCRIPTS & PERSISTENCE

```
# MULTI / EXEC: queue, then run as one block
> MULTI
OK
> DECRBY product:7:stock 2
QUEUED
> RPUSH order:1001:items "7:2"
QUEUED
> EXEC
1) (integer) 8
2) (integer) 1

# WATCH: optimistic lock on the stock key
> WATCH product:7:stock
> GET product:7:stock            # "8", client checks it's >= 2
> MULTI
> DECRBY product:7:stock 2
> EXEC                           # (nil) = changed by someone else, retry

# Lua: check and decrement in one atomic step
$ redis-cli EVAL "
    local s = tonumber(redis.call('GET', KEYS[1]))
    if s and s >= tonumber(ARGV[1]) then
      return redis.call('DECRBY', KEYS[1], ARGV[1]) end
    return -1" 1 product:7:stock 2

# Functions (Redis 7+): load a library once, call by name
$ redis-cli FUNCTION LOAD "$(cat shop.lua)"
> FCALL reserve_stock 1 product:7:stock 2

# Persistence and replication
> CONFIG SET appendonly yes
> BGSAVE                         # RDB snapshot in the background
> REPLICAOF 10.0.0.5 6379        # follow a primary
```

CODE EXPLAINED

- → `QUEUED` means accepted for later; nothing runs until `EXEC`, which returns one reply per command
- → `WATCH` must come before `MULTI`; any write to the key by another client makes `EXEC` return nil, so wrap it in a retry loop
- → The `1` after the script is the number of keys; then come `KEYS[1]` and `ARGV[1]`
- → A function library starts with `#!lua name=shop` and calls `redis.register_function`

SQL EQUIVALENT

- ⇄ `MULTI` / `EXEC` ↔ `BEGIN` … `COMMIT`, but Redis has no `ROLLBACK`
- ⇄ `WATCH` ↔ optimistic locking with a version column (`UPDATE … WHERE version = 3`); SQL also offers `SELECT … FOR UPDATE`
- ⇄ Lua and Functions ↔ stored procedures · See SQL 68, 66

ACROSS ENGINES

- **MongoDB** single-document writes are atomic; `session.withTransaction()` spans documents and rolls back on error
- **DynamoDB** `TransactWriteItems`: up to 100 actions, all-or-nothing, with condition checks
- **Cassandra** no multi-partition transactions; lightweight transactions (`IF`) per partition; logged batches apply eventually
- **Neo4j** full ACID; every Cypher query runs in a transaction that rolls back on error

IN YOUR CODE

- **Node** `await client.multi().decrBy("product:7:stock", 2).rPush("order:1001:items", "7:2").exec()`
- **Python** `r.pipeline().decrby("product:7:stock", 2).rpush("order:1001:items", "7:2").execute()`
- **Java** `Transaction t = jedis.multi(); t.decrBy("product:7:stock", 2); t.exec();`

ATOMICITY OPTIONS

TOOL	ATOMIC	USE FOR
`Single command`	Yes	Counters, `SET NX EX` locks
`Pipeline`	No	Fewer round trips
`MULTI / EXEC`	Yes, no rollback	Grouping writes
`WATCH + MULTI`	Aborts on change	Check-then-set
`EVAL (Lua)`	Yes	Logic between reads and writes
`Functions`	Yes	Reusable server logic

PERSISTENCE & TOPOLOGY

OPTION	WHAT IT GIVES YOU
`RDB`	Snapshots (`save`, `BGSAVE`); fast restart, loses recent writes
`AOF`	Logs every write; `everysec` loses ≤ ~1 s
`RDB + AOF`	Both; the usual durable setup
`Replication`	Async copies via `REPLICAOF`
`Sentinel`	Automatic failover for primary + replicas
`Cluster`	16,384 hash slots across primaries

⚠ COMMON MISTAKES

- Expecting `EXEC` to roll back when one command fails
- Treating a pipeline as a transaction; other clients can run between its commands
- Using `WATCH` without a retry loop, so a nil `EXEC` silently drops the write

✓ TIPS & BEST PRACTICES

- Reach for a single atomic command first (`INCRBY`, `SET NX EX`, `LMOVE`)
- Prefer Functions over `EVAL` for logic you call often; `EVALSHA` avoids resending scripts
- Run `INFO persistence` after config changes to confirm AOF is on

QUICK SUMMARY

- `MULTI` / `EXEC` — atomic, no rollback
- `WATCH` — optimistic lock
- Lua / Functions — server-side logic
- RDB + AOF — durability

DynamoDB Concepts

MANAGED KEY-VALUE AT ANY SCALE

DynamoDB is AWS's fully managed, serverless key-value and document database. A **table** holds **items** (up to 400 KB each) made of **attributes**. Only the **primary key** is declared: a **partition key** alone, or a partition key plus a **sort key**. Items with the same partition key form an **item collection**, stored together and sorted by sort key. You pay per request (**on-demand**) or for reserved read / write capacity (**provisioned**). **DynamoDB Local** runs it on your machine.

NOTES

- Every item needs the full primary key; other attributes vary per item
- Partition key = HASH, sort key = RANGE in the API
- Data is replicated across three Availability Zones
- Global tables replicate across regions

CREATE A TABLE AND LOAD SHOP ITEMS

```
docker run -d -p 8000:8000 amazon/dynamodb-local
export AWS_ENDPOINT_URL=http://localhost:8000    # point the CLI at Local

aws dynamodb create-table --table-name Shop \
  --attribute-definitions AttributeName=PK,AttributeType=S \
                          AttributeName=SK,AttributeType=S \
  --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE \
  --billing-mode PAY_PER_REQUEST

# items read together share a partition key
aws dynamodb put-item --table-name Shop --item '{
  "PK": {"S": "CUSTOMER#42"}, "SK": {"S": "PROFILE"},
  "name": {"S": "Ada Lovelace"}, "email": {"S": "ada@example.com"} }'
aws dynamodb put-item --table-name Shop --item '{
  "PK": {"S": "CUSTOMER#42"}, "SK": {"S": "ORDER#2026-11-01#1001"},
  "status": {"S": "pending"}, "total": {"N": "49.97"} }'
aws dynamodb put-item --table-name Shop --item '{
  "PK": {"S": "ORDER#1001"}, "SK": {"S": "ITEM#7"},
  "qty": {"N": "2"}, "unitPrice": {"N": "19.99"} }'

aws dynamodb describe-table --table-name Shop
```

CODE EXPLAINED

- → `AWS_ENDPOINT_URL` sends every AWS CLI call to DynamoDB Local (any credentials work)
- → Only `PK` and `SK` are declared; `name`, `status` and `qty` are free-form
- → The customer's profile and orders share `CUSTOMER#42`, so one Query returns both
- → Typed JSON (`{"S": …}`, `{"N": …}`) is the low-level format; SDK document clients hide it

SQL EQUIVALENT

- ⇄ Table ↔ table without fixed columns · PK + SK ↔ a composite primary key
- ⇄ No joins, no ad hoc `WHERE`: every read names a key or an index · See SQL 56

ACROSS ENGINES

- **MongoDB** a collection per entity with `_id` as key
- **Redis** key names like `customer:42` do the same job as partition keys
- **Cassandra** closest cousin: partition key + clustering columns
- **Neo4j** nodes with labels; no partition key to design

IN YOUR CODE

- **Node** `const doc = DynamoDBDocumentClient.from(new DynamoDBClient({ region: "us-east-2" }))`
- **Python** `table = boto3.resource("dynamodb").Table("Shop")`
- **Java** `DynamoDbTable<Order> t = DynamoDbEnhancedClient.create().table("Shop", TableSchema.fromBean(Order.class));`

TERMS

TERM	MEANING
Item	One record (≤ 400 KB)
Attribute	A named value
Partition key	Decides storage partition
Sort key	Orders items within a partition
Item collection	All items with one partition key
RCU / WCU	Read / write capacity units

LIMITS WORTH KNOWING

LIMIT	VALUE
Item size	400 KB
Per partition	~3,000 reads / 1,000 writes per s
Query / Scan page	1 MB
GSIs per table	20 (default quota)
Transaction	100 actions
BatchWriteItem	25 puts / deletes

⚠ COMMON MISTAKES

- Designing tables like SQL tables, one per entity with no access-pattern plan
- Using a low-cardinality partition key such as `status`
- Forgetting that number attributes are sent as strings in the low-level API

✓ TIPS & BEST PRACTICES

- Prefix key values with the entity type: `CUSTOMER#42`
- Start with on-demand capacity
- Use the document client so you work with plain objects

QUICK SUMMARY

- Table → items → attributes
- Partition key + sort key
- On-demand or provisioned
- DynamoDB Local for dev

Item Operations

PutItem · GetItem · UpdateItem · DeleteItem · condition expressions · update expressions

PUT, GET, UPDATE, DELETE

PutItem writes a whole item, replacing any item with the same key. **GetItem** reads one item by its full primary key. **UpdateItem** changes attributes with an **update expression** and creates the item if it doesn't exist. **DeleteItem** removes one item. Any write can carry a **condition expression**; if it's false the write fails with `ConditionalCheckFailedException`, which gives you insert-only puts and safe decrements.

NOTES

- Attribute names that are reserved words (`name` , `status`) need `#aliases`
- Values always go through `:placeholders`
- `ReturnValues` returns old or new attributes
- Writes are billed per 1 KB, reads per 4 KB

ITEM OPERATIONS WITH CONDITIONS

```
# insert-only put
aws dynamodb put-item --table-name Shop \
  --item '{"PK":{"S":"PRODUCT#7"},"SK":{"S":"META"},
           "name":{"S":"SQL Pocket Guide"},"stock":{"N":"10"}}' \
  --condition-expression "attribute_not_exists(PK)"

# read one item, two attributes
aws dynamodb get-item --table-name Shop \
  --key '{"PK":{"S":"PRODUCT#7"},"SK":{"S":"META"}}' \
  --projection-expression "#n, stock" \
  --expression-attribute-names '{"#n":"name"}'

# conditional decrement plus other changes
aws dynamodb update-item --table-name Shop \
  --key '{"PK":{"S":"PRODUCT#7"},"SK":{"S":"META"}}' \
  --update-expression "SET stock = stock - :q, updatedAt = :now ADD sold :q REMOVE promo" \
  --condition-expression "stock >= :q" \
  --expression-attribute-values '{":q":{"N":"2"},":now":{"S":"2026-11-07T10:00:00Z"}}' \
  --return-values UPDATED_NEW
# stock < 2 → ConditionalCheckFailedException

# delete only if it exists
aws dynamodb delete-item --table-name Shop \
  --key '{"PK":{"S":"PRODUCT#7"},"SK":{"S":"META"}}' \
  --condition-expression "attribute_exists(PK)"
```

CODE EXPLAINED

- → `attribute_not_exists(PK)` makes the put fail if the item already exists
- → `#n` stands in for `name` , a reserved word
- → One update expression can `SET` , `ADD` and `REMOVE` in a single atomic write
- → `UPDATED_NEW` returns only the attributes that changed, with new values

SQL EQUIVALENT

- ⇄ `PutItem` ↔ `INSERT` (or a full replace) · condition ↔ the `WHERE` of an `UPDATE`
- ⇄ `UPDATE products SET stock = stock - 2 WHERE product_id = 7 AND stock >= 2` · See SQL 50, 51

ACROSS ENGINES

- **MongoDB** `updateOne({ _id: 7, stock: { $gte: 2 } }, { $inc: { stock: -2 } })`
- **Redis** Lua or `WATCH` for a conditional decrement
- **Cassandra** `UPDATE … IF stock = 10` (lightweight transaction)
- **Neo4j** `MATCH (p:Product {id: 7}) WHERE p.stock >= 2 SET p.stock = p.stock - 2`

IN YOUR CODE

- **Node** `new UpdateCommand({ TableName: "Shop", Key, UpdateExpression: "SET stock = stock - :q", ConditionExpression: "stock >= :q", ExpressionAttributeValues: { ":q": 2 } })`
- **Python** `table.update_item(Key=key, UpdateExpression="SET stock = stock - :q", ConditionExpression="stock >= :q", ExpressionAttributeValues={":q": 2})`
- **Java** `table.updateItem(UpdateItemEnhancedRequest.builder(Product.class).item(p).conditionExpression(cond).build())`

UPDATE EXPRESSION ACTIONS

ACTION	EXAMPLE
`SET`	`SET stock = stock - :q`
`SET + function`	`SET tags = list_append(tags, :t)`
`SET default`	`SET n = if_not_exists(n, :zero)`
`ADD`	Add to a number or a set
`REMOVE`	Delete attributes
`DELETE`	Remove values from a set

CONDITION FUNCTIONS

FUNCTION	TRUE WHEN
`attribute_exists(a)`	Attribute present
`attribute_not_exists(a)`	Attribute absent
`attribute_type(a, :t)`	Has that type
`begins_with(a, :s)`	String prefix
`contains(a, :v)`	Substring or set member
`size(a)`	Length, for comparisons

⚠ COMMON MISTAKES

- Using `PutItem` to change one attribute and wiping the others
- Read-then-write instead of a condition expression
- Forgetting reserved words like `name` , `status` , `date`

✓ TIPS & BEST PRACTICES

- Use conditions for every "only if" rule
- Use `UpdateItem` for partial changes
- Catch `ConditionalCheckFailedException` as a normal outcome

QUICK SUMMARY

- `PutItem` — whole item
- `UpdateItem` — expressions, upserts
- Condition expressions guard writes
- `#name` for reserved words

Query vs Scan

READ BY KEY, NOT BY SEARCH

Query reads one item collection: it needs the partition key with = and can narrow by sort key (`begins_with`, `BETWEEN`, <, >), returning items in sort-key order. **Scan** reads the entire table. A **filter expression** removes items after they're read, so it saves bandwidth but not cost. Results come in pages of up to **1 MB**; continue from **LastEvaluatedKey**. **PartiQL** offers SQL-like syntax over the same rules.

NOTES

- The CLI follows pages for you; SDKs give you `LastEvaluatedKey`
- `Limit` caps items read, before filtering
- Parallel Scan splits a table into segments for exports
- A PartiQL `SELECT` without a key condition becomes a Scan

PRODUCTION CHECK

- `Scan` reads every item and bills for all of it, even with a filter; use Query on a key or a GSI
- Filters run after the read: a Query that returns 10 of 10,000 items still pays for 10,000
- Results come in 1 MB pages; loop on `LastEvaluatedKey` or you silently miss data

QUERY, FILTERS, PAGING, SCAN & PARTIQL

```
# a customer's orders, newest first
aws dynamodb query --table-name Shop \
  --key-condition-expression "PK = :c AND begins_with(SK, :o)" \
  --expression-attribute-values '{":c":{"S":"CUSTOMER#42"},":o":{"S":"ORDER#"}}' \
  --no-scan-index-forward --max-items 10

# sort-key range: November 2026
aws dynamodb query --table-name Shop \
  --key-condition-expression "PK = :c AND SK BETWEEN :a AND :b" \
  --expression-attribute-values '{":c":{"S":"CUSTOMER#42"},
     ":a":{"S":"ORDER#2026-11"},":b":{"S":"ORDER#2026-12"}}'

# filter after reading (still billed for what was read)
  --filter-expression "#s = :p" --expression-attribute-names '{"#s":"status"}'

# next page with the CLI
  --max-items 10 --starting-token <NextToken from the last call>

# Scan: whole table; segments for parallel exports
aws dynamodb scan --table-name Shop --total-segments 4 --segment 0

# PartiQL: SQL-like, same rules underneath
aws dynamodb execute-statement --statement \
  "SELECT * FROM Shop WHERE PK = 'CUSTOMER#42' AND begins_with(SK, 'ORDER#')"
```

CODE EXPLAINED

- `--no-scan-index-forward` reverses sort-key order: newest dates first
- `ORDER#2026-12` sorts after every November key, so `BETWEEN` covers the month
- The filter lines are added to a query like the one above
- PartiQL with a key condition runs as a Query; without one, as a Scan

SQL EQUIVALENT

- `Query` ↔ `SELECT … WHERE pk = ?` using the index · `Scan` ↔ a full table scan
- Filter expression ↔ `WHERE` on an unindexed column, applied after reading · See SQL 73

ACROSS ENGINES

- **MongoDB** any field can be filtered; indexes decide the cost
- **Redis** key lookups, or `SCAN` through keys
- **Cassandra** same rule: `WHERE` must name the partition key
- **Neo4j** `MATCH` on an indexed property seeks; otherwise it scans the label

IN YOUR CODE

- **Node** `new QueryCommand({ TableName: "Shop", KeyConditionExpression: "PK = :c", ExpressionAttributeValues: { ":c": "CUSTOMER#42" }, ScanIndexForward: false })`
- **Python** `table.query(KeyConditionExpression=Key("PK").eq("CUSTOMER#42") & Key("SK").begins_with("ORDER#"), ScanIndexForward=False)`
- **Java** `table.query(QueryConditional.sortBeginsWith(Key.builder().partitionValue("CUSTOMER#42").sortValue("ORDER#").build()))`

KEY CONDITIONS

ON	ALLOWED
Partition key	= only
Sort key	=, <, <=, >, >=
Sort key	BETWEEN :a AND :b
Sort key	begins_with(SK, :p)
Anything else	Filter expression (after read)

QUERY VS SCAN

	QUERY	SCAN
Reads	One item collection	Whole table
Cost	Items read	Every item
Order	By sort key	None
Use for	App access patterns	Exports, migrations

⚠ COMMON MISTAKES

- Using `Scan` + filter as a general query
- Ignoring pagination and returning the first page only
- Expecting `Limit` to mean "results after filtering"

✓ TIPS & BEST PRACTICES

- Design keys so every frequent read is a Query
- Put dates in sort keys in ISO format so they sort correctly
- Use parallel Scan or export to S3 for bulk jobs

QUICK SUMMARY

- Query — partition key + sort condition
- Scan — whole table, avoid
- Filters don't save cost
- Page on `LastEvaluatedKey`

Secondary Indexes

GSI · LSI · projections · sparse indexes · index overloading · eventual consistency

OTHER WAYS IN

A **global secondary index (GSI)** is a copy of selected attributes with a different partition and sort key, so you can Query by something other than the table key. It can be added any time and is **eventually consistent**. A **local secondary index (LSI)** keeps the table's partition key with a different sort key, supports strong reads, but must be created with the table. Items without the index key attributes are left out, which makes **sparse indexes** a cheap filter.

NOTES

- GSIs have their own capacity on provisioned tables
- Projection decides which attributes are copied
- One GSI can serve several entity types (overloading)
- LSIs cap an item collection at 10 GB

PRODUCTION CHECK

- GSIs are eventually consistent: a Query right after a write may not see it
- A throttled GSI holds back writes to the base table; give GSIs enough capacity (on-demand helps)
- Every projected attribute is copied and billed on write; project only what the query needs

ADD, FILL & QUERY A GSI

```
# GSI1: orders by status, newest first
aws dynamodb update-table --table-name Shop \
  --attribute-definitions AttributeName=GSI1PK,AttributeType=S \
                          AttributeName=GSI1SK,AttributeType=S \
  --global-secondary-index-updates '[{"Create":{"IndexName":"GSI1",
     "KeySchema":[{"AttributeName":"GSI1PK","KeyType":"HASH"},
                  {"AttributeName":"GSI1SK","KeyType":"RANGE"}],
     "Projection":{"ProjectionType":"INCLUDE","NonKeyAttributes":["total"]}}}]'

# an item joins the index by having the GSI key attributes
aws dynamodb update-item --table-name Shop \
  --key '{"PK":{"S":"CUSTOMER#42"},"SK":{"S":"ORDER#2026-11-01#1001"}}' \
  --update-expression "SET GSI1PK = :s, GSI1SK = :d" \
  --expression-attribute-values '{":s":{"S":"STATUS#pending"},
                                  ":d":{"S":"2026-11-01#1001"}}'

aws dynamodb query --table-name Shop --index-name GSI1 \
  --key-condition-expression "GSI1PK = :s" \
  --expression-attribute-values '{":s":{"S":"STATUS#pending"}}' \
  --no-scan-index-forward

# LSI: only at create-table time
#   --local-secondary-indexes '[{"IndexName":"ByTotal","KeySchema":[
#     {"AttributeName":"PK","KeyType":"HASH"},{"AttributeName":"total","KeyType":"RANGE"}],
#     "Projection":{"ProjectionType":"KEYS_ONLY"}}]'
```

CODE EXPLAINED

- → Generic names (`GSI1PK`) let one index serve many access patterns
- → `INCLUDE` copies the keys plus `total` ; other attributes need a second read
- → Removing `GSI1PK` (when an order ships) drops it from the index: a sparse index of pending orders
- → A busy status value is a hot GSI partition; add a shard suffix if it gets heavy

SQL EQUIVALENT

- ⇄ GSI ↔ `CREATE INDEX` on other columns, but kept as an async copy · LSI ↔ an index sharing the partition
- ⇄ `INCLUDE` projection ↔ `CREATE INDEX … INCLUDE (total)` · See SQL 72

ACROSS ENGINES

- **MongoDB** secondary indexes are synchronous and always consistent
- **Redis** hand-built index sets, or `FT.CREATE`
- **Cassandra** SAI, or a second table per query
- **Neo4j** indexes on label properties, always consistent

IN YOUR CODE

- **Node** `new QueryCommand({ TableName: "Shop", IndexName: "GSI1", KeyConditionExpression: "GSI1PK = :s", ExpressionAttributeValues: { ":s": "STATUS#pending" } })`
- **Python** `table.query(IndexName="GSI1", KeyConditionExpression=Key("GSI1PK").eq("STATUS#pending"))`
- **Java** `table.index("GSI1").query(QueryConditional.keyEqualTo(k -> k.partitionValue("STATUS#pending")))`

GSI VS LSI

	GSI	LSI
Partition key	Any attribute	Same as table
Create	Any time	With the table only
Consistency	Eventual	Eventual or strong
Capacity	Its own	The table's
Per table	20 (quota)	5
Size cap	None	10 GB per partition key

PROJECTION TYPES

TYPE	COPIES
KEYS_ONLY	Table and index keys
INCLUDE	Keys + listed attributes
ALL	Every attribute

⚠ COMMON MISTAKES

- Reading from a GSI right after writing and expecting the new item
- Projecting `ALL` on every index
- Planning on an LSI after the table exists

✓ TIPS & BEST PRACTICES

- Overload one or two GSIs instead of creating many
- Use sparse indexes for "items in state X"
- Shard hot GSI keys: `STATUS#pending#3`

QUICK SUMMARY

- GSI — new key, any time, eventual
- LSI — same partition, at create
- Sparse indexes filter for free
- Project only what you read

Single-Table Design

ONE TABLE, MANY ENTITIES

Single-table design stores several entity types in one table, with generic key names (PK, SK) and typed key values (`CUSTOMER#42`, `ORDER#1001`). Entities read together share a partition key, so one Query returns a **pre-joined** item collection. It starts from an **access-pattern table**: each pattern maps to a key condition on the table or a GSI. It shines for known, high-volume patterns and hurts when questions change often.

NOTES

- Store a `type` attribute on every item
- Copy display fields into child items to avoid extra reads
- GSI overloading: one index, many entity types
- Adjacency lists model many-to-many

PRODUCTION CHECK

- Hot partitions: one partition key taking most traffic is throttled (about 3,000 reads / 1,000 writes per second); use high-cardinality keys or write sharding
- With an LSI, each item collection is capped at 10 GB
- Single-table makes new ad hoc questions hard; if access patterns are still unclear, start with one table per entity

SHOP AS ONE TABLE

```
// Items in table "Shop" (typed key values, generic names)
{ PK: "CUSTOMER#42", SK: "PROFILE", type: "customer",
  name: "Ada Lovelace", email: "ada@example.com" }
{ PK: "CUSTOMER#42", SK: "ORDER#2026-11-01#1001", type: "order",
  status: "pending", total: 49.97, GSI1PK: "ORDER#1001", GSI1SK: "META" }
{ PK: "ORDER#1001", SK: "ITEM#7", type: "line",
  qty: 2, unitPrice: 19.99, name: "SQL Pocket Guide" }
{ PK: "ORDER#1001", SK: "ITEM#3", type: "line",
  qty: 1, unitPrice: 9.99, name: "USB-C Cable" }
{ PK: "PRODUCT#7", SK: "META", type: "product",
  name: "SQL Pocket Guide", stock: 10,
  GSI1PK: "CATEGORY#Books", GSI1SK: "PRODUCT#7" }

// Access patterns → key conditions
// AP1 profile + recent orders      PK = CUSTOMER#42
// AP2 orders, newest first         PK = CUSTOMER#42 AND begins_with(SK, "ORDER#"), reversed
// AP3 an order's lines             PK = ORDER#1001
// AP4 order header by id           GSI1: GSI1PK = ORDER#1001
// AP5 products in a category       GSI1: GSI1PK = CATEGORY#Books
```

CODE EXPLAINED

- → AP1 returns the profile and all orders in one Query because they share a partition key
- → Line items copy the product name so an order page needs no product lookups
- → GSI1 serves two unrelated patterns because its key values are typed
- → Adding AP6 later may mean a new GSI or a backfill of new key attributes

SQL EQUIVALENT

- ⇄ Several entities in one table is the opposite of normalization; an item collection is a stored join
- ⇄ Adjacency lists ↔ a junction table for many-to-many · See SQL 61, 62

ACROSS ENGINES

- **MongoDB** one collection per entity, with related data embedded
- **Redis** key prefixes play the same role (`order:1001:items`)
- **Cassandra** one table per query rather than one table for everything
- **Neo4j** no keys to design; relationships carry the structure

IN YOUR CODE

- **Node** ElectroDB or DynamoDB Toolbox model several entities on one table
- **Python** PynamoDB models, or boto3 with small key-builder functions
- **Java** Enhanced Client beans per entity with `@DynamoDbPartitionKey` / `@DynamoDbSortKey` on one table

KEY DESIGN PATTERNS

PATTERN	EXAMPLE
Composite sort key	ORDER#<date>#<id>
Hierarchy	COUNTRY#US#STATE#IA#CITY#Ames
Adjacency list	PK = A, SK = B per edge
Sparse index	Only some items have GSI1PK
Write sharding	STATUS#pending#0…#9
GSI overloading	Typed GSI key values

SINGLE TABLE OR NOT?

SITUATION	CHOOSE
Patterns known and stable	Single table
High volume, low latency	Single table
Patterns still changing	Table per entity
Heavy analytics needs	Export to an analytics store
Team new to DynamoDB	Table per entity first

⚠ COMMON MISTAKES

- Starting with the item layout instead of the access-pattern table
- Generic keys with no `type` attribute, impossible to debug
- Putting every customer's orders under one partition key like `ORDERS`

✓ TIPS & BEST PRACTICES

- Keep the access-pattern table next to the code
- Write helper functions that build keys
- Use NoSQL Workbench to visualize the table

QUICK SUMMARY

- Entities share one table
- Typed key values
- Item collections = stored joins
- Design from access patterns

Streams, TTL & Transactions

CHANGES, EXPIRY, ALL-OR-NOTHING

DynamoDB Streams records every item change for 24 hours, and **Lambda** can process it to update caches, search indexes or aggregates. **TTL** deletes items after a timestamp attribute (epoch seconds) passes, at no cost, usually within a few days. **Transactions** (`TransactWriteItems`, `TransactGetItems`) group up to 100 actions across items and tables: all succeed or all fail, and a client token makes retries idempotent.

NOTES

- TTL deletions appear in the stream, marked as service deletions
- Filter Lambda events to pay only for the ones you need
- Transactions cost twice the capacity of normal writes
- The idempotency token is honored for 10 minutes

STREAMS, TTL & TRANSACTWRITEITEMS

```
# change log, read by Lambda
aws dynamodb update-table --table-name Shop \
  --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES
aws lambda create-event-source-mapping --function-name onShopChange \
  --event-source-arn <stream-arn> --starting-position LATEST \
  --filter-criteria '{"Filters":[{"Pattern":"{\"eventName\":[\"INSERT\"]}"}]}'

# expire items when expiresAt (epoch seconds) passes
aws dynamodb update-time-to-live --table-name Shop \
  --time-to-live-specification Enabled=true,AttributeName=expiresAt

# place an order: reserve stock and create the order together
aws dynamodb transact-write-items --client-request-token order-1002-v1 \
  --transact-items '[
  {"Update": {"TableName": "Shop",
    "Key": {"PK": {"S": "PRODUCT#7"}, "SK": {"S": "META"}},
    "UpdateExpression": "SET stock = stock - :q",
    "ConditionExpression": "stock >= :q",
    "ExpressionAttributeValues": {":q": {"N": "2"}}}},
  {"Put": {"TableName": "Shop",
    "Item": {"PK": {"S": "CUSTOMER#42"}, "SK": {"S": "ORDER#2026-11-07#1002"},
             "status": {"S": "pending"}},
    "ConditionExpression": "attribute_not_exists(SK)"}}
]'
```

CODE EXPLAINED

- → `NEW_AND_OLD_IMAGES` puts both versions of the item in each stream record
- → The filter means Lambda only runs for inserts
- → If stock is short, the whole transaction is canceled and no order is created
- → Retrying with the same `--client-request-token` won't apply it twice

SQL EQUIVALENT

- ⇄ Streams ↔ logical decoding / CDC · TTL ↔ a scheduled `DELETE`
- ⇄ `TransactWriteItems` ↔ `BEGIN … COMMIT` ; the token ↔ an idempotency key · See SQL 67, 68

ACROSS ENGINES

- **MongoDB** change streams; TTL indexes; `withTransaction()`
- **Redis** keyspace notifications or streams; `EXPIRE` ; `MULTI` / `EXEC`
- **Cassandra** CDC; `USING TTL` ; lightweight transactions per partition
- **Neo4j** CDC; no built-in TTL; ACID transactions

IN YOUR CODE

- **Node** `await doc.send(new TransactWriteCommand({ TransactItems: [...], ClientRequestToken: "order-1002-v1" }))`
- **Python** `client.transact_write_items(TransactItems=[...], ClientRequestToken="order-1002-v1")`
- **Java** `enhanced.transactWriteItems(r -> r.addUpdateItem(products, p).addPutItem(orders, o))`

STREAM VIEW TYPES

TYPE	RECORD HOLDS
KEYS_ONLY	Key attributes
NEW_IMAGE	Item after the change
OLD_IMAGE	Item before the change
NEW_AND_OLD_IMAGES	Both

TRANSACTIONS

FACT	VALUE
Actions per call	Up to 100
Action types	Put, Update, Delete, ConditionCheck
Cost	2× normal capacity
Failure	`TransactionCanceledException` with reasons
Idempotency	Token, 10 minutes

⚠ COMMON MISTAKES

- Expecting TTL deletes at the exact second; filter expired items in reads
- Non-idempotent Lambda handlers (records can repeat)
- Using transactions for things one conditional update can do

✓ TIPS & BEST PRACTICES

- Filter expired items with `expiresAt > :now` until TTL catches up
- Use `ConditionCheck` to assert on items you don't change
- Read cancellation reasons to tell which condition failed

QUICK SUMMARY

- Streams + Lambda — react to changes
- TTL — free, delayed deletes
- Transactions — 100 actions, all-or-nothing
- Client token — safe retries

Cassandra Architecture

A RING OF EQUAL NODES

Cassandra spreads data over a **ring** of equal nodes with no primary. The **partition key** is hashed (Murmur3) to a token, which decides which nodes own the row, and each row is copied to **replication factor (RF)** nodes. Any node can coordinate a request, and the **consistency level** says how many replicas must answer. Writes go to a commit log and an in-memory table, later flushed to immutable **SSTables** that **compaction** merges, which is why writes are fast.

NOTES

- Nodes share cluster state by gossip
- Datacenters are groups of nodes; replication is set per datacenter
- This set uses Cassandra 5.0
- Amazon Keyspaces and Astra offer managed CQL

RUN, INSPECT AND CREATE THE SHOP KEYSPACE

```
docker run -d --name cass1 -p 9042:9042 cassandra:5.0
docker exec -it cass1 nodetool status        # UN = Up / Normal
docker exec -it cass1 cqlsh

cqlsh> CREATE KEYSPACE shop WITH replication =
         { 'class': 'NetworkTopologyStrategy', 'datacenter1': 1 };   -- 3 in production
cqlsh> USE shop;
cqlsh:shop> CREATE TABLE orders_by_customer (
              customer_id int, order_date date, order_id int,
              status text, total decimal,
              PRIMARY KEY ((customer_id), order_date, order_id)
            ) WITH CLUSTERING ORDER BY (order_date DESC, order_id DESC);
cqlsh:shop> INSERT INTO orders_by_customer
              (customer_id, order_date, order_id, status, total)
              VALUES (42, '2026-11-01', 1001, 'pending', 49.97);
cqlsh:shop> CONSISTENCY QUORUM;
cqlsh:shop> SELECT * FROM orders_by_customer WHERE customer_id = 42 LIMIT 10;
cqlsh:shop> TRACING ON;
```

CODE EXPLAINED

- → `nodetool status` lists nodes, their load and token ownership
- → With one node, RF must be 1; production clusters use 3 per datacenter
- → All of customer 42's orders live in one partition, sorted newest first
- → `TRACING ON` shows which replicas served the next query and how long each step took

SQL EQUIVALENT

- ⇄ Keyspace ↔ database · table ↔ table, but rows are spread across servers by partition key
- ⇄ No joins, no foreign keys, limited `WHERE` · See SQL 60

ACROSS ENGINES

- **MongoDB** replica sets with one primary, unlike Cassandra's masterless ring
- **Redis** Cluster hash slots are a simpler cousin of token ranges
- **DynamoDB** same partition + sort key idea, fully managed
- **Neo4j** clustered primaries and secondaries; not partitioned by key

IN YOUR CODE

- **Node** `new cassandra.Client({ contactPoints: ["127.0.0.1"], localDataCenter: "datacenter1", keyspace: "shop" })`
- **Python** `session = Cluster(["127.0.0.1"]).connect("shop")`
- **Java** `CqlSession.builder().withLocalDatacenter("datacenter1").withKeyspace("shop").build()`

ARCHITECTURE TERMS

TERM	MEANING
`Token ring`	Hash range split among nodes
`Replication factor`	Copies of each row
`Coordinator`	Node handling a request
`Gossip`	Peer-to-peer cluster state
`Hinted handoff`	Stores writes for a down replica
`Repair`	Syncs replicas

WRITE & READ PATH

STEP	WHAT HAPPENS
`Write`	Commit log + memtable, then ack
`Flush`	Memtable → immutable SSTable
`Compaction`	Merge SSTables, drop old data
`Read`	Memtable + SSTables, merged by timestamp
`Bloom filters`	Skip SSTables without the key

⚠ COMMON MISTAKES

- RF 3 on a single dev node, so `QUORUM` queries fail
- Treating CQL as SQL and expecting joins
- Running Cassandra with too little heap on a laptop

✓ TIPS & BEST PRACTICES

- Use `NetworkTopologyStrategy` even with one datacenter
- Set `localDataCenter` in drivers
- Use Docker or Astra for learning

QUICK SUMMARY

- Masterless ring
- Partition key → token → nodes
- Replication factor per datacenter
- Fast writes, compaction later

Sheet 49

CQL: Keyspaces & Tables

CREATE KEYSPACE · partition key · clustering columns · PRIMARY KEY ((a), b) · data types · collections

KEYS DECIDE EVERYTHING

CQL looks like SQL, but the **primary key** controls storage. In PRIMARY KEY `((a, b), c, d)`, the inner parentheses are the **partition key** (which node, and the unit of a fast read) and the rest are **clustering columns** (sort order inside the partition). Columns are typed, and **collections** (`list`, `set`, `map`) and **user-defined types** hold small nested data. A keyspace sets the **replication** strategy and factor.

NOTES

- `frozen<>` stores a collection or UDT as one value
- Adding columns is cheap; changing the primary key means a new table
- `timeuuid` gives unique, time-ordered ids
- Every write is an upsert

PRODUCTION CHECK

- `QUORUM` needs a majority of replicas: with RF 3, two must answer; with RF 1, any down node fails the query
- In multi-datacenter clusters use `LOCAL_QUORUM` so requests don't wait on the far region
- Choose partition keys that spread data and keep each partition under about 100 MB

KEYSPACE, TYPES & TABLES

```
CREATE KEYSPACE IF NOT EXISTS shop
  WITH replication = { 'class': 'NetworkTopologyStrategy', 'dc1': 3, 'dc2': 3 };

CREATE TYPE IF NOT EXISTS shop.line_item (
  product_id int, name text, qty int, unit_price decimal);

CREATE TABLE IF NOT EXISTS shop.orders (
  order_id    int,
  customer_id int,
  order_date  date,
  status      text,
  total       decimal,
  items       list<frozen<line_item>>,
  tags        set<text>,
  attrs       map<text, text>,
  created_at  timestamp,
  PRIMARY KEY (order_id)                          -- partition key only
);

CREATE TABLE IF NOT EXISTS shop.orders_by_customer (
  customer_id int, order_date date, order_id int,
  status text, total decimal,
  PRIMARY KEY ((customer_id), order_date, order_id)
) WITH CLUSTERING ORDER BY (order_date DESC, order_id ASC);

-- composite partition key: one partition per product per month
CREATE TABLE IF NOT EXISTS shop.events_by_product (
  product_id int, month text, ts timeuuid, type text,
  PRIMARY KEY ((product_id, month), ts));

ALTER TABLE shop.orders ADD coupon text;
DESCRIBE TABLE shop.orders;
```

CODE EXPLAINED

- `dc1: 3, dc2: 3` keeps three copies in each of two datacenters
- The order lines become a list of a user-defined type inside the order row
- `orders_by_customer` sorts each customer's orders by date, newest first
- Adding `month` to the partition key stops one product's events growing into one giant partition

SQL EQUIVALENT

- `PRIMARY KEY ((a), b)` ↔ `PRIMARY KEY (a, b)`, except `a` picks the server and `b` sorts rows inside it
- UDT ↔ a composite type · collections ↔ arrays or child tables · See SQL 55, 56

ACROSS ENGINES

- **MongoDB** no table DDL; collections take any document
- **Redis** no DDL; keys appear when written
- **DynamoDB** only the key schema is declared: partition = HASH, sort = RANGE
- **Neo4j** no table DDL; only constraints and indexes

IN YOUR CODE

- **Node** `await client.execute("INSERT INTO orders_by_customer (customer_id, order_date, order_id) VALUES (?, ?, ?)", [42, "2026-11-01", 1001], { prepare: true })`
- **Python** `session.execute(session.prepare("INSERT INTO orders_by_customer (customer_id, order_date, order_id) VALUES (?, ?, ?)"), (42, date(2026, 11, 1), 1001))`
- **Java** `session.execute(session.prepare(insertCql).bind(42, LocalDate.of(2026, 11, 1), 1001));`

COMMON TYPES

TYPE	HOLDS
`int / bigint`	32 / 64-bit integers
`text`	UTF-8 string
`timestamp / date`	Instant / calendar date
`uuid / timeuuid`	Random / time-based id
`list / set / map`	Small collections
`vector<float, n>`	Embeddings (5.0)

CONSISTENCY LEVELS

LEVEL	REPLICAS THAT ANSWER
`ONE`	Any one
`QUORUM`	Majority across all datacenters
`LOCAL_QUORUM`	Majority in the local datacenter
`EACH_QUORUM`	Majority in every datacenter (writes)
`ALL`	Every replica
`SERIAL`	For lightweight transactions

⚠ COMMON MISTAKES

- A partition key with few values (`status`), creating huge, hot partitions
- Large collections; they're read whole
- Expecting to change a table's primary key with `ALTER`

✓ TIPS & BEST PRACTICES

- Name tables after their query: `orders_by_customer`
- Bucket time series by day or month in the partition key
- Use prepared statements for every repeated query

QUICK SUMMARY

- `((partition), clustering…)`
- Partition — where; clustering — order
- Collections and UDTs for small nesting
- Replication per datacenter

Query-First Modeling

ONE TABLE PER QUERY

Cassandra modeling starts from the queries and builds **one table for each**, keyed so the query reads a single partition in order. The same data is written to several tables (**denormalized**), often together in a **logged batch**. `WHERE` must give the whole partition key (with = or `IN`) and may restrict clustering columns in order. **SAI** indexes (5.0) allow filtering on other columns within the rules, so fewer extra tables are needed.

NOTES

- Logged batches guarantee all statements apply eventually, without isolation
- Don't use batches to speed up bulk loads
- Materialized views are experimental; prefer your own tables
- Query a range only on the last restricted clustering column

TWO QUERIES, TWO TABLES

```
-- Q1: a customer's orders, newest first → partition by customer
CREATE TABLE orders_by_customer (
  customer_id int, order_date date, order_id int, status text, total decimal,
  PRIMARY KEY ((customer_id), order_date, order_id)
) WITH CLUSTERING ORDER BY (order_date DESC, order_id DESC);

-- Q2: orders by status for one day → another table, same data
CREATE TABLE orders_by_status_day (
  status text, day date, order_id int, customer_id int, total decimal,
  PRIMARY KEY ((status, day), order_id));

-- write both copies together
BEGIN BATCH
  INSERT INTO orders_by_customer (customer_id, order_date, order_id, status, total)
    VALUES (42, '2026-11-01', 1001, 'pending', 49.97);
  INSERT INTO orders_by_status_day (status, day, order_id, customer_id, total)
    VALUES ('pending', '2026-11-01', 1001, 42, 49.97);
APPLY BATCH;

-- allowed: full partition key, then clustering columns in order
SELECT * FROM orders_by_customer
 WHERE customer_id = 42 AND order_date >= '2026-10-01';
-- rejected: no partition key
SELECT * FROM orders_by_customer WHERE order_date = '2026-11-01';

-- SAI (5.0): filter a non-key column without a new table
CREATE INDEX orders_status_sai ON orders_by_customer (status) USING 'sai';
SELECT * FROM orders_by_customer WHERE customer_id = 42 AND status = 'pending';
```

CODE EXPLAINED

- → Each table answers exactly one question with one partition read
- → `(status, day)` in the partition key bounds each partition to one day of one status
- → The rejected query would have to ask every node; Cassandra refuses unless you add `ALLOW FILTERING`
- → SAI combined with the partition key stays a single-partition read

SQL EQUIVALENT

- ⇄ One table per query ↔ denormalized reporting tables kept in sync by the app
- ⇄ Logged batch ↔ a transaction without isolation or rollback · See SQL 61, 74

ACROSS ENGINES

- **MongoDB** usually one collection plus several indexes covers many queries
- **Redis** one key per lookup: the same query-first mindset
- **DynamoDB** GSIs do what the extra tables do here
- **Neo4j** one graph answers many questions; model the traversals

IN YOUR CODE

- **Node** `await client.execute("SELECT * FROM orders_by_customer WHERE customer_id = ?", [42], { prepare: true })`
- **Python** cqlengine: `class OrderByCustomer(Model): customer_id = columns.Integer(partition_key=True)`
- **Java** Spring Data Cassandra: `@PrimaryKeyColumn(name = "customer_id", type = PrimaryKeyType.PARTITIONED)`

WHERE RULES

RESTRICTION	ALLOWED?
All partition key columns with = / IN	Yes
Clustering columns, left to right	Yes
Range on the last restricted clustering column	Yes
Skip a clustering column	No (SAI may help)
Non-key column	Only with SAI or ALLOW FILTERING

MODELING STEPS

STEP	OUTPUT
List queries	Q1, Q2, …
Choose partition keys	Spread, bounded size
Choose clustering columns	Sort order
Write paths	Which tables each change touches
Check sizes	Rows per partition, bytes

⚠ COMMON MISTAKES

- Modeling from an ER diagram and then fighting `WHERE` rules
- Unlogged batches across many partitions to "speed up" writes
- Partitions that grow forever (all events for one product)

✓ TIPS & BEST PRACTICES

- Write the query first, then the table
- Use logged batches only to keep denormalized copies in step
- Try SAI before adding another table

QUICK SUMMARY

- One table per query
- Denormalize writes
- Partition key required in `WHERE`
- SAI for extra filters

Writes, TTL & Operations

EXPIRY, COMPARE-AND-SET, UPKEEP

Writes can expire with **USING TTL** (or a table default). Because data is immutable on disk, a delete or expiry writes a **tombstone** that hides old values until compaction removes it after **gc_grace_seconds** (10 days). **Lightweight transactions** (`IF NOT EXISTS`, `IF col = value`) use Paxos for compare-and-set within one partition, at higher latency. **Counter** columns live in their own tables. Keep clusters healthy with regular **repair** and watch **compaction**.

NOTES

- Every cell stores a write timestamp; the newest wins
- LWTs cost about four round trips
- Counters can't mix with normal columns or use TTL
- Unified Compaction Strategy arrived in 5.0

PRODUCTION CHECK

- Deletes and expired TTLs write tombstones; reading partitions full of them is slow and fails at 100,000 (`tombstone_failure_threshold`). Avoid queue-like delete patterns
- `ALLOW FILTERING` scans whole partitions or the whole cluster; never ship it on large tables
- Run repair within `gc_grace_seconds` (10 days) or deleted data can come back

TTL, LWT, COUNTERS & DELETES

```
-- TTL per write, read it back, table default
INSERT INTO sessions (id, customer_id) VALUES (uuid(), 42) USING TTL 1800;
UPDATE carts USING TTL 86400 SET items = items + [7] WHERE customer_id = 42;
SELECT TTL(customer_id), WRITETIME(customer_id) FROM sessions WHERE id = ?;
ALTER TABLE sessions WITH default_time_to_live = 1800;

-- lightweight transactions: compare-and-set in one partition
INSERT INTO users_by_email (email, customer_id)
  VALUES ('ada@example.com', 42) IF NOT EXISTS;
UPDATE products SET stock = 8 WHERE product_id = 7 IF stock = 10;
--  [applied] | stock
--      False |     9        ← not applied; shows the current value

-- counters live in their own tables
CREATE TABLE product_views (product_id int PRIMARY KEY, views counter);
UPDATE product_views SET views = views + 1 WHERE product_id = 7;

-- a delete writes a tombstone
DELETE FROM orders_by_customer
 WHERE customer_id = 42 AND order_date = '2026-11-01' AND order_id = 1001;

-- whole partitions delete cheaply (one tombstone)
DELETE FROM orders_by_customer WHERE customer_id = 42;
```

CODE EXPLAINED

- → `TTL()` shows seconds left; `WRITETIME()` shows the write's microsecond timestamp
- → `IF NOT EXISTS` makes email unique without a read-then-write race
- → A failed LWT returns `[applied] = False` and the current values
- → Deleting a whole partition writes one tombstone instead of one per row

SQL EQUIVALENT

- ⇄ `IF NOT EXISTS` ↔ `INSERT … ON CONFLICT DO NOTHING` · `IF stock = 10` ↔ optimistic `UPDATE … WHERE stock = 10`
- ⇄ Counter ↔ `UPDATE … SET n = n + 1` ; TTL has no SQL equivalent · See SQL 53, 70

ACROSS ENGINES

- **MongoDB** TTL indexes; conditional updates through filters
- **Redis** `EXPIRE` ; `SET NX` ; `INCR`
- **DynamoDB** TTL attribute; condition expressions; atomic `ADD`
- **Neo4j** no TTL; `MERGE` + uniqueness constraints for insert-if-absent

IN YOUR CODE

- **Node** `const rs = await client.execute(cql, [8, 7, 10], { prepare: true }); rs.wasApplied()`
- **Python** `session.execute(stmt, (8, 7, 10)).was_applied`
- **Java** `session.execute(bound).wasApplied()`

NODETOOL

COMMAND	DOES
`status`	Nodes, load, ownership
`repair -pr shop`	Sync replicas (primary ranges)
`compactionstats`	Running compactions
`tablestats shop.t`	Sizes, latencies, tombstones
`snapshot -t tag shop`	Hard-link backup
`drain`	Flush before shutdown

COMPACTION STRATEGIES

STRATEGY	FITS
`STCS (size-tiered)`	Write-heavy, general default
`LCS (leveled)`	Read-heavy, updates
`TWCS (time-window)`	Time series with TTL
`UCS (unified, 5.0)`	Tunable replacement for all three

⚠ COMMON MISTAKES

- Using a table as a queue (insert, read, delete) and drowning in tombstones
- LWTs on hot paths where plain writes would do
- Skipping repairs for weeks

✓ TIPS & BEST PRACTICES

- Use TWCS for TTL-heavy time series
- Delete whole partitions rather than many rows
- Schedule repairs with Cassandra Reaper or your platform's tool

QUICK SUMMARY

- `USING TTL` — expiry
- Tombstones — deletes cost reads
- `IF` — compare-and-set (LWT)
- Repair within 10 days

Sheet 52

The Graph Model & Neo4j

nodes · relationships · labels · properties · Neo4j Desktop / Aura / Docker · Browser · shop data as a graph

NODES, RELATIONSHIPS, PROPERTIES

A **property graph** has **nodes** (things), **relationships** (directed, typed connections) and **properties** (key-value data on both). **Labels** group nodes, such as `:Customer` or `:Product`. Relationships are stored as direct pointers, so following them costs the same however big the graph gets, which is where graphs beat joins. **Neo4j** runs as Aura (managed), Desktop or Docker, and you query it in **Cypher** from the Browser or a driver.

NOTES

- Neo4j 2025.06+ supports Cypher 25 (prefix a query with `CYPHER 25`)
- Every relationship has exactly one type and a direction
- Browser at port 7474, Bolt protocol on 7687
- Community is GPLv3; Enterprise adds clustering and RBAC

SHOP DATA AS A GRAPH

```
// docker run -p 7474:7474 -p 7687:7687 \
//   -e NEO4J_AUTH=neo4j/shop-pass-123 neo4j:latest
CREATE (ada:Customer {id: 42, name: "Ada Lovelace", city: "Des Moines"})
CREATE (book:Product {id: 7, name: "SQL Pocket Guide", price: 19.99, stock: 10})
CREATE (cable:Product {id: 3, name: "USB-C Cable", price: 9.99, stock: 120})
CREATE (books:Category {name: "Books"}), (elec:Category {name: "Electronics"})
CREATE (o:Order {id: 1001, date: date("2026-11-01"), status: "pending", total: 49.97})
CREATE (ada)-[:PLACED]->(o)
CREATE (o)-[:CONTAINS {qty: 2, unitPrice: 19.99}]->(book)
CREATE (o)-[:CONTAINS {qty: 1, unitPrice: 9.99}]->(cable)
CREATE (book)-[:IN_CATEGORY]->(books), (cable)-[:IN_CATEGORY]->(elec)
CREATE (ada)-[:REVIEWED {stars: 5}]->(book);

// look at what you built
MATCH (c:Customer {id: 42})-[r]->(n) RETURN c, r, n;
CALL db.schema.visualization();
```

CODE EXPLAINED

- → Variables (`ada` , `book`) name nodes within one statement
- → The `order_items` table becomes `CONTAINS` relationships carrying `qty` and `unitPrice`
- → Categories become nodes, so "products in Books" is a short traversal
- → `db.schema.visualization()` draws labels and relationship types

SQL EQUIVALENT

- ⇄ Node ↔ row · label ↔ table · relationship ↔ a foreign key or junction row, stored as a pointer
- ⇄ Relationship properties ↔ columns on a junction table (`order_items.quantity`) · See SQL 62

ACROSS ENGINES

- **MongoDB** references plus `$graphLookup` for recursive walks
- **Redis** no graph engine in Redis Open Source 8
- **DynamoDB** adjacency-list pattern (sheet 46); Neptune is AWS's graph database
- **Cassandra** no traversals; copy relationships into query tables

IN YOUR CODE

- **Node** `const driver = neo4j.driver("neo4j://localhost:7687", neo4j.auth.basic("neo4j", pw))`
- **Python** `driver = GraphDatabase.driver("neo4j://localhost:7687", auth=("neo4j", pw))`
- **Java** `Driver driver = GraphDatabase.driver(uri, AuthTokens.basic("neo4j", pw));`

GRAPH VOCABULARY

TERM	MEANING
Node	An entity
Label	A node's type (`:Customer`); several allowed
Relationship	Typed, directed link (`:PLACED`)
Property	Key-value on node or relationship
Path	Nodes joined by relationships
Traversal	Following relationships

WAYS TO RUN NEO4J

OPTION	NOTES
Aura Free / Pro	Managed in the cloud
Neo4j Desktop	Local projects with a GUI
Docker neo4j	Community or Enterprise
Browser	Query and visualize
cypher-shell	Terminal client
Bloom	Visual exploration

⚠ COMMON MISTAKES

- Modeling everything as properties and nothing as relationships
- Generic relationship types like `:RELATED_TO`
- Running `CREATE` twice and getting duplicate nodes (use `MERGE`, sheet 53)

✓ TIPS & BEST PRACTICES

- Name relationships as verbs in UPPER_SNAKE_CASE
- Name labels in PascalCase, singular
- Sketch the graph on paper from your questions first

QUICK SUMMARY

- Nodes + relationships + properties
- Labels group nodes
- Traversals instead of joins
- Cypher from Browser or drivers

Cypher Basics

PATTERNS IN ASCII ART

Cypher describes graph **patterns** with ASCII art: `(n:Label)` is a node, `-[:TYPE]->` a relationship. **`MATCH`** finds patterns, **`WHERE`** filters, **`RETURN`** chooses output. **`CREATE`** adds data, **`SET`** / **`REMOVE`** change properties and labels, **`MERGE`** matches a whole pattern or creates it, and **`DELETE`** removes; **`DETACH DELETE`** also removes a node's relationships. Pass values as **parameters** (`$id`) rather than building strings.

NOTES

- `OPTIONAL MATCH` is the left outer join of Cypher
- `MERGE` matches the whole pattern; merge nodes first, then relationships
- Strings use single or double quotes
- Statements end with `;` in Browser multi-statement mode

READ, CREATE, UPDATE, MERGE, DELETE

```
// read
MATCH (p:Product) WHERE p.price < 20
RETURN p.name, p.price ORDER BY p.price LIMIT 10;
MATCH (c:Customer {id: 42})-[:PLACED]->(o:Order)
RETURN o.id, o.total ORDER BY o.date DESC;
MATCH (p:Product) WHERE p.name STARTS WITH "SQL" OR p.name CONTAINS "Cable"
RETURN p;
MATCH (p:Product {id: 7})
OPTIONAL MATCH (p)<-[r:REVIEWED]-(c:Customer)
RETURN p.name, c.name, r.stars;

// create and update
CREATE (:Product {id: 21, name: "Desk Mat", price: 24.50, stock: 40});
MATCH (p:Product {id: 7}) SET p.stock = p.stock - 2, p.updatedAt = datetime();
MATCH (p:Product {id: 21}) SET p:Clearance REMOVE p.promo;
MATCH (p:Product {id: 7}) SET p += {featured: true};

// MERGE: get-or-create
MERGE (c:Customer {id: 43})
  ON CREATE SET c.name = "Grace Hopper", c.createdAt = datetime()
  ON MATCH SET c.lastSeen = datetime();
MATCH (c:Customer {id: 43}), (p:Product {id: 7})
MERGE (c)-[v:VIEWED]->(p) SET v.at = datetime();

// delete
MATCH (p:Product {id: 21}) DETACH DELETE p;
MATCH ()-[v:VIEWED]->() WHERE v.at < datetime() - duration("P90D") DELETE v;
```

CODE EXPLAINED

- → `{id: 42}` inside a node pattern is shorthand for `WHERE c.id = 42`
- → `SET p:Clearance` adds a label; `REMOVE p.promo` drops a property
- → `SET p += {…}` merges a map into the properties without removing others
- → Plain `DELETE` on a node with relationships fails; `DETACH DELETE` removes them first

SQL EQUIVALENT

- ⇄ `MATCH` ↔ `SELECT … FROM … JOIN` · `CREATE` ↔ `INSERT` · `SET` ↔ `UPDATE` · `MERGE` ↔ upsert
- ⇄ `OPTIONAL MATCH` ↔ `LEFT JOIN` · `DETACH DELETE` ↔ `DELETE … CASCADE` · See SQL 13, 53

ACROSS ENGINES

- **MongoDB** `find` / `insertOne` / `updateOne` / `deleteOne`; `upsert: true` is like `MERGE`
- **Redis** commands per data type; no query language
- **DynamoDB** Get / Put / Update / Delete; PartiQL for SQL-like syntax
- **Cassandra** CQL looks like SQL but `WHERE` is limited to keys

IN YOUR CODE

- **Node** `await driver.executeQuery("MATCH (p:Product {id: $id}) RETURN p", { id: 7 })`
- **Python** `driver.execute_query("MATCH (p:Product {id: $id}) RETURN p", id=7)`
- **Java** `driver.executableQuery("MATCH (p:Product {id: $id}) RETURN p").withParameters(Map.of("id", 7)).execute()`

CLAUSES

CLAUSE	DOES
`MATCH / OPTIONAL MATCH`	Find patterns
`WHERE`	Filter
`RETURN`	Output (with `ORDER BY`, `SKIP`, `LIMIT`)
`CREATE`	Add nodes / relationships
`MERGE`	Match or create
`SET / REMOVE`	Change properties, labels
`DELETE / DETACH DELETE`	Remove

OPERATORS

OPERATOR	EXAMPLE
Comparison	=, <>, <, >=
Boolean	AND, OR, NOT, XOR
String	STARTS WITH, ENDS WITH, CONTAINS, =~
List	IN [1, 2]
Null	IS NULL, IS NOT NULL
Map merge	SET n += {…}

⚠ COMMON MISTAKES

- `MERGE` on a whole pattern when one end already exists, creating duplicate nodes
- Building Cypher strings from user input
- Forgetting relationship direction in `CREATE`

✓ TIPS & BEST PRACTICES

- Always use parameters (`$id`)
- `MERGE` nodes on constrained keys, then `MERGE` relationships
- Use `EXPLAIN` before running big writes

QUICK SUMMARY

- `MATCH` · `WHERE` · `RETURN`
- `CREATE` · `SET` · `REMOVE`
- `MERGE` — get-or-create
- `DETACH DELETE` — with relationships

Sheet 54

Paths, Patterns & Aggregation

variable-length *1..3 · shortestPath · quantified path patterns · count · collect · WITH

MULTI-HOP QUESTIONS

Patterns can chain many hops: "customers who bought what Ada bought" is one MATCH. **Variable-length** relationships (*1..3) and **quantified path patterns** ({1,3}) follow a relationship a range of times, and **SHORTEST** finds the shortest paths. Aggregation is implicit: any non-aggregated value in RETURN or WITH is a grouping key. **WITH** pipes results to the next part of the query, which also lets you filter after aggregating.

NOTES

- Always bound variable-length patterns (`*1..3` , not `*`)
- `collect()` builds a list; `UNWIND` turns a list into rows
- Quantified path patterns arrived in Neo4j 5.9
- `DISTINCT` removes repeated rows from multiple paths

HOPS, PATHS & AGGREGATION

```
// customers who bought what Ada bought
MATCH (:Customer {id: 42})-[:PLACED]->(:Order)-[:CONTAINS]->(p:Product)
      <-[:CONTAINS]-(:Order)<-[:PLACED]-(other:Customer)
WHERE other.id <> 42
RETURN other.name, count(DISTINCT p) AS shared
ORDER BY shared DESC LIMIT 5;

// variable length: 1 to 3 hops
MATCH (a:Customer {id: 42})-[:FOLLOWS*1..3]->(b:Customer)
RETURN DISTINCT b.name;

// quantified path pattern
MATCH (a:Customer {id: 42}) ((x)-[:FOLLOWS]->(y)){1,3} (b:Customer)
RETURN DISTINCT b.name;

// shortest path
MATCH p = SHORTEST 1 (a:Customer {id: 42})-[:FOLLOWS]-+(b:Customer {id: 99})
RETURN length(p), [n IN nodes(p) | n.name];

// grouping: non-aggregated columns are the keys
MATCH (c:Customer)-[:PLACED]->(o:Order)
RETURN c.name, count(o) AS orders, sum(o.total) AS revenue,
       collect(o.id)[..5] AS firstIds;

// WITH: aggregate, then filter (like HAVING)
MATCH (p:Product)<-[r:CONTAINS]-(:Order)
WITH p, sum(r.qty) AS units
WHERE units > 100
RETURN p.name, units ORDER BY units DESC;
```

CODE EXPLAINED

- → The pattern walks out from Ada to products and back to other customers in one statement
- → `*1..3` and `{1,3}` both mean one to three hops; the quantified form can add conditions inside the group
- → `-+` means one or more hops; `SHORTEST 1` returns one shortest path
- → `[n IN nodes(p) | n.name]` is a list comprehension

SQL EQUIVALENT

- ⇄ Multi-hop patterns ↔ chains of self joins, or a recursive CTE for variable depth
- ⇄ `collect()` ↔ `array_agg` · `WITH … WHERE` ↔ `HAVING` · See SQL 42, 29

ACROSS ENGINES

- **MongoDB** `$graphLookup` with `maxDepth` for recursive lookups
- **Redis** none; walk sets in the app
- **DynamoDB** adjacency lists: one hop per Query
- **Cassandra** precompute relationships into tables

IN YOUR CODE

- **Node** `const { records } = await driver.executeQuery(cypher, { id: 42 }); records.map(r => r.get("shared"))`
- **Python** `records, summary, keys = driver.execute_query(cypher, id=42)`
- **Java** `var result = driver.executableQuery(cypher).withParameters(Map.of( "id", 42)).execute(); result.records()`

PATH SYNTAX

SYNTAX	MEANS
`-[:T]->`	One hop, directed
`-[:T]-`	Either direction
`-[:T*1..3]->`	1 to 3 hops
`((a)-[:T]->(b)){1,3}`	Quantified path pattern
`-[:T]-+`	One or more hops
`SHORTEST k`	k shortest paths
`shortestPath((a)-[*]-(b))`	Older shortest-path function

AGGREGATING FUNCTIONS

FUNCTION	RESULT
`count(x) / count(*)`	Count
`sum / avg`	Total / mean
`min / max`	Extremes
`collect(x)`	List of values
`percentileCont`	Percentile
`stDev`	Standard deviation

⚠ COMMON MISTAKES

- Unbounded `*` patterns that explore the whole graph
- Counting paths when you meant distinct nodes
- Expecting `GROUP BY` ; grouping comes from the non-aggregated columns

✓ TIPS & BEST PRACTICES

- Start traversals from an indexed node
- Use `count(DISTINCT x)` across multiple paths
- Break long queries into `WITH` steps

QUICK SUMMARY

- Multi-hop patterns in one `MATCH`
- `*1..3` / `{1,3}` — variable length
- `SHORTEST` — shortest paths
- Implicit grouping + `WITH`

Graph Modeling, Indexes & Constraints

relationship vs property · intermediate nodes · `CREATE INDEX` · uniqueness constraint · `PROFILE` · recommendations

MODEL THE QUESTIONS, INDEX THE ENTRY POINTS

Make something a **node** when you query from it or connect it to many things; make it a **property** when it just describes something. Turn events into **intermediate nodes** (`(:Customer)-[:PLACED]->(:Order)-[:CONTAINS]->(:Product)` rather than one `:BOUGHT` relationship) so they can hold data and links. Traversals start from a node you look up, so **index** and **constrain** those lookup properties, then check plans with **`PROFILE`**.

NOTES

- A uniqueness constraint also creates an index
- Node key and type constraints are Enterprise features
- Specific relationship types are faster than filtering on properties
- Check plans: NodeIndexSeek good, NodeByLabelScan to fix

PRODUCTION CHECK

- A `MATCH` on an unindexed property scans every node with that label; index your lookup properties
- `MERGE` without a uniqueness constraint can create duplicates under concurrent writes
- Super nodes with millions of relationships slow traversals; split them with intermediate nodes or more specific relationship types

CONSTRAINTS, INDEXES, PLANS & RECOMMENDATIONS

```
// constraints (each also creates an index)
CREATE CONSTRAINT customer_id IF NOT EXISTS
  FOR (c:Customer) REQUIRE c.id IS UNIQUE;
CREATE CONSTRAINT product_id IF NOT EXISTS
  FOR (p:Product) REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT order_key IF NOT EXISTS
  FOR (o:Order) REQUIRE o.id IS NODE KEY;            // Enterprise

// indexes
CREATE INDEX product_category IF NOT EXISTS FOR (p:Product) ON (p.category);
CREATE TEXT INDEX product_name IF NOT EXISTS FOR (p:Product) ON (p.name);
CREATE FULLTEXT INDEX product_search IF NOT EXISTS
  FOR (p:Product) ON EACH [p.name, p.description];
CREATE VECTOR INDEX product_embedding IF NOT EXISTS
  FOR (p:Product) ON (p.embedding)
  OPTIONS { indexConfig: { `vector.dimensions`: 384,
                           `vector.similarity_function`: 'cosine' } };
SHOW INDEXES;
SHOW CONSTRAINTS;

// check the plan
PROFILE MATCH (p:Product {category: "Books"}) RETURN p.name;
// NodeIndexSeek = uses the index · NodeByLabelScan = add one

// recommendations: bought together with product 7
MATCH (:Product {id: 7})<-[:CONTAINS]-(:Order)-[:CONTAINS]->(rec:Product)
RETURN rec.name, count(*) AS together
ORDER BY together DESC LIMIT 5;
```

CODE EXPLAINED

- → Uniqueness constraints make `MERGE` on `id` safe and fast
- → TEXT indexes speed up `CONTAINS` / `ENDS WITH` on strings
- → The full-text index is queried with `db.index.fulltext.queryNodes`
- → Orders as intermediate nodes make "bought together" a two-hop pattern

SQL EQUIVALENT

- ⇄ Uniqueness ↔ `UNIQUE` · node key ↔ `PRIMARY KEY` · property existence ↔ `NOT NULL`
- ⇄ `PROFILE` ↔ `EXPLAIN ANALYZE`; intermediate node ↔ a junction table with its own columns · See SQL 57, 73

ACROSS ENGINES

- **MongoDB** unique indexes; `explain()`
- **Redis** no constraints; `SET NX` for uniqueness
- **DynamoDB** uniqueness via a conditional put on a key item
- **Cassandra** uniqueness via `IF NOT EXISTS` on a lookup table

IN YOUR CODE

- **Node** plain driver with `MERGE` on constrained keys (or an OGM such as Neogma)
- **Python** neomodel: `class Product(StructuredNode): pid = IntegerProperty(unique_index=True)`
- **Java** Spring Data Neo4j: `@Node class Product { @Id Long id; @Relationship("IN_CATEGORY") Category category; }`

INDEX TYPES

TYPE	SPEEDS UP
Range	=, ranges, STARTS WITH, sorting
Text	CONTAINS, ENDS WITH
Point	Distance and bounding box
Full-text	Lucene word search
Vector	Similarity search on embeddings
Lookup	Labels / types (built in)

MODELING RULES

QUESTION	CHOOSE
Do you start queries from it?	Node
Does it connect many things?	Node
Just describes one thing?	Property
An event with data?	Intermediate node
A fixed category of nodes?	Label

⚠ COMMON MISTAKES

- No constraint on the property every `MERGE` uses
- Turning every attribute into a node
- One giant node (`:Country {name: "US"}`) linked to everything

✓ TIPS & BEST PRACTICES

- Create constraints before loading data
- Profile your top five queries
- Use specific relationship types: `:PLACED`, `:REVIEWED`, `:VIEWED`

QUICK SUMMARY

- Index the entry points
- Constraints make `MERGE` safe
- Intermediate nodes for events
- `PROFILE` every hot query

Sheet 56

Search Engines: Elasticsearch & OpenSearch

inverted index · mappings · analyzers · `match` / `bool` query · relevance scoring · aggregations

RELEVANCE IS THE PRODUCT

Elasticsearch and its Apache-licensed fork **OpenSearch** are document stores built around an **inverted index** (word → documents). **Analyzers** break text into normalized tokens, **mappings** set field types, and queries are scored for **relevance** (BM25 by default). A `bool` query mixes scored `must` clauses with unscored `filter` clauses, and **aggregations** compute facets in the same request. They're usually a search copy of data whose source of truth lives elsewhere.

NOTES

- `text` fields are analyzed; `keyword` fields match exactly
- Changing a field's mapping means reindexing
- Elasticsearch offers AGPLv3 among its licences; OpenSearch is Apache 2.0
- Both also do vector search (kNN)

MAPPING, INDEXING & A BOOL QUERY

```
PUT /products
{ "mappings": { "properties": {
    "name":     { "type": "text", "analyzer": "english" },
    "category": { "type": "keyword" },
    "price":    { "type": "scaled_float", "scaling_factor": 100 },
    "stock":    { "type": "integer" } } } }

PUT /products/_doc/7
{ "name": "SQL Pocket Guide", "category": "Books", "price": 19.99, "stock": 10 }

GET /products/_search
{ "query": { "bool": {
    "must":   [ { "match": { "name": { "query": "pocket gide",
                                       "fuzziness": "AUTO" } } } ],
    "filter": [ { "term":  { "category": "Books" } },
                { "range": { "price": { "lte": 30 } } } ] } },
  "aggs": { "by_category": { "terms": { "field": "category" } },
            "avg_price":   { "avg":   { "field": "price" } } },
  "size": 10 }
```

CODE EXPLAINED

→ The `english` analyzer lowercases, removes stop words and stems ("guides" → "guid")

→ `fuzziness: AUTO` lets "gide" match "guide"

→ `filter` clauses don't affect the score and are cached

→ `aggs` return facet counts alongside the hits

SQL EQUIVALENT

⇄ Inverted index ↔ a GIN index on `tsvector` · `match` ↔ `@@ to_tsquery` · `filter` ↔ `WHERE`

⇄ `aggs` ↔ `GROUP BY`; relevance ranking ↔ `ts_rank`, much simpler · See SQL 80

ACROSS ENGINES

- **MongoDB** Atlas Search runs Lucene beside the database (sheet 24)
- **Redis** Query Engine `FT.SEARCH`, in memory (sheet 40)
- **DynamoDB** zero-ETL integration feeds OpenSearch Service
- **Cassandra** SAI handles filters; no relevance scoring
- **Neo4j** full-text indexes built on Lucene

IN YOUR CODE

- **Node** `await es.search({ index: "products", query: { match: { name: "pocket guide" } } })`
- **Python** `es.search(index="products", query={"match": {"name": "pocket guide"}})`
- **Java** `client.search(s -> s.index("products").query(q -> q.match(m -> m.field("name").query("pocket guide"))), Product.class)`

QUERY TYPES

QUERY	USE FOR
`match`	Analyzed full-text
`multi_match`	Several fields, with boosts
`term / terms`	Exact keyword values
`range`	Numbers and dates
`bool`	Combine must / should / filter / must_not
`knn`	Vector similarity

FIELD TYPES

TYPE	BEHAVIOR
`text`	Analyzed, scored
`keyword`	Exact, sortable, aggregatable
`integer / scaled_float`	Numbers
`date`	Dates and times
`dense_vector`	Embeddings

⚠ COMMON MISTAKES

- Mapping categories as `text` and failing to aggregate them
- Relying on dynamic mapping in production
- Using the search engine as the only copy of the data

✓ TIPS & BEST PRACTICES

- Define mappings explicitly
- Put non-scoring conditions in `filter`
- Reindex through an alias so you can swap indexes with no downtime

QUICK SUMMARY

- Inverted index + analyzers
- `text` vs `keyword`
- `bool`: must + filter
- A search copy, not the source

Sheet 57

Vector Databases & Embeddings

embeddings · similarity (cosine, dot) · ANN / HNSW · metadata filters · pgvector · Atlas / Redis / dedicated stores

SEARCH BY MEANING

An **embedding** model turns text, images or audio into a **vector** of numbers, placed so that similar meanings land close together. A vector store finds the **nearest neighbors** of a query vector using **approximate nearest neighbor (ANN)** indexes such as **HNSW**, usually combined with **metadata filters**. This powers semantic search, recommendations and retrieval-augmented generation (RAG). You may not need a new database: PostgreSQL (pgvector), MongoDB, Redis, Cassandra and Neo4j all store vectors.

EMBED, STORE, QUERY (PGVECTOR)

```
# 1. turn text into vectors (any embedding model)
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")              # 384 dimensions
docs = [{"id": 7, "text": "SQL Pocket Guide: a quick SQL reference"},
        {"id": 3, "text": "USB-C cable, 2 m, fast charging"}]
vecs = model.encode([d["text"] for d in docs], normalize_embeddings=True)

# 2. store them (pgvector here; same idea in every store)
import psycopg
from pgvector.psycopg import register_vector
conn = psycopg.connect("dbname=shop", autocommit=True)
conn.execute("CREATE EXTENSION IF NOT EXISTS vector")
register_vector(conn)
conn.execute("CREATE TABLE IF NOT EXISTS items "
             "(id int PRIMARY KEY, body text, embedding vector(384))")
conn.execute("CREATE INDEX IF NOT EXISTS items_embedding_idx ON items "
             "USING hnsw (embedding vector_cosine_ops)")
for d, v in zip(docs, vecs):
    conn.execute("INSERT INTO items VALUES (%s, %s, %s)", (d["id"], d["text"], v))

# 3. query: embed the question, take the nearest neighbors
q = model.encode("cable to charge my laptop", normalize_embeddings=True)
rows = conn.execute("SELECT id, body, 1 - (embedding <=> %s) AS score "
                    "FROM items ORDER BY embedding <=> %s LIMIT 5", (q, q)).fetchall()
```

NOTES

- Use the same model for documents and queries
- Normalize vectors to use dot product as cosine
- More dimensions cost more memory and time
- Store the model name and version with each vector

CODE EXPLAINED

- → `normalize_embeddings=True` scales vectors to length 1
- → `register_vector` lets psycopg send NumPy arrays as `vector` values
- → The HNSW index makes nearest-neighbor search approximate but fast
- → `<=>` is cosine distance, so `1 - distance` is similarity

SQL EQUIVALENT

- ⇄ pgvector keeps vectors in SQL: `ORDER BY embedding <=> $1 LIMIT 5` with an HNSW index
- ⇄ Metadata filters are ordinary `WHERE` clauses · See SQL 72

ACROSS ENGINES

- **MongoDB** `$vectorSearch` with pre-filters (sheet 24)
- **Redis** VECTOR fields in `FT.CREATE`; Redis 8 also adds vector sets
- **DynamoDB** no native vector index; pair it with OpenSearch
- **Cassandra** `vector<float, 384>` columns with SAI and `ORDER BY … ANN OF`
- **Neo4j** vector indexes, combined with graph traversals

IN YOUR CODE

- **Node** `await coll.aggregate([{ $vectorSearch: { index, path: "embedding", queryVector: q, numCandidates: 100, limit: 5 } }]).toArray()`
- **Python** `conn.execute("SELECT id FROM items ORDER BY embedding <=> %s LIMIT 5", (q,))`
- **Java** Spring AI: `vectorStore.similaritySearch(SearchRequest.builder().query("cable").topK(5).build())`

SIMILARITY METRICS

METRIC	NOTES
`Cosine`	Angle only; the usual choice for text
`Dot product`	Same as cosine when normalized; fastest
`Euclidean (L2)`	Straight-line distance
`Hamming`	Binary vectors

INDEX ALGORITHMS

ALGORITHM	TRADE-OFF
`Flat (exact)`	Perfect recall, slow at scale
`HNSW`	Fast and accurate; uses memory
`IVF`	Clusters vectors; less memory
`DiskANN`	Large sets on SSD
`Quantization`	Smaller vectors, a little less accuracy

⚠ COMMON MISTAKES

- Mixing vectors from different models in one index
- No metadata filters, so results ignore stock or permissions
- Judging quality without a set of test questions

✓ TIPS & BEST PRACTICES

- Start with the database you already run
- Combine vector and keyword search (hybrid) for better results
- Re-embed everything when you change models

QUICK SUMMARY

- Embeddings place meaning in space
- ANN indexes (HNSW) find neighbors
- Filter by metadata
- Often no new database needed

Sheet 58

Polyglot Persistence

SQL + cache · SQL + search · CQRS read models · outbox / CDC · dual-write pitfalls · consistency between stores

MORE THAN ONE STORE

Polyglot persistence means using different stores for different jobs: SQL or MongoDB as the source of truth, Redis as a cache, a search engine for text, a graph for recommendations. The hard part is keeping copies in step. Writing to two stores in one request (**dual write**) drifts when one write fails. Instead, record changes once and propagate them: the **outbox** pattern or **change data capture (CDC)** feeds events to consumers that update **read models** (CQRS).

NOTES

- Copies are eventually consistent by design
- Consumers must be idempotent: events can repeat
- Debezium reads change logs from PostgreSQL, MySQL, MongoDB and more
- Each extra store adds operations work

PRODUCTION CHECK

- Dual writes leave stores out of sync when the second write fails; use an outbox or CDC
- Name one source of truth for each piece of data; every other store is a rebuildable copy
- Plan for rebuilds: you should be able to reindex search or refill a cache from the source

OUTBOX, RELAY & CONSUMERS

```
// 1. change + event in ONE database transaction (outbox)
await sql.begin(async tx => {
  await tx`UPDATE products SET price = ${price} WHERE product_id = ${id}`
  await tx`INSERT INTO outbox (aggregate, aggregate_id, type, payload)
           VALUES ('product', ${id}, 'PriceChanged',
                   ${JSON.stringify({ id, price })})`
})

// 2. relay (or Debezium CDC) publishes outbox rows, then marks them sent
const batch = await sql`SELECT * FROM outbox WHERE sent_at IS NULL ORDER BY id LIMIT 100`
for (const evt of batch) {
  await producer.send({ topic: "product-events",
    messages: [ { key: String(evt.aggregate_id), value: evt.payload } ] })
  await sql`UPDATE outbox SET sent_at = now() WHERE id = ${evt.id}`
}

// 3. consumers update their own stores (safe to run twice)
on("PriceChanged", async ({ id, price }) => {
  await redis.del(`product:${id}`)                                  // cache
  await es.update({ index: "products", id: String(id), doc: { price } })  // search
  await mongo.collection("productViews")
    .updateOne({ _id: id }, { $set: { price } }, { upsert: true })   // read model
})
```

CODE EXPLAINED

- → The price change and its event commit together, or neither does
- → If the relay crashes after sending but before marking, the event is sent again; consumers must tolerate that
- → Keying messages by product id keeps one product's events in order
- → Each consumer owns its store and can be rebuilt by replaying events

SQL EQUIVALENT

- ⇄ The outbox is an ordinary SQL table written in the same transaction
- ⇄ CDC reads PostgreSQL's write-ahead log through logical decoding · See SQL 68, 67

ACROSS ENGINES

- **MongoDB** a source of truth or a read model; change streams act as CDC
- **Redis** cache and session store in front of the source of truth
- **DynamoDB** often the source of truth; Streams feed other stores
- **Cassandra** high-volume events or time series beside a SQL core
- **Neo4j** recommendation and relationship queries on a copy synced from SQL

IN YOUR CODE

- **Node** postgres.js + kafkajs + node-redis, or a CDC connector instead of a relay
- **Python** SQLAlchemy outbox table + Debezium; consumers with confluent-kafka
- **Java** Spring Modulith event publication registry, or Debezium Embedded

INTEGRATION PATTERNS

PATTERN	USE
Cache-aside	Speed up reads (sheet 39)
CQRS read model	Query-shaped copies
Outbox	Reliable events from a database
CDC	Events from the change log
Saga	Multi-step workflows across services
Dual write	Avoid

TYPICAL ROLES

NEED	STORE
Source of truth, joins	PostgreSQL / MySQL
Flexible documents	MongoDB
Hot reads, sessions	Redis
Text relevance	Elasticsearch / OpenSearch
Relationships	Neo4j
Huge event volume	Cassandra

⚠ COMMON MISTAKES

- Writing to the database and the cache / search index in the same request
- No way to rebuild a copy from the source
- Consumers that break when an event arrives twice

✓ TIPS & BEST PRACTICES

- Add stores only for a measured need
- Make every consumer idempotent with upserts and version checks
- Monitor lag between the source and each copy

QUICK SUMMARY

- Different stores, different jobs
- One source of truth
- Outbox or CDC, never dual writes
- Idempotent consumers

Testing & Schema Migration

Testcontainers · DynamoDB Local · in-memory fakes · seed data · schema versioning · lazy vs batch migration

REAL ENGINES IN TESTS, GRADUAL SCHEMA CHANGE

Test against the real engine: **Testcontainers** starts MongoDB, Redis, Cassandra or Neo4j in Docker for a test run, and **DynamoDB Local** or LocalStack stands in for AWS. Seed known data before each test and clean it after. Schemas still change without ALTER TABLE: **lazy migration** upgrades documents when they're read, **batch migration** rewrites them in the background. Tag documents with a `schemaVersion` so code knows what it's looking at.

NOTES

- In-memory fakes are fast but miss real query behavior
- Reuse containers across a test run; Cassandra starts slowly
- Keep migration scripts in version control
- Code must read both old and new shapes during a migration

TESTCONTAINERS & MIGRATIONS

```
// Testcontainers: a real MongoDB for the test run
import { MongoDBContainer } from "@testcontainers/mongodb"
import { MongoClient } from "mongodb"

let mongo, client
beforeAll(async () => {
  mongo = await new MongoDBContainer("mongo:8").start()
  client = new MongoClient(mongo.getConnectionString(), { directConnection: true })
  await client.connect()
  await client.db("shop").collection("products").insertMany(seedProducts)
}, 60_000)
afterAll(async () => { await client.close(); await mongo.stop() })

test("reserves stock atomically", async () => {
  const products = client.db("shop").collection("products")
  const r = await products.updateOne({ _id: 7, stock: { $gte: 2 } },
                                     { $inc: { stock: -2 } })
  expect(r.modifiedCount).toBe(1)
})

// lazy migration: upgrade when read
function upgrade(doc) {
  if ((doc.schemaVersion ?? 1) < 2) {
    const [first, ...rest] = doc.name.split(" ")
    return { ...doc, name: { first, last: rest.join(" ") }, schemaVersion: 2 }
  }
  return doc
}

// batch migration (mongosh): rewrite old documents in place
db.customers.updateMany({ schemaVersion: { $exists: false } }, [
  { $set: { name: { first: { $arrayElemAt: [ { $split: ["$name", " "] }, 0 ] },
                    last:  { $arrayElemAt: [ { $split: ["$name", " "] }, 1 ] } },
            schemaVersion: 2 } } ])
```

CODE EXPLAINED

- → The container is a single-node replica set, so transactions and change streams work in tests
- → `directConnection: true` talks to that one node directly
- → Lazy migration spreads the work but leaves old shapes until each document is read
- → The batch migration uses an update pipeline so it can read `name` while rewriting it

SQL EQUIVALENT

- ⇄ Migration tools like Flyway and Liquibase ↔ migrate-mongo, Mongock, Neo4j-Migrations
- ⇄ `ALTER TABLE` happens at once; NoSQL migrations are usually gradual · See SQL 59

ACROSS ENGINES

- **MongoDB** Testcontainers or `mongodb-memory-server`; migrate-mongo / Mongock
- **Redis** Testcontainers; `FLUSHDB` between tests
- **DynamoDB** DynamoDB Local or LocalStack; create tables in test setup
- **Cassandra** Testcontainers with a reused container; cassandra-migration
- **Neo4j** Testcontainers or Neo4j Harness; Neo4j-Migrations

IN YOUR CODE

- **Node** `@testcontainers/mongodb`, `@testcontainers/redis`
- **Python** `with MongoDbContainer("mongo:8") as m: client = m.get_connection_client()`
- **Java** `@Container static MongoDBContainer mongo = new MongoDBContainer("mongo:8");`

TEST OPTIONS

OPTION	TRADE-OFF
Testcontainers	Real engine; needs Docker
Emulator (DynamoDB Local)	Close to real; some gaps
In-memory server	Fast; may differ in edge cases
Mocks of the driver	Fastest; tests little

MIGRATION STRATEGIES

STRATEGY	HOW
Lazy	Upgrade on read, save on next write
Batch	Background job rewrites documents
Dual-shape code	Read old and new until done
Expand / contract	Add new field, migrate, remove old

⚠ COMMON MISTAKES

- Mocking the driver and missing real query bugs
- Tests that depend on leftover data from earlier tests
- Removing support for the old shape before every document is migrated

✓ TIPS & BEST PRACTICES

- Seed each test with only the data it needs
- Run batch migrations in small, resumable chunks
- Count documents per `schemaVersion` to track progress

QUICK SUMMARY

- Testcontainers — real engines
- Seed and clean per test
- Lazy or batch migration
- `schemaVersion` on documents

Sheet 60

Quick Reference — MongoDB & Redis

mongosh CRUD · query / update operators · aggregation stages · index types · Redis commands by type · TTL

MONGOSH CRUD

COMMAND	DOES
`insertOne / insertMany`	Add documents
`find(filter, proj)`	Query; returns a cursor
`findOne`	First match
`updateOne / updateMany`	Change with operators
`replaceOne`	Swap whole document
`deleteOne / deleteMany`	Remove
`findOneAndUpdate`	Change and return
`bulkWrite`	Many ops, one trip
`countDocuments`	Count matches

QUERY OPERATORS

OPERATOR	MATCHES
`$eq $ne $gt $gte $lt $lte`	Comparisons
`$in / $nin`	In / not in list
`$and $or $nor $not`	Logic
`$exists / $type`	Presence / BSON type
`$regex`	Pattern
`$elemMatch`	One array element, all conditions
`$all / $size`	Contains all / array length
`$expr`	Aggregation expression

UPDATE OPERATORS

OPERATOR	DOES
`$set / $unset`	Write / remove field
`$inc / $mul`	Math
`$min / $max`	Conditional write
`$push / $addToSet`	Append / append unique
`$pull / $pop`	Remove from array
`$ $[] $[id]`	Positional
`$setOnInsert`	Only on upsert insert
`$currentDate`	Now

AGGREGATION STAGES

STAGE	DOES
`$match`	Filter
`$project / $set / $unset`	Shape
`$group`	Group + accumulate
`$sort / $limit / $skip`	Order, trim
`$lookup / $unwind`	Join / flatten arrays
`$facet / $bucket`	Multi-result / ranges
`$setWindowFields`	Window functions
`$out / $merge`	Write results
`$search / $vectorSearch`	Atlas text / vector

INDEX TYPES & OPTIONS

INDEX	NOTES
`{ a: 1 }`	Single field
`{ a: 1, b: -1 }`	Compound (ESR order)
`Array field`	Multikey
`{ a: "hashed" }`	Hashed (sharding)
`{ "$**": 1 }`	Wildcard
`unique: true`	No duplicates
`partialFilterExpression`	Subset of docs
`expireAfterSeconds`	TTL
`.explain("executionStats")`	Check IXSCAN vs COLLSCAN

REDIS KEYS & STRINGS

COMMAND	DOES
`SET k v NX EX 30`	Write if missing, 30 s TTL
`GET / MGET`	Read
`INCR / INCRBY`	Atomic counter
`DEL / UNLINK`	Delete
`EXISTS / TYPE`	Inspect
`SCAN 0 MATCH p:*`	Iterate keys
`EXPIRE / TTL`	Set / read TTL
`PERSIST`	Remove TTL
`maxmemory-policy`	Eviction rule

REDIS HASHES, LISTS & SETS

COMMAND	DOES
`HSET / HGET / HGETALL`	Hash fields
`HINCRBY`	Atomic field math
`HEXPIRE`	Field TTL (7.4+)
`LPUSH / RPUSH / LPOP`	List ends
`LRANGE / LTRIM`	Read / cap list
`BLPOP / LMOVE`	Worker queues
`SADD / SISMEMBER`	Set membership
`SINTER / SUNION / SDIFF`	Combine sets

REDIS SORTED SETS, STREAMS & MORE

COMMAND	DOES
`ZADD / ZINCRBY`	Score members
`ZRANGE … REV WITHSCORES`	Top N
`ZRANGE … BYSCORE`	Score range
`XADD / XREAD`	Append / read stream
`XREADGROUP / XACK`	Consumer groups
`PUBLISH / SUBSCRIBE`	Pub/Sub
`MULTI / EXEC / WATCH`	Transactions
`EVAL / FCALL`	Lua / functions
`JSON.SET / FT.SEARCH`	JSON / search

Sheet 61

Quick Reference — DynamoDB, Cassandra, Cypher & SQL Map

DynamoDB API · expressions · CQL · consistency levels · Cypher clauses · SQL ↔ NoSQL concept map

DYNAMODB API

OPERATION	DOES
PutItem	Create or replace
GetItem	Read by full key
UpdateItem	Change attributes (upsert)
DeleteItem	Remove
Query	One partition, sort-key condition
Scan	Whole table
BatchGetItem / BatchWriteItem	100 reads / 25 writes
TransactWriteItems	Up to 100 actions, atomic
ExecuteStatement	PartiQL

DYNAMODB EXPRESSIONS

EXPRESSION	EXAMPLE
Key condition	PK = :pk AND begins_with(SK, :p)
Filter	#s = :status (after read)
Projection	#n, price
Update	SET a = :v ADD n :one REMOVE b
Condition	attribute_not_exists(PK)
Names / values	#name / :value
Functions	if_not_exists, list_append, size, contains

DYNAMODB KEYS & INDEXES

FEATURE	RULE
Partition key	Spreads data; use high cardinality
Sort key	Orders items; ranges and prefixes
GSI	Any key, any time, eventual
LSI	Same PK, at create, strong reads
Sparse index	Only items with the key
TTL	Epoch-seconds attribute
Streams	24 h change log

CQL

STATEMENT	NOTES
CREATE KEYSPACE … replication	Strategy + RF per DC
PRIMARY KEY ((pk), ck)	Partition + clustering
WITH CLUSTERING ORDER BY	Sort within partition
INSERT / UPDATE	Both upsert
USING TTL n	Expire after n s
IF NOT EXISTS / IF col = v	Lightweight transaction
BEGIN BATCH … APPLY BATCH	Logged batch
CREATE INDEX … USING 'sai'	SAI index (5.0)

CONSISTENCY LEVELS

LEVEL	REPLICAS
ONE / TWO / THREE	That many
QUORUM	Majority, all DCs
LOCAL_QUORUM	Majority, local DC
EACH_QUORUM	Majority in each DC
ALL	Every replica
SERIAL / LOCAL_SERIAL	LWT reads
Rule	R + W > RF → strong

CYPHER CLAUSES

CLAUSE	DOES
MATCH (a:L)-[:T]->(b)	Find pattern
OPTIONAL MATCH	Left-join style
WHERE	Filter
RETURN … ORDER BY … LIMIT	Output
WITH	Pipe and aggregate
CREATE / MERGE	Add / get-or-create
SET / REMOVE	Change
DETACH DELETE	Remove with relationships
UNWIND	List → rows

SQL ↔ NOSQL CONCEPT MAP

SQL	MONGODB	REDIS	DYNAMODB	CASSANDRA	NEO4J
Table	Collection	Key prefix	Table	Table	Label
Row	Document	Hash / value	Item	Row	Node
Primary key	_id	Key	PK (+ SK)	PRIMARY KEY	Constraint
JOIN	$lookup / embed	App-side	Item collection	Copy per query	Traversal
WHERE	Filter document	Key name	Key condition	Key columns	WHERE
GROUP BY	$group	Counters	Precompute	Partition GROUP BY	Implicit
Index	createIndex	FT.CREATE	GSI / LSI	SAI	CREATE INDEX
Transaction	withTransaction	MULTI / EXEC	TransactWriteItems	LWT	Always